

# Machine Learning

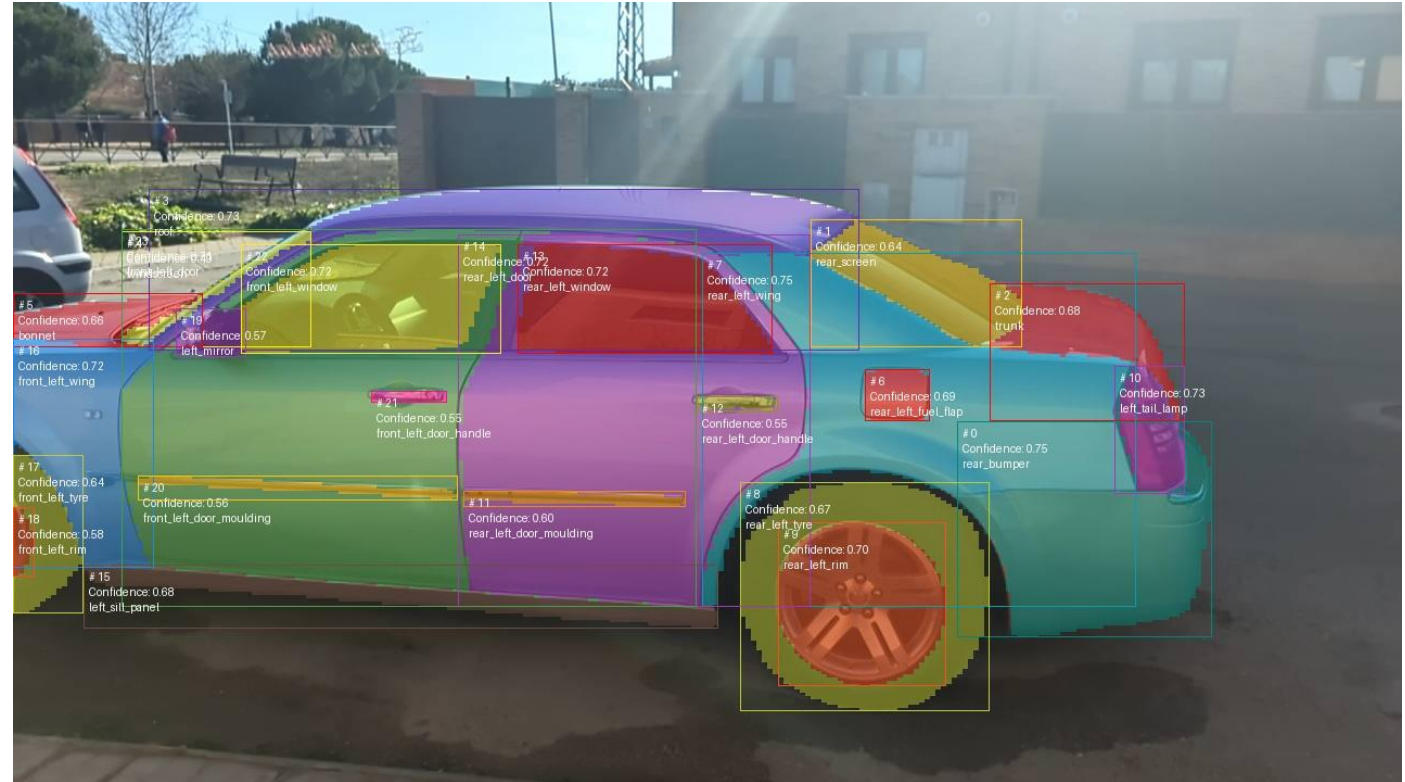
---

Máster en Innovación Tecnológica (MINT)

*imanolgranada@gmail.com*

# Who is your teacher?

- Imanol Granada  
([imanolgranada@gmail.com](mailto:imanolgranada@gmail.com))
- BsC in Telecom. Engineering@2015
- MsC in Telecom. Engineering@2017
- PhD in Telecom. Engineering@2021
- Data Scientist @ Bdeo.
- My interests?
  - Machine Learning
  - Deep Learning





19c6d06c-3673-5eaf-9732-0dc0e66dcff8 — 7 identities across 8 photos



0. CCPDW9RIHJqkuZNDn6R4LK (2 inst)

1. X\_yBBQLS1so3JA5Ti24jrf (0 inst)

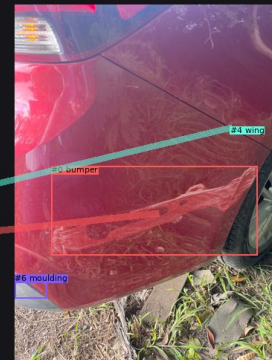
2. aVg\_r3i4Msbcd1rVURsWq0 (0 inst)



3. fo-\_b6Aw6gYM7u0nzjRD34 (1 inst)

4. j2SMXIYr5TuCVrizwWmBzr (0 inst)

5. kIkP\_NULXLKbCgIDUv5-7y (1 inst)



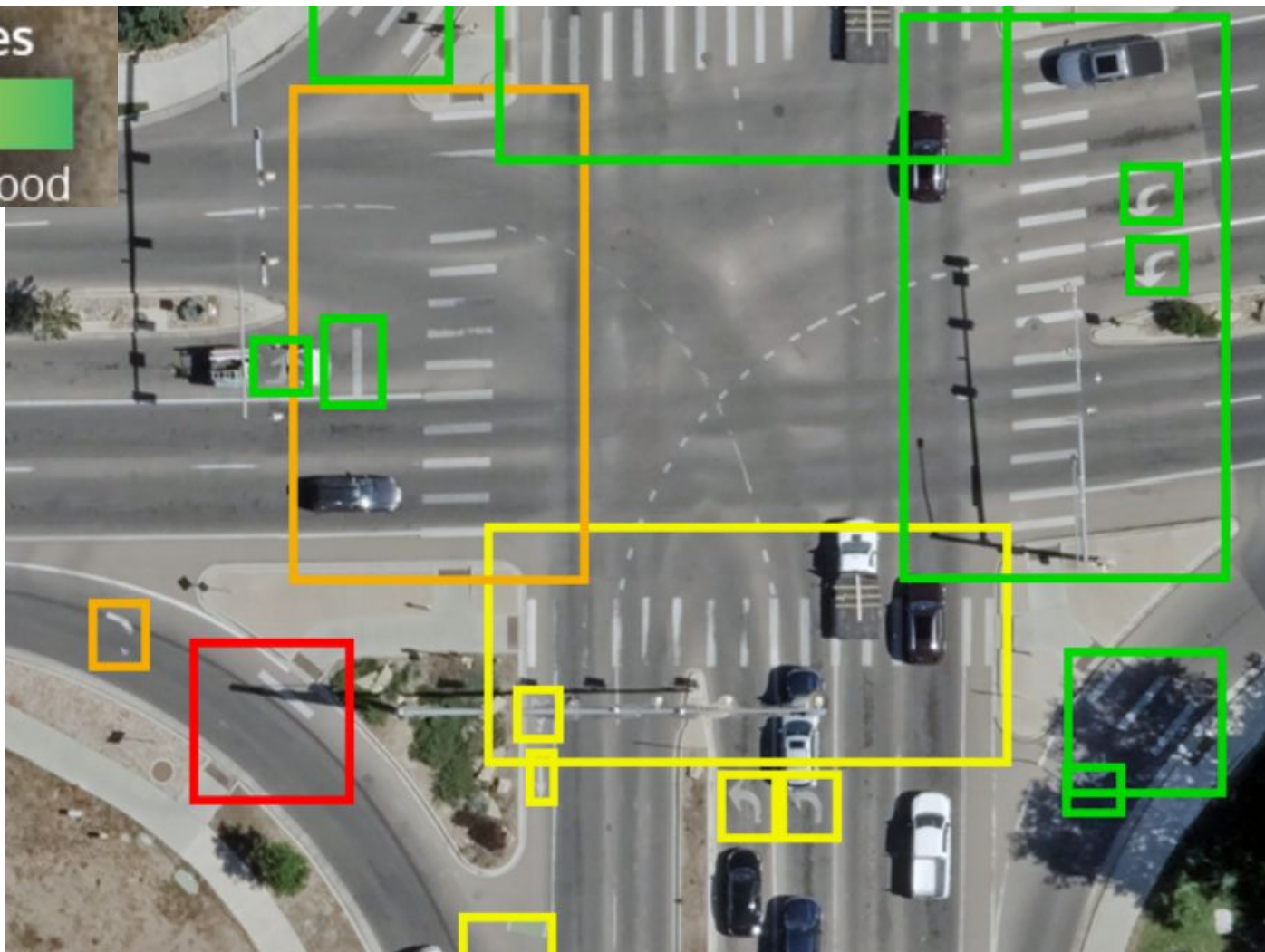
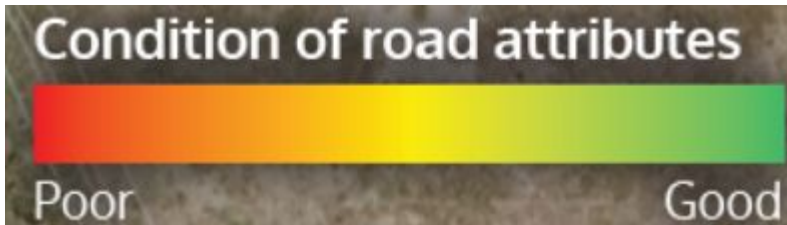
6. qOJU2lrzk14\_IQoEmNo3Hu (3 inst)

7. rwC\_dbXHCsu3vi3Pc2gvE8 (3 inst)

#0 bumper/bumper · 3/8 photos  
#2 sill\_panel/sill\_panel · 1/8 photos  
#6 moulding/spoiler · 1/8 photos

#4 wing/wing · 2/8 photos  
#3 inferior\_grille/bumper · 1/8 photos

#1 door/door · 1/8 photos  
#5 wing/wing · 1/8 photos





# What about you?



# Course overview: [ML-course-labs](#)

| #                                                                       | Lab                                                      | Topics                                                                                          | Colab                                                                                                               |
|-------------------------------------------------------------------------|----------------------------------------------------------|-------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| 0                                                                       | <a href="#">Python for Machine Learning</a>              | Python essentials, OOP ( <code>fit</code> / <code>predict</code> ), NumPy, pandas, Matplotlib   |  <a href="#">Open in Colab</a>   |
| <b>Part 1 — Classical Machine Learning</b>                              |                                                          |                                                                                                 |                                                                                                                     |
| Scikit-learn / NumPy. Small datasets, runs comfortably on a laptop CPU. |                                                          |                                                                                                 |                                                                                                                     |
| #                                                                       | Lab                                                      | Topics                                                                                          | Colab                                                                                                               |
| 1                                                                       | <a href="#">Ordinary Least Squares Regression</a>        | Linear model, normal equation, gradient descent from scratch                                    |  <a href="#">Open in Colab</a>   |
| 2                                                                       | <a href="#">Polynomial Regression and Regularisation</a> | Basis expansion, bias–variance trade-off, Ridge/Lasso, pipelines                                |  <a href="#">Open in Colab</a>   |
| 3                                                                       | <a href="#">Logistic Regression</a>                      | Sigmoid, cross-entropy, gradient descent from scratch, ROC/AUC                                  |  <a href="#">Open in Colab</a>   |
| 4                                                                       | <a href="#">Decision Trees and Random Forests</a>        | Impurity criteria, pruning, bagging, feature importance                                         |  <a href="#">Open in Colab</a> |
| 4b                                                                      | <a href="#">Decision Trees from Scratch</a>              | CART from scratch in NumPy: impurity, greedy split search, recursive growth, feature importance |  <a href="#">Open in Colab</a> |
| 5                                                                       | <a href="#">Support Vector Machines</a>                  | Maximum margin, soft margin, the kernel trick, hyperparameter search                            |  <a href="#">Open in Colab</a> |
| 6                                                                       | <a href="#">XGBoost and Gradient Boosting</a>            | Boosting theory, regularised objective, early stopping, tuning                                  |  <a href="#">Open in Colab</a> |

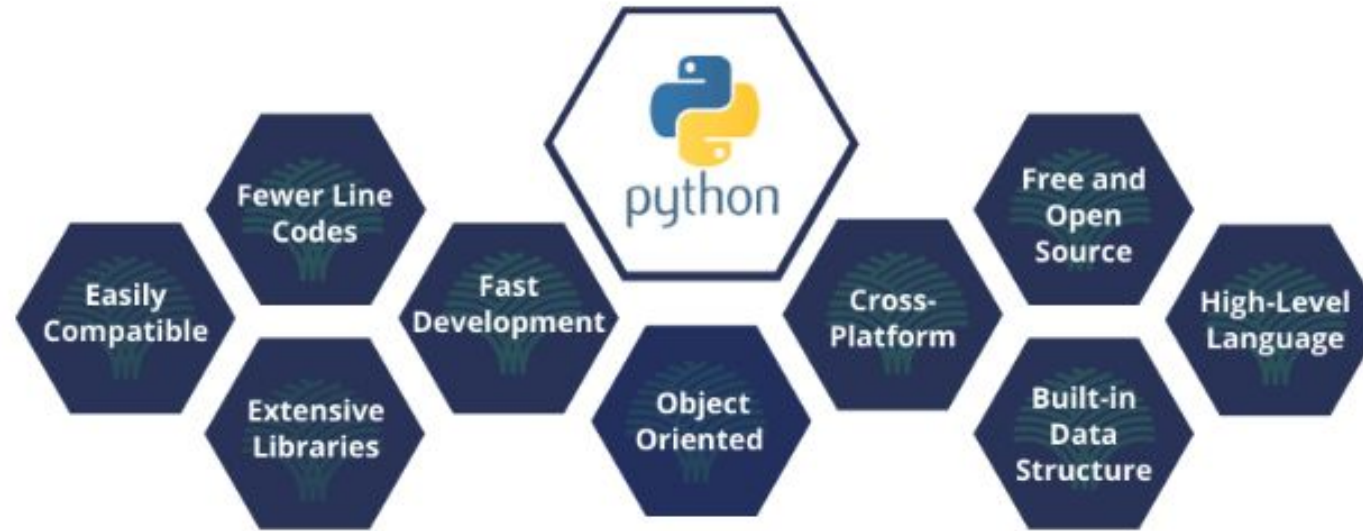
# Course overview: [ML-course-labs](#)

## ML course notes

Self-contained notes for the classical machine learning part of the course. Each notebook contains the theory with full derivations, plus short code cells that reproduce every plot. The notebooks are saved with their outputs, so they can be read on GitHub, in VS Code or in Colab without running anything.

| #  | Notebook                                            | Contents                                                                                                                                                                                                                   |
|----|-----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 00 | <a href="#">Supervised learning, splits and CV</a>  | problem setup, notation, empirical risk, bias-variance (proof), train/val/test, k-fold CV, leakage                                                                                                                         |
| 01 | <a href="#">Linear regression</a>                   | problem, MSE loss, gradient (derived), batch GD, learning rate (stability proof), normal equation, convexity, MLE equivalence, metrics                                                                                     |
| 02 | <a href="#">Logistic regression</a>                 | sigmoid, log-odds, cross-entropy = MLE, gradient, GD, convexity, separability; confusion matrix, precision/recall/F1, thresholds, ROC/AUC, PR curve, calibration                                                           |
| 03 | <a href="#">Regularisation, inputs, assumptions</a> | L2/Ridge, L1/Lasso (soft-thresholding), Elastic Net, MAP view, regularised logistic regression; scaling, one-hot and the dummy trap, collinearity/VIF; assumptions of linear and logistic regression with diagnostic plots |
| 04 | <a href="#">Softmax regression</a>                  | softmax, categorical cross-entropy, gradient, convexity, from-scratch implementation, multiclass metrics                                                                                                                   |
| 05 | <a href="#">Support vector classifier</a>           | margin geometry, hard/soft margin, equivalence with the hinge loss, sub-gradient descent from scratch, support vectors, dual and kernels                                                                                   |
| 06 | <a href="#">Decision trees and CART</a>             | impurity (Gini, entropy, variance), split search, CART from scratch, regression trees, cost-complexity pruning                                                                                                             |
| 07 | <a href="#">Bagging, random forests, boosting</a>   | variance of an average (proof), bootstrap and OOB, bagging, random forests, feature importance; AdaBoost and gradient boosting in brief                                                                                    |

# Python: Why?

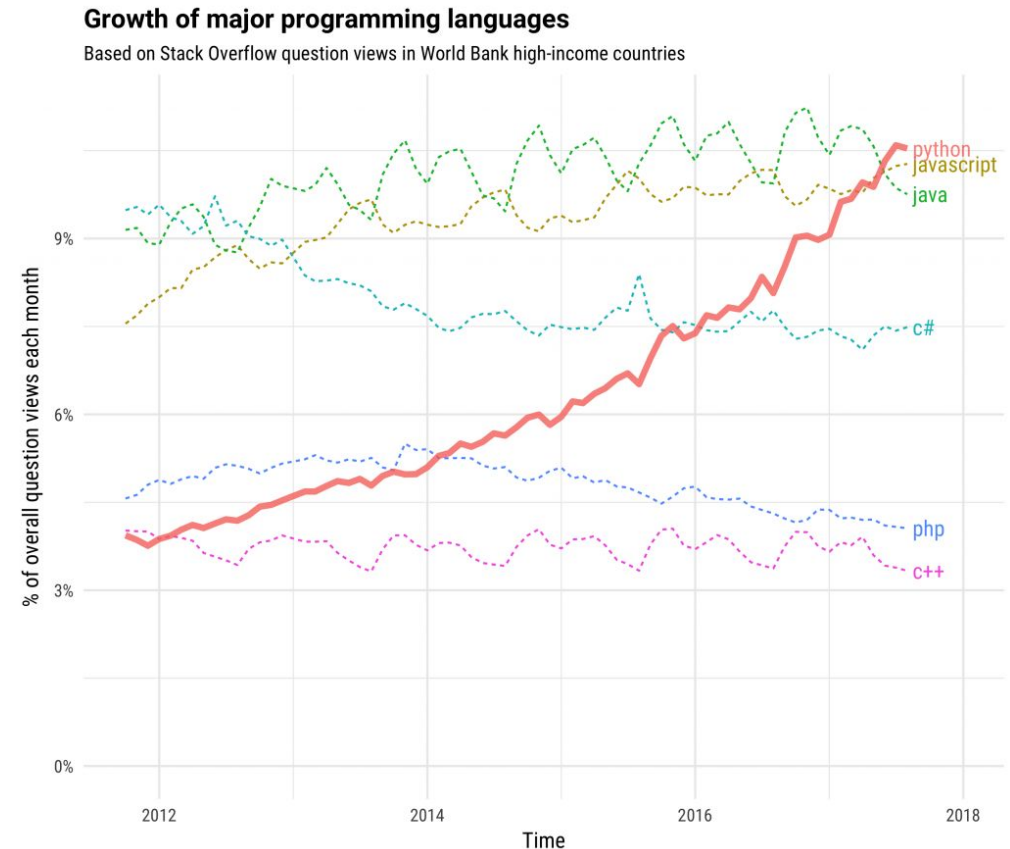


- Drawback?

Python is an interpreted scripting language, which makes it very convenient, but makes it is slower than other programming languages like c/c++. Many libraries are written in c/c++ (fast) under the hood .

# Python: Fundamentals for ML

- Python basics:
  - Datatypes
  - Lists and iterables
  - Functions
  - Conditionals
  - Object Oriented Programming (POO)
- Numpy:
  - Fast: Written in C
  - <https://numpy.org/doc/1.23/reference/index.html>
- Pandas:
  - Open-source data analysis and manipulation tool
  - <https://pandas.pydata.org/docs/reference/index.html>
- Sklearn:
  - <https://scikit-learn.org/stable/index.html>
- Matplotlib:
  - <https://matplotlib.org/>



# Python: Fundamentals for ML

[Python - Notebook](#)

# Python: Best fit curve

[First lab python](#)

# Python: Fundamentals for ML

```
[2] import pandas as pd
```

```
url = "https://raw.githubusercontent.com/twiradina/datasets/main/AmesHousing_simple.csv"  
df = pd.read_csv(url)  
df.head()
```



|   | BsmtFin SF 1 | Bsmt Unf SF | Total Bsmt SF | 1st Flr SF | 2nd Flr SF | Gr Liv Area | SalePrice |
|---|--------------|-------------|---------------|------------|------------|-------------|-----------|
| 0 | 639.0        | 441.0       | 1080.0        | 1656       | 0          | 1656        | 215000    |
| 1 | 468.0        | 270.0       | 882.0         | 896        | 0          | 896         | 105000    |
| 2 | 923.0        | 406.0       | 1329.0        | 1329       | 0          | 1329        | 172000    |
| 3 | 1065.0       | 1045.0      | 2110.0        | 2110       | 0          | 2110        | 244000    |
| 4 | 791.0        | 137.0       | 928.0         | 928        | 701        | 1629        | 189900    |

# Python: Fundamentals for ML

```
[4] print(df.shape)
     print(df.columns)
     df.head()
```

```
↗ (2930, 7)
Index(['BsmtFin SF 1', 'Bsmt Unf SF', 'Total Bsmt SF', '1st Flr SF',
      '2nd Flr SF', 'Gr Liv Area', 'SalePrice'],
      dtype='object')
```

|   | BsmtFin SF 1 | Bsmt Unf SF | Total Bsmt SF | 1st Flr SF | 2nd Flr SF | Gr Liv Area | SalePrice |
|---|--------------|-------------|---------------|------------|------------|-------------|-----------|
| 0 | 639.0        | 441.0       | 1080.0        | 1656       | 0          | 1656        | 215000    |
| 1 | 468.0        | 270.0       | 882.0         | 896        | 0          | 896         | 105000    |
| 2 | 923.0        | 406.0       | 1329.0        | 1329       | 0          | 1329        | 172000    |
| 3 | 1065.0       | 1045.0      | 2110.0        | 2110       | 0          | 2110        | 244000    |
| 4 | 791.0        | 137.0       | 928.0         | 928        | 701        | 1629        | 189900    |

# Python: Fundamentals for ML

```
print(df.isnull().sum())
```

```
BsmtFin SF 1      1  
Bsmt Unf SF      1  
Total Bsmt SF      1  
1st Flr SF        0  
2nd Flr SF        0  
Gr Liv Area       0  
SalePrice         0  
dtype: int64
```

```
df = df.dropna()
```

```
print(df.isnull().sum())
```

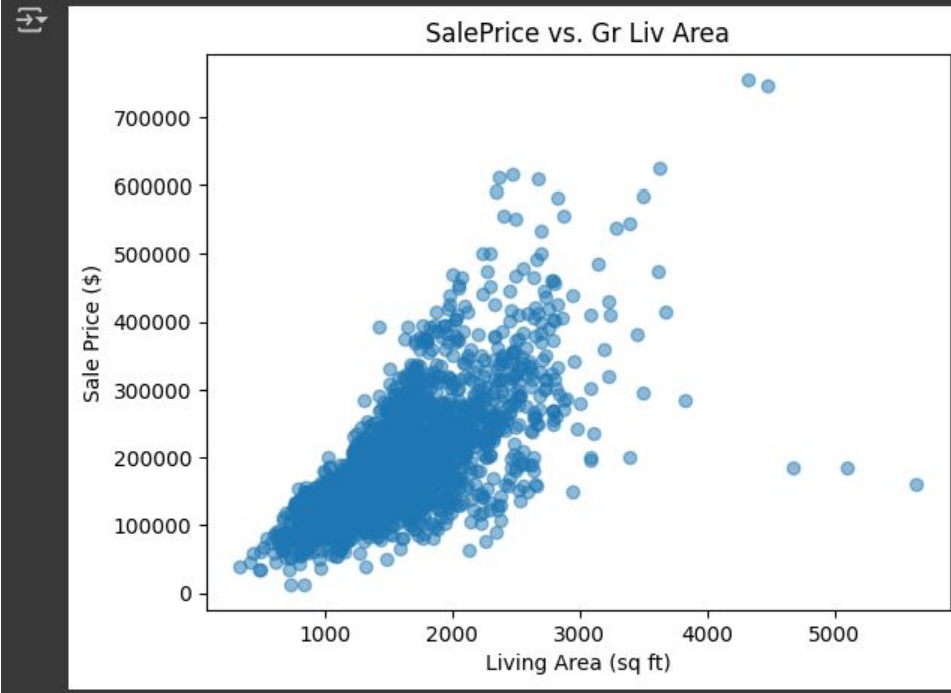
```
BsmtFin SF 1      0  
Bsmt Unf SF      0  
Total Bsmt SF      0  
1st Flr SF        0  
2nd Flr SF        0  
Gr Liv Area       0  
SalePrice         0  
dtype: int64
```

# Python: Fundamentals for ML

```
[10] import matplotlib.pyplot as plt

X = df["Gr Liv Area"].values
y = df["SalePrice"].values

plt.scatter(X, y, alpha=0.5)
plt.xlabel("Living Area (sq ft)")
plt.ylabel("Sale Price ($)")
plt.title("SalePrice vs. Gr Liv Area")
plt.show()
```



# Python: Fundamentals for ML

```
[25] import numpy as np

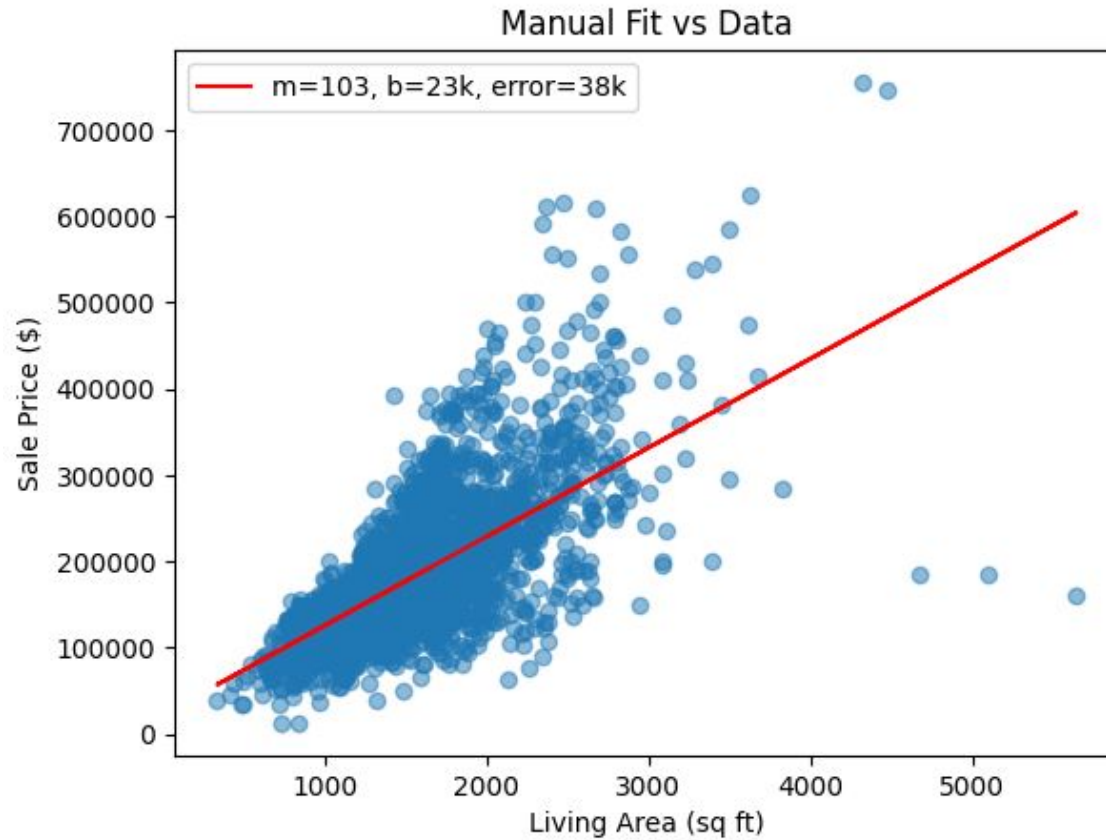
def calculate_error(y_true, y_pred):
    return np.mean(np.abs((y_true - y_pred)))

def return_line_values(m, b, X):
    return m * X + b
```

Find the line that  
minimizes that error



# Python: Fundamentals for ML



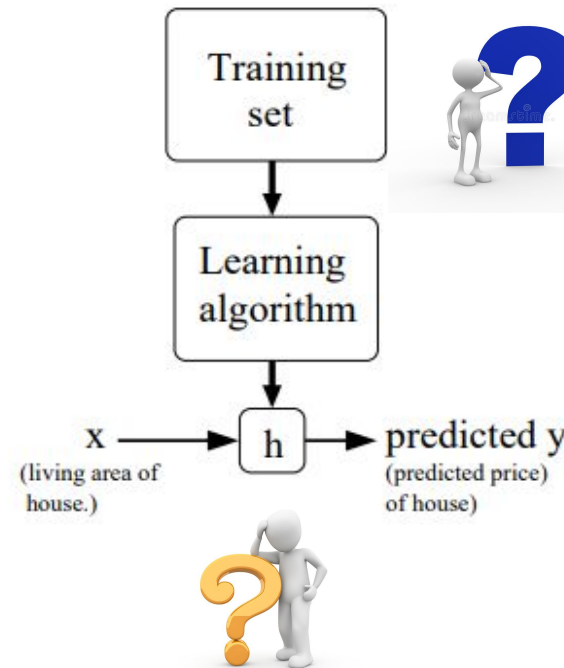
# Supervised Learning

Notation:

- $x_j^{(i)}$  feature  $j$  of sample  $i$
- $\mathbf{x}^{(i)}$  feature vector  $i$  (size  $n \times 1$ )
- $y^{(i)}$  target  $i$
- $(\mathbf{x}^{(i)}, y^{(i)})$  training pair  $i$
- $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^m$  training set
- $\mathcal{X}$  input space
- $\mathcal{Y}$  output space
- $h : \mathcal{X} \rightarrow \mathcal{Y}$  hypothesis

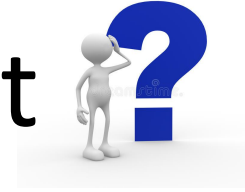
Objective:

- given a training set
- learn a function  $h$  so that  $h(\mathbf{x})$  is a “good” predictor of  $y$ .



- continuous - regression
- discrete - classification

# Dataset: Train – Validation – Test



- **Training Dataset.**

- Fit the model – m samples

- **Validation Dataset:**

- Provide an unbiased evaluation
- Used for tuning model hyperparameters or model selection.
- Becomes more biased (model occasionally sees this data, but never does it “Learn” from this).

- **Test Dataset.**

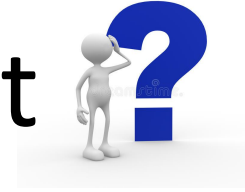
- Unbiased evaluation (data never seen).
- Present results

**Important:**

- Same distribution.
- No data leakage.
- Distribution?
  - Depends on number of hyperparameters and number of data samples.
  - (70-15-15) or (70-20-10) or (80-10-10) (or 90-7-3)



# Dataset: Train – Validation – Test

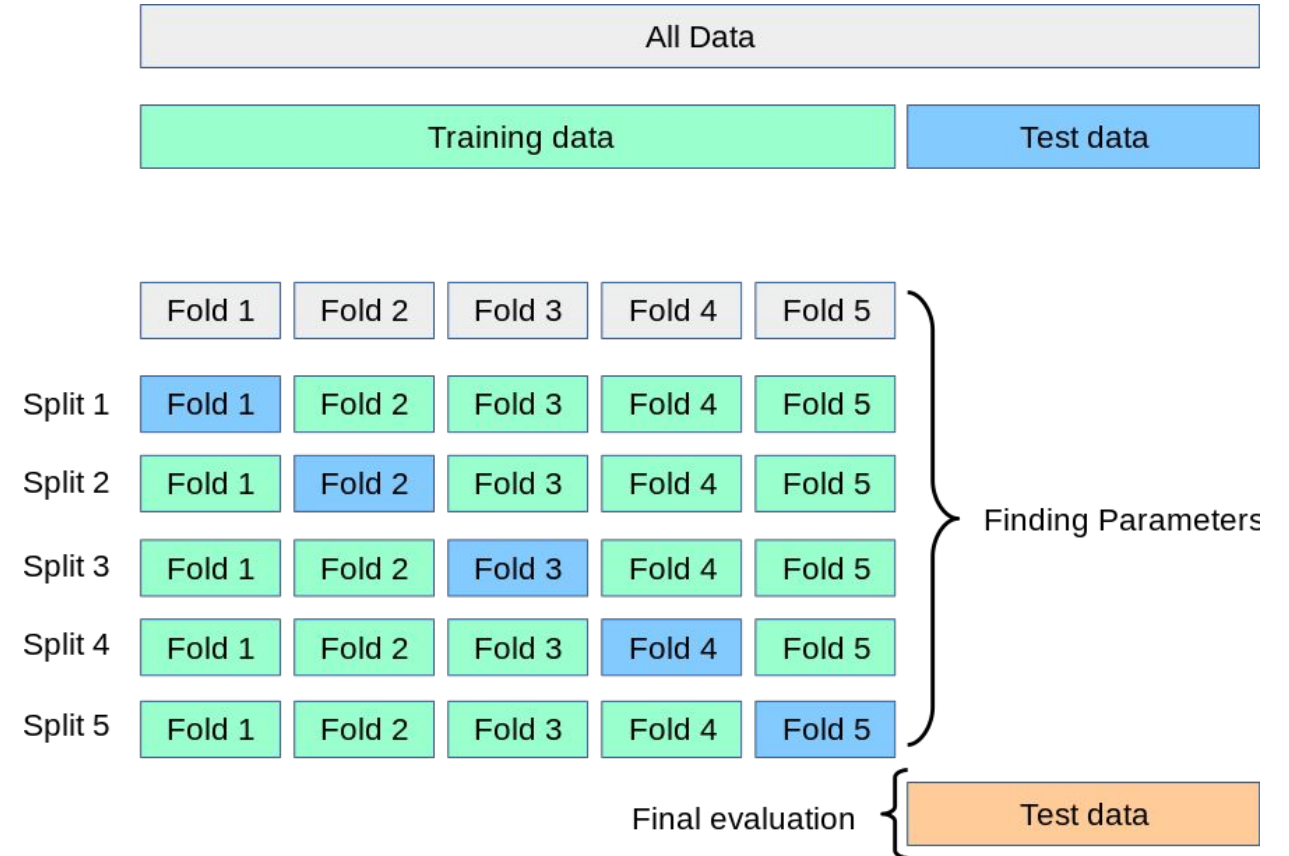


What if we only have a small dataset?

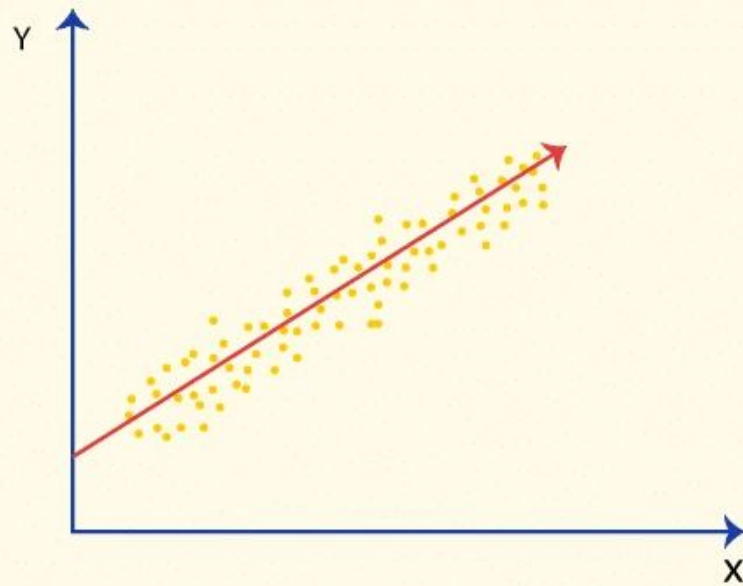
## K-fold cross validation:

1. Separate test data.
2. Split data into K folds.
3. Calculate K metrics:
  - Train on K-1 folds.
  - Validate in the remaining one.
4. Average the K metrics.

A frequent choice of K is 5 or 10



# Linear Regression

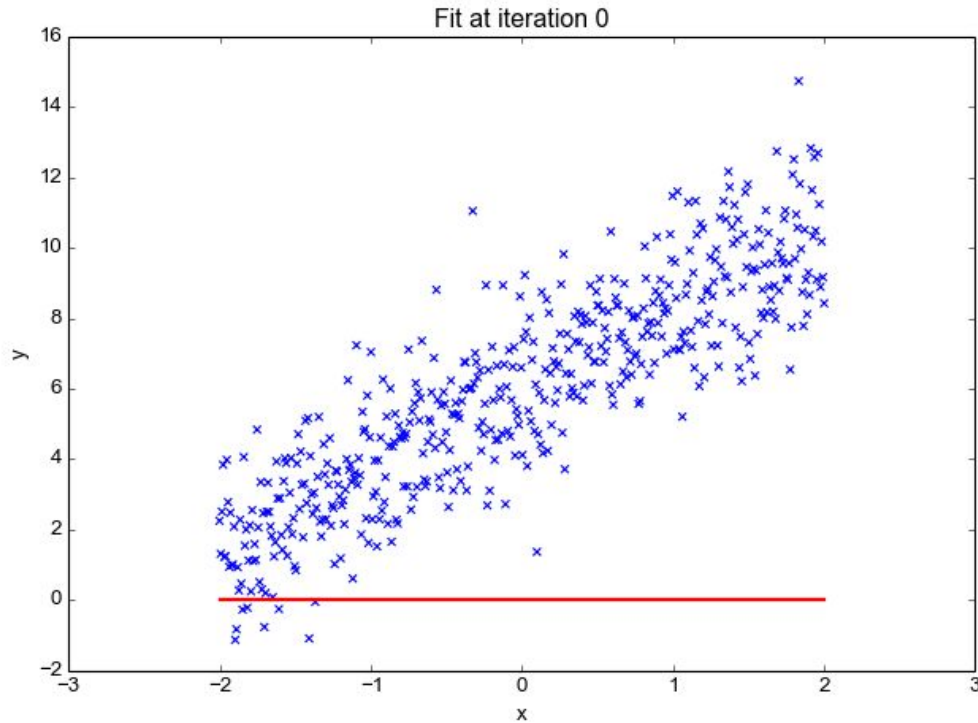


# Linear Regression – Real Word examples.

- Market sales.
- Medical insurance costs.
- Real Estate prices.
- Red Wine Quality.
- Taxi Fares.
- Life Expectancy.



# Linear Regression?



Approximate  $y$  as a linear function of  $x$

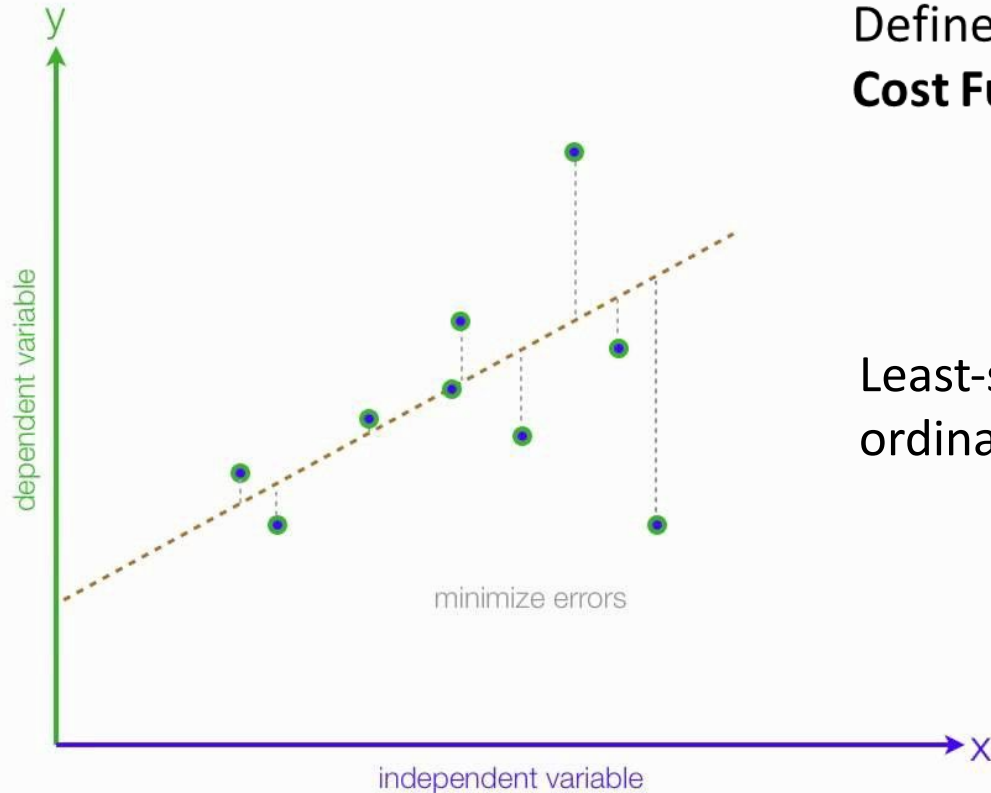
$$\begin{aligned}h_{\theta}(\mathbf{x}) &= \theta_0 + \theta_1 \cdot x_1 + \dots + \theta_n \cdot x_n \\ &= \theta^T \mathbf{x} = \sum_{i=0}^n \theta_i \cdot x_i\end{aligned}$$

$\theta$ 's are called the parameters (also called weights). For  $n = 1$ ,  $\theta_0$  is often denoted as the intercept and  $\theta_1$  the slope. Notice that for convenience we made  $x_0 = 1$

How do we select the  $\theta$ 's?



# Linear Regression ?



Define a function that measures, for each  $\theta$ 's, how "close".  
**Cost Function  $J(\theta)$**  (think of examples).

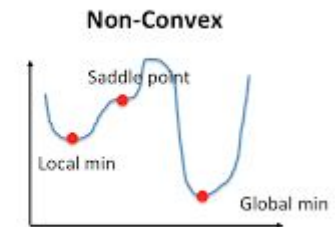
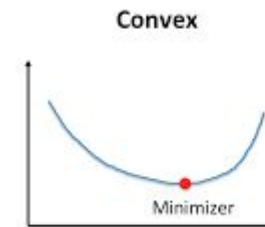
$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m \left( h_{\theta}(x^{(i)}) - y^{(i)} \right)^2$$

Least-squares cost function that gives rise to the ordinary least squares regression model.

How do we minimize the Cost function  $J$ ?

- Derivate and equal to zero\*.
- Apply gradient descent.

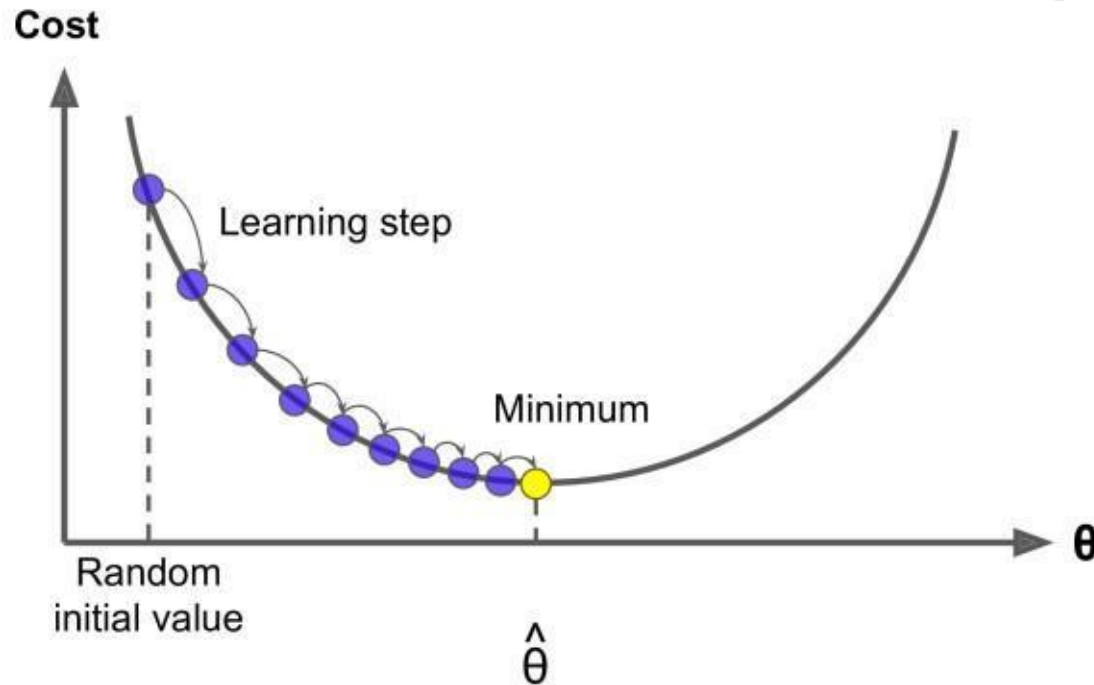
Convex function (global minimum)



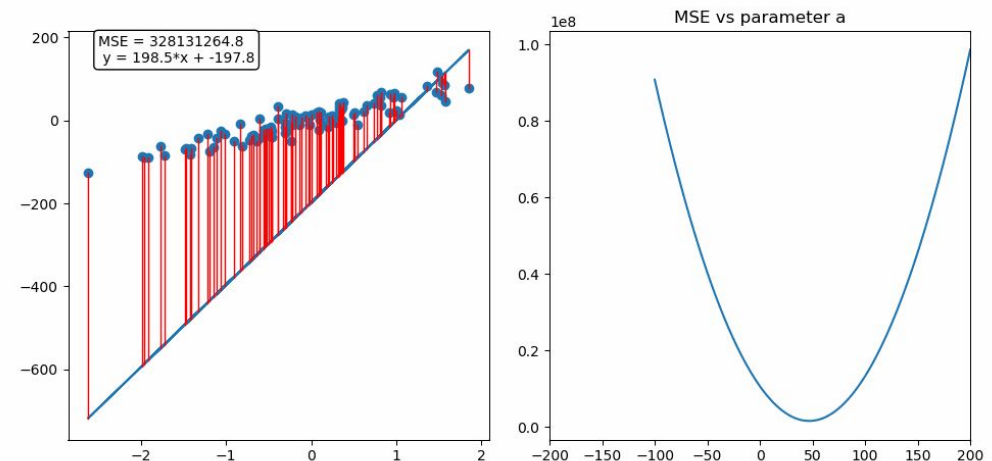
# Linear Regression – Gradient Descent

Repeat until convergence (and simultaneously among all parameters, i.e. for  $j \in \{0, \dots, n\}$ ):

$$\theta_j := \theta_j - \alpha \cdot \frac{\partial J(\theta)}{\partial \theta_j}$$

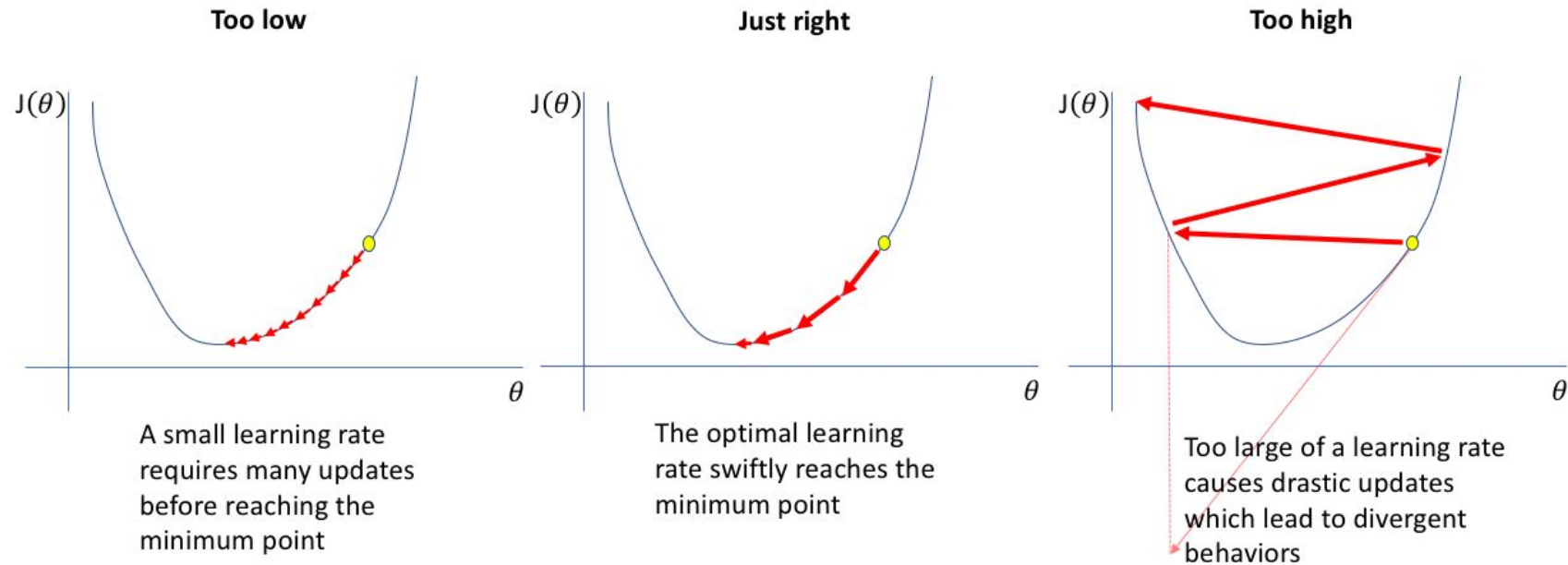


- Initial values of  $\theta_j$  ? Random (convex function).
- $\alpha$  is the learning rate. What value?
- $\frac{\partial J(\theta)}{\partial \theta_j}$  is the gradient (direction of maximum slope).



# Linear Regression – Gradient Descent

How do we select the correct  $\alpha$ ?



# Linear Regression – Gradient Descent

Repeat until convergence (and simultaneously among all parameters, i.e. for  $j \in \{0, \dots, n\}$ ):

$$\theta_j := \theta_j - \alpha \cdot \frac{\partial J(\theta)}{\partial \theta_j}$$

What is  $\frac{\partial J(\theta)}{\partial \theta_j}$  ?

$$\begin{aligned} \frac{\partial J(\theta)}{\partial \theta_j} &= \frac{\partial \left( \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 \right)}{\partial \theta_j} \\ &= \frac{1}{2m} \sum_{i=1}^m \frac{\partial \left( (h_{\theta}(x^{(i)}) - y^{(i)})^2 \right)}{\partial \theta_j} \\ &= \frac{1}{m} \sum_{i=1}^m \left( h_{\theta}(x^{(i)}) - y^{(i)} \right) \cdot \frac{\partial (h_{\theta}(x^{(i)}) - y^{(i)})}{\partial \theta_j} \\ &= \frac{1}{m} \sum_{i=1}^m \left( h_{\theta}(x^{(i)}) - y^{(i)} \right) \cdot x_j^{(i)} \end{aligned}$$

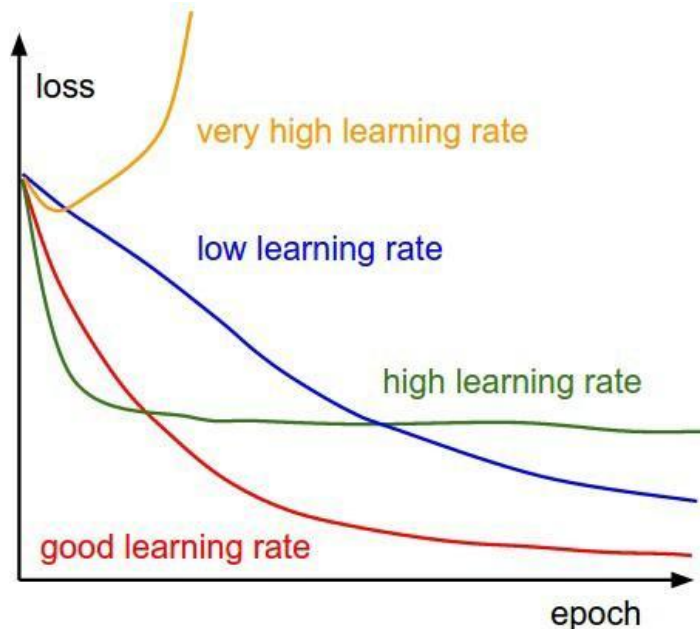
$$\theta_j := \theta_j - \alpha \cdot \frac{1}{m} \left( \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \right) \cdot x_j^{(i)}$$

“batch” gradient descent.  
(the  $m$  training samples in each step).

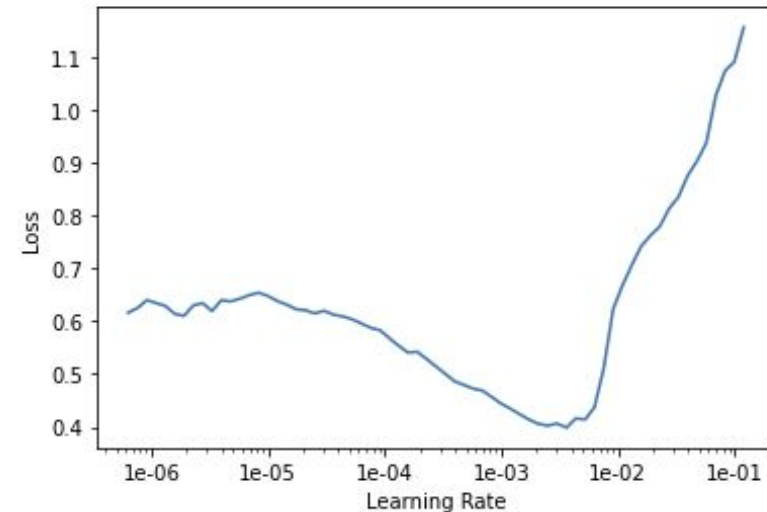
# Linear Regression – Gradient Descent

How do we make sure G.D. is working?

- $J(\theta)$  decreases for every step.  $\alpha$  is too large or there is a bug.
- If it decreases too slow.  $\alpha$  is too small..



```
In [65]: #take a look at the plot of loss  
learn.recorder.plot()
```



Try and select a consistent  $\alpha$ . Examples: 0.001, 0.003, 0.01, 0.03

# Linear Regression – Normal Equation.

Matrix notation:

$$y = \begin{pmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(m)} \end{pmatrix}_{m \times 1}, \quad \theta = \begin{pmatrix} \theta^{(0)} \\ \theta^{(1)} \\ \vdots \\ \theta^{(n)} \end{pmatrix}_{(n+1) \times 1}, \quad X = \begin{pmatrix} -(x^{(1)})^T - \\ -(x^{(2)})^T - \\ \vdots \\ -(x^{(m)})^T - \end{pmatrix} \begin{pmatrix} 1 & x_1^{(1)} & \cdots & x_n^{(1)} \\ 1 & x_1^{(2)} & \cdots & x_n^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(m)} & \cdots & x_n^{(m)} \end{pmatrix}_{m \times (n+1)}$$

Taking this into account:

$$\hat{y} = X\theta = \begin{pmatrix} (x^{(1)})^T \theta \\ (x^{(2)})^T \theta \\ \vdots \\ (x^{(m)})^T \theta \end{pmatrix}_{m \times 1} \quad E_{m \times 1} = y - \hat{y} = y - X\theta$$

And finally:

$$J(\theta) = (y - X\theta)^T (y - X\theta) = y^T y - y^T X\theta - \theta^T X^T y + \theta^T X^T X\theta$$

We can optimize by equating  $\nabla_{\theta}(J) = \mathbf{0}$   $((n+1) \times 1)$

It turns out that there is a known closed-form solution!

# Linear Regression – Normal Equation.

$$\begin{aligned}\nabla_{\theta} J(\theta) &= \nabla_{\theta} (y^T y - y^T X \theta - \theta^T X^T y + \theta^T X^T X \theta) \\ &= 0 - X^T y - X^T y + 2X^T X \theta \\ &= -2X^T y + 2X^T X \theta\end{aligned}$$

So we want to solve  $X^T X \theta = X^T y$ .

$$\theta = (X^T X)^{-1} X^T y$$

Unique only if  $X^T X$  is invertible.  
(can be replaced with pseudo-inverse).

Slow if  $n > 10000$  (use gradient descent)

HINT 1:

$$\nabla_{\theta} (\mathbf{a}^T \theta) = \nabla_{\theta} (a_0 \cdot \theta_0 + \dots + a_n \cdot \theta_n) = \mathbf{a}$$

$$\nabla_{\theta} (\theta^T \mathbf{a}) = \nabla_{\theta} (a_0 \cdot \theta_0 + \dots + a_n \cdot \theta_n) = \mathbf{a}$$

HINT 2:

$$\nabla_{\theta} (\theta^T X^T X \theta) = 2X^T X \theta$$

$$\begin{aligned}\theta^T X^T X \theta &= \sum_{i=0}^n \theta_i \left( \sum_{j=0}^n [X^T X]_{i,j} \theta_j \right) = \sum_{i=0}^n \sum_{j=0}^n [X^T X]_{i,j} \theta_i \theta_j \\ \frac{\partial (\theta^T X^T X \theta)}{\partial \theta_m} &= \sum_{i=0}^n \sum_{j=0}^n [X^T X]_{i,j} \frac{\partial (\theta_i \theta_j)}{\partial \theta_m} \\ &= \sum_{i=0}^n [X^T X]_{i,m} \theta_i + \sum_{j=0}^n [X^T X]_{m,j} \theta_j \\ &= 2 \sum_{j=0}^n [X^T X]_{m,j} \theta_j = 2 [X^T X \theta]_m\end{aligned}$$

# Linear Regression – Normal Equation.

A critical point (where  $\nabla_{\theta} J = 0$ ) is not automatically a minimum — this requires a second-order check. From Section 3, the Hessian is

$$\nabla_{\theta}^2 J(\theta) = 2X^T X.$$

**Positive semi-definiteness holds for any  $X$ .** For an arbitrary vector  $v$ :

$$v^T (X^T X) v = (Xv)^T (Xv) = \|Xv\|^2 \geq 0$$

Since the Hessian is positive semi-definite everywhere,  $J(\theta)$  is a **convex** function of  $\theta$ . For a convex function, every critical point is automatically a **global minimum** — there are no saddle points or local maxima to worry about.

**If  $X^T X$  is invertible**, it is positive semi-definite *with no zero eigenvalues*, hence **positive definite**:  $\|Xv\|^2 > 0$  for all  $v \neq 0$  (since  $Xv = 0 \Rightarrow v \in \ker(X) = \{0\}$  when  $X^T X$  is invertible). This makes  $J(\theta)$  **strictly convex**, so the minimum found in Section 4 is not merely global but **unique** — consistent with the uniqueness argument above.

# Linear Regression – Newton method.

Newton's method for optimization updates a guess  $\theta_{\text{old}}$  via:

$$\theta_{\text{new}} = \theta_{\text{old}} - [\nabla_{\theta}^2 J(\theta_{\text{old}})]^{-1} \nabla_{\theta} J(\theta_{\text{old}})$$

Plug in the results from Sections 2 and 3:

$$\nabla_{\theta} J(\theta_{\text{old}}) = -2X^T y + 2X^T X \theta_{\text{old}}, \quad \nabla_{\theta}^2 J(\theta_{\text{old}}) = 2X^T X \quad (\text{constant — independent of } \theta)$$

Substituting, starting from *any*  $\theta_{\text{old}}$ :

$$\begin{aligned} \theta_{\text{new}} &= \theta_{\text{old}} - (2X^T X)^{-1} (-2X^T y + 2X^T X \theta_{\text{old}}) \\ &= \theta_{\text{old}} - (X^T X)^{-1} X^T X \theta_{\text{old}} + (X^T X)^{-1} X^T y \\ &= \theta_{\text{old}} - \theta_{\text{old}} + (X^T X)^{-1} X^T y \\ &= (X^T X)^{-1} X^T y \end{aligned}$$

The  $\theta_{\text{old}}$  terms cancel *exactly*, regardless of the starting point.

**Interpretation.** Because  $J(\theta)$  is exactly quadratic, its Hessian  $2X^T X$  is constant — it does not depend on  $\theta$ . This means Newton's method, which normally requires many iterations to converge, lands *exactly* on the global minimum after a **single step**, from any starting point. The normal equation is precisely this one Newton step. This is the essential contrast with gradient descent, which takes many small steps to approach the same point, whereas Newton's method here reaches it in closed form immediately — at the cost of computing and inverting  $X^T X$ , which is why it becomes impractical for large  $n$  (as noted: slow if  $n > 10,000$ , favoring gradient descent instead).

# Linear Regression – lab 0: OLS from scratch

## Simple Linear Regression

The goal of this notebook is to demonstrate a linear regression model from the ground up using numpy.

```
[ ] %matplotlib inline
    from IPython.display import HTML
    import numpy as np
    import matplotlib.pyplot as plt
```

## Download dataset to colab

```
[ ] !ls

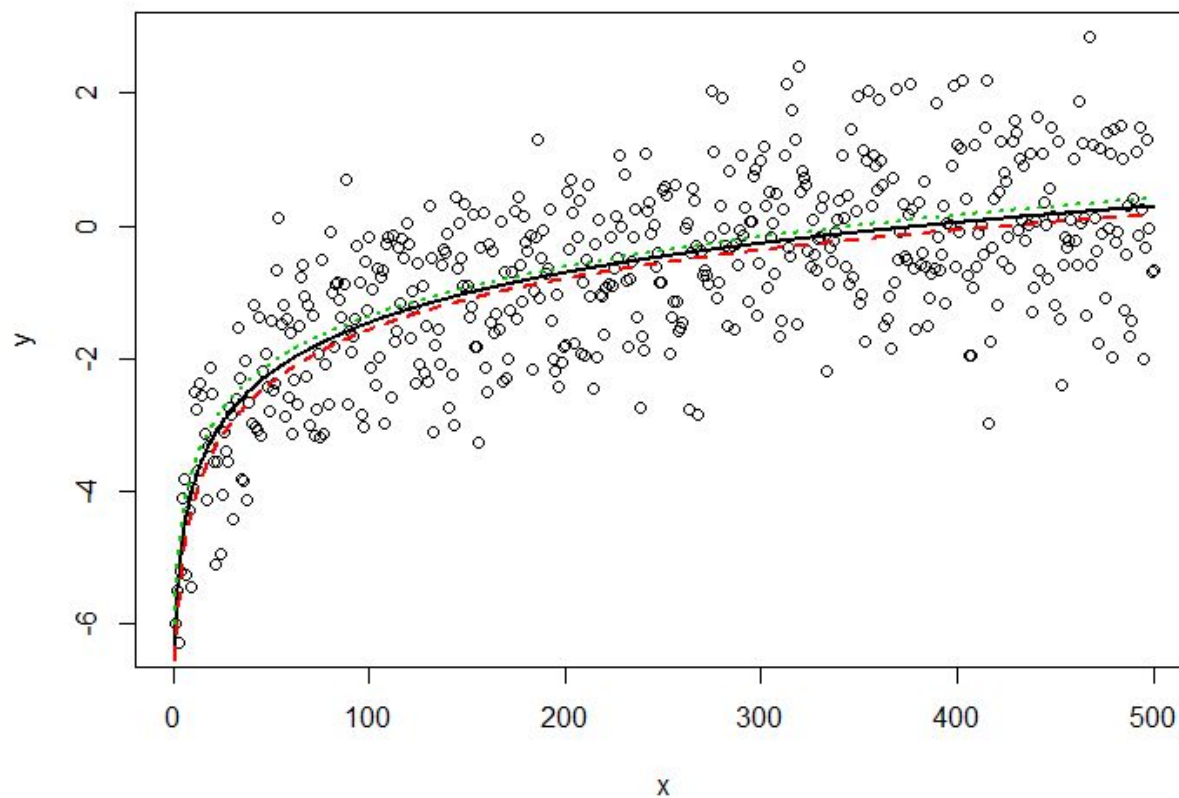
data.csv sample_data

[ ] import urllib
    urllib.request.urlretrieve("https://github.com/imagra93/MINT-ML/blob/main/data1.csv?raw=true", "data.csv")

('data.csv', <http.client.HTTPMessage at 0x7f27f3191198>)
```

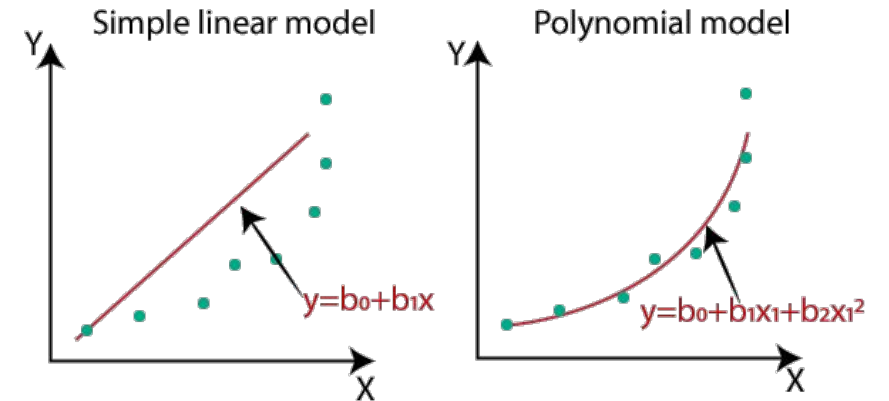
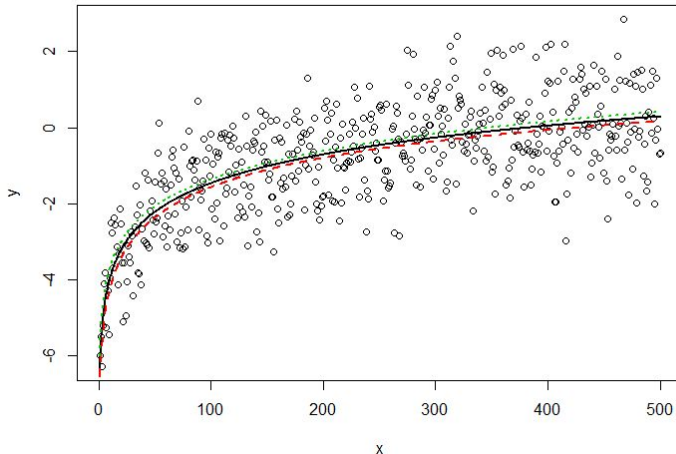
[Lab OLR](#)

# Linear Regression – Feature engineering



# Linear Regression – Feature engineering

- Make new features that represent important qualities. For example:
  - $x_1$  is the length of a property.
  - $x_2$  is the width of a property.
    - Make the new feature  $x_3 = x_1 \cdot x_2$  just a new column of your dataset.
- Polynomial regression. Make new features  $x_1^p$  for any  $p$ .
- Logarithmic regression.

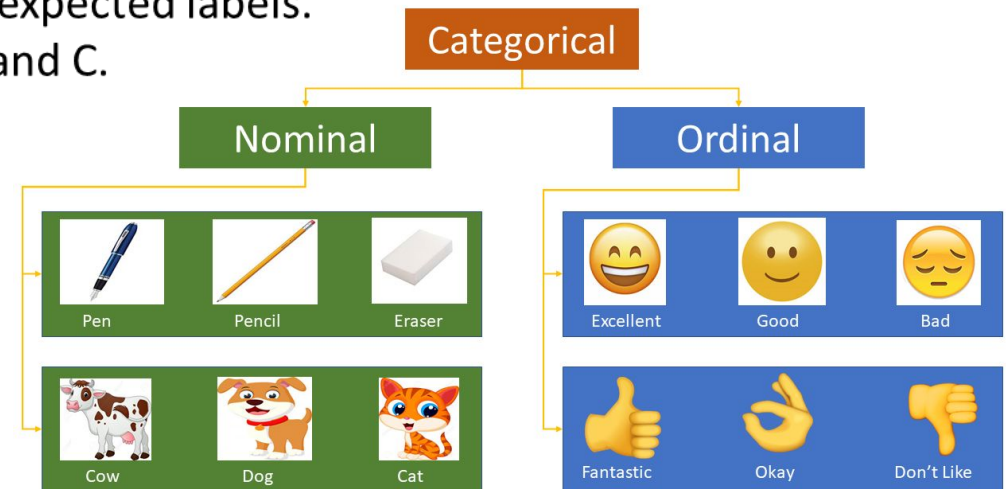


- Feature scaling for s.g.d.
- Regularization.

# Linear Regression – Feature encoding

Data is not always numerical. You must encode the categorical data to numbers before fitting your model.

- Ordinal Variables.
  - The “categories” argument as a list with the rank order of all expected labels.
  - Ordinal Encoding: Substitute names for numbers between 1 and C.
    - Bad -> 1, Good -> 2, Excellent -> 3
- Nominal Variables (no ordered relation).
  - One-Hot Encoding – 1 bit per category (new “features”)
    - Cow -> [1,0,0], Dog -> [0,1,0], Cat -> [0,0,1]
  - Dummy Encoding – C categories with C-1 binary variables.
    - Cow -> [1,0], Dog -> [0,1], Cat -> [0,0]
    - Same data is represented.
    - For algorithms with intercept term (such as linear regression).
      - One-Hot Encoding makes  $X^T X$  non-invertible (linear dependence).



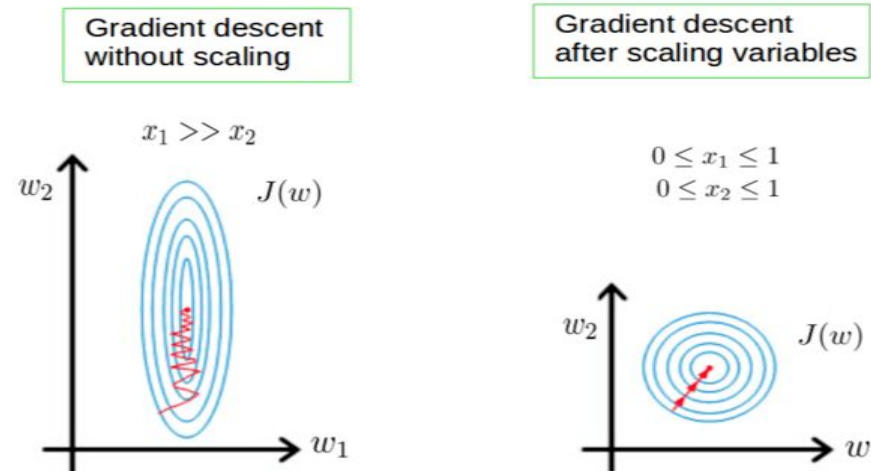
# Linear Regression – Feature scaling

- Why? It helps with convergence.
- Normalization (or Min-Max scaling):
  - $X' = \frac{X - X_{min}}{X_{max} - X_{min}}$
  - Values between 0 and 1.
- Standardization (values are centered around the mean with a unit standard deviation):
  - $X' = \frac{X - \mu}{\sigma}$
  - not restricted to a particular range.

When to use each? Not clear.

- Data does not follow a Gaussian distribution (Normalization?)
- Outliers do not affect standardization.

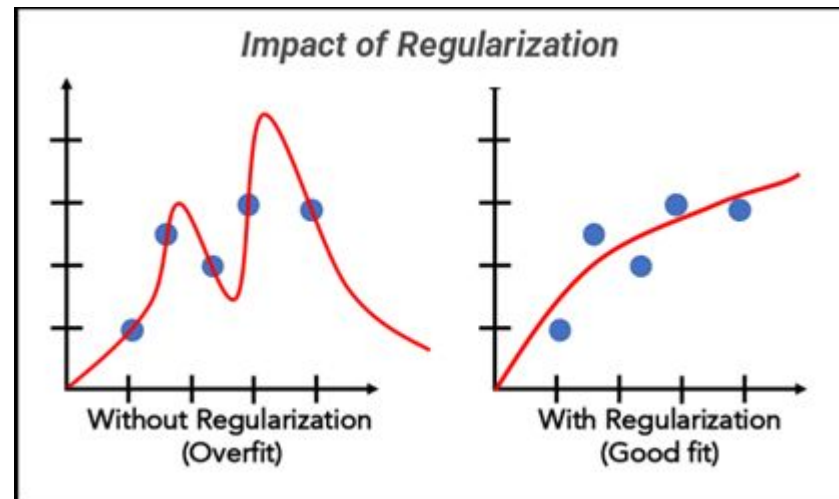
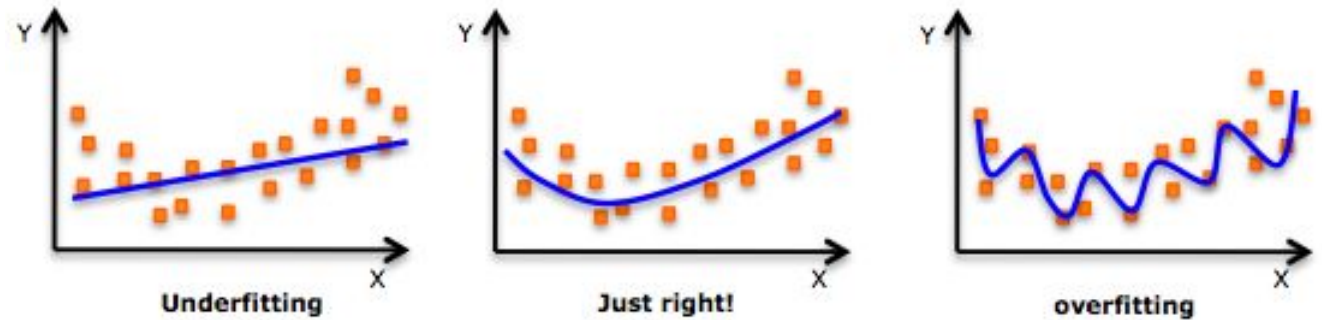
<https://www.analyticsvidhya.com/blog/2020/04/feature-scaling-machine-learning-normalization-standardization/>



Steps for gradient descent are updated at the same rate for all the features.

# Linear Regression – Regularization

- Poly. Regression of degree  $m$ .  $\rightarrow J(\theta) = 0$
- Good model for unseen samples?

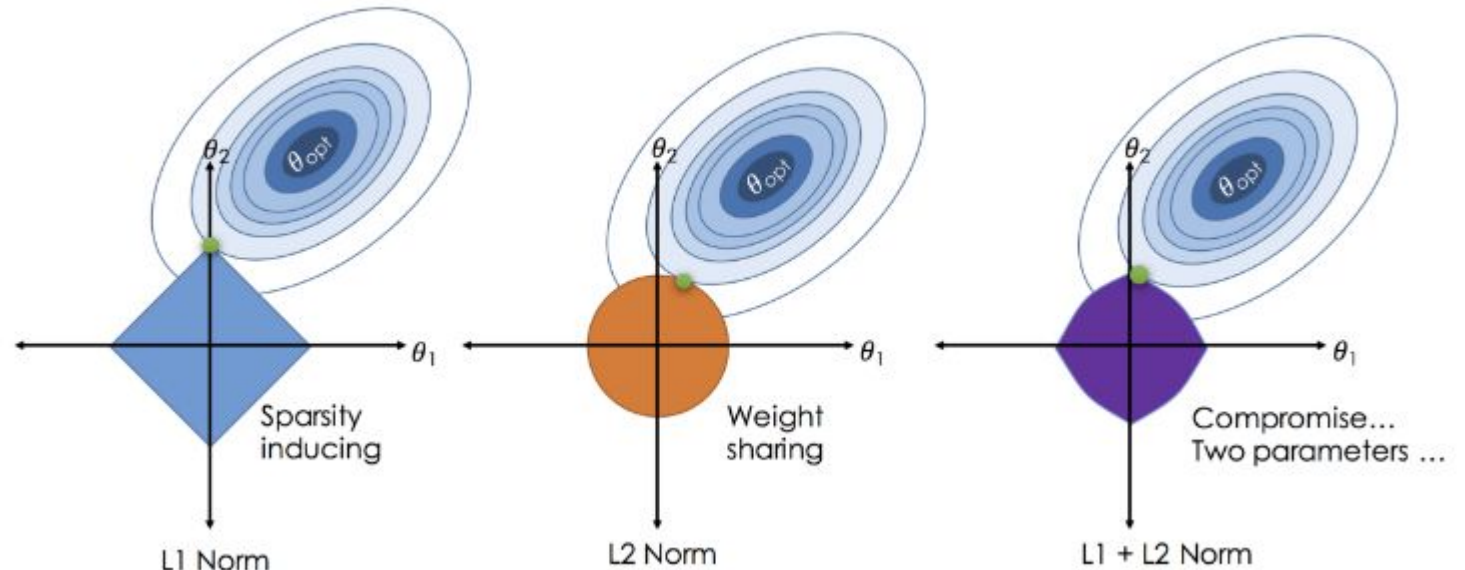


# Linear Regression – Regularization

Add penalization in the cost function to weights!

- **L2 (Ridge regression)** -  $J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \cdot \sum_{i=1}^n \theta_i^2$  (not  $i=0$ )
  - Shrinks the coefficients
- **L1 (Lasso regression)** -  $J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \cdot \sum_{i=1}^n |\theta_i|$ 
  - Forces some of the coefficients to be zero and results in a simpler model (feature selection)
  - Not differentiable

- **Elastic Net:** the combination of the L1 regularization and L2 regularization.



# Linear Regression – Metrics

Metrics are used to monitor and measure the performance of a model (during training and testing), and don't need to be differentiable (as Losses).

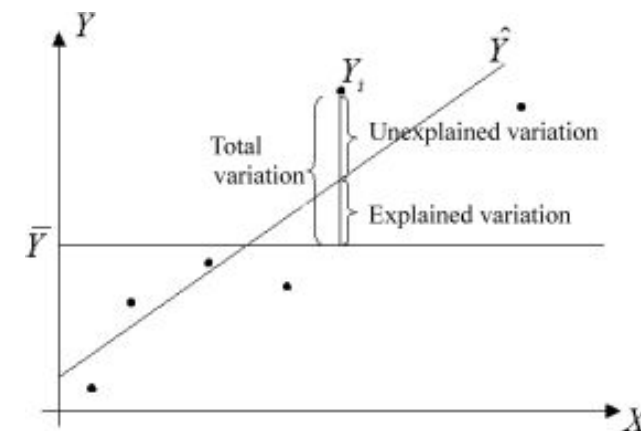
- Mean Square Error (MSE):  $\frac{1}{m} \sum_i (y_i - \hat{y}_i)^2$ 
  - Squaring factor -> more prone to outliers.
  - Interpreted in square form.
- Mean Absolute Error (MAE):  $\frac{1}{m} \sum_i |y_i - \hat{y}_i|$ 
  - Robust towards outliers
  - Non-differentiable
  - Interpretable (used only as metric).
- Root Mean Square Error (RMSE):  $\sqrt{\frac{1}{m} \sum_i (y_i - \hat{y}_i)^2}$ 
  - Retains the differentiable property of MSE.
  - Error interpretation can be done smoothly.

# Linear Regression – Metrics

“How much (what %) of the total variation in Y(target) is explained by the variation in X(regression line)”

- Coefficient of determination  $R^2 = 1 - \frac{\sum_i^m (y_i - \hat{y}_i)^2}{\sum_i^m (y_i - \text{mean}(y))^2}$ 
  - close to 1 (Ideal)
  - close to 0, couldn't capture any variance
  - Vanilla  $R^2$  method suffers for improving when model overfits the data

Best predictor before I knew X vs knowing it  
(weight of a person vs weight after knowing height)



- Adjusted  $R_a^2 = 1 - \left( \frac{m-1}{m-n-1} \cdot R^2 \right)$ , where m is the number of samples and n the number of independent variables.
  - Only increases if adding the actual feature really helps.

# Linear Regression – Probabilistic interpretation

Assume target variables and inputs are related via:

$$y^{(i)} = \theta^T x^{(i)} + e^{(i)}, \quad \text{with } e^{(i)} \sim \mathcal{N}(0, \sigma^2) \text{ I.I.D.}$$

$$p(e^{(i)}) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(e^{(i)})^2}{2\sigma^2}\right),$$

which implies that

$$p(y^{(i)}|x^{(i)}, \theta) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right).$$

Given  $X$  and  $\theta$ , the probability of the data is given by  $p(y|X; \theta)$ . We will define it as a function of  $\theta$ ,  $L(\theta) = p(y|X; \theta)$ . Assuming also i.i.d. samples:

$$\begin{aligned} L(\theta) &= \prod_{i=1}^m p(y^{(i)}|x^{(i)}, \theta) \\ &= \prod_{i=1}^m \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right). \end{aligned}$$

To make the problem simpler we can maximize the log of  $L(\theta)$  (monotonically increasing func.)

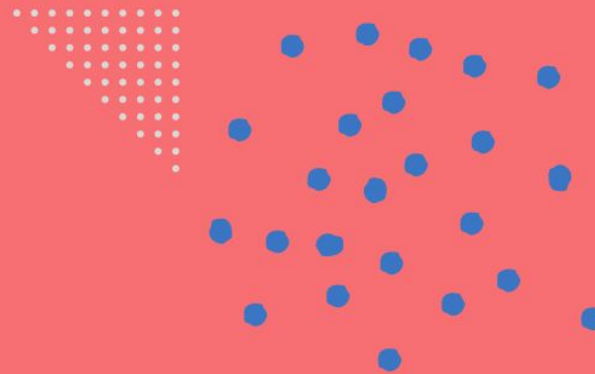
$$\begin{aligned} l(\theta) &= \log(L(\theta)) \\ &= \log\left(\prod_{i=1}^m \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right)\right) \\ &= \sum_{i=1}^m \log\left(\frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right)\right) \\ &= m \log\left(\frac{1}{\sigma\sqrt{2\pi}}\right) - \frac{1}{2\sigma^2} \sum_{i=1}^m (y^{(i)} - \theta^T x^{(i)})^2 \end{aligned}$$

Equivalent to minimizing:

$$\frac{1}{2m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)}\right)^2$$

**Maximum likelihood:** choose  $\theta$  to make the data as highly probable as possible, i.e., maximize  $L(\theta)$ .

# Logistic Regression



# Logistic Regression - Motivation

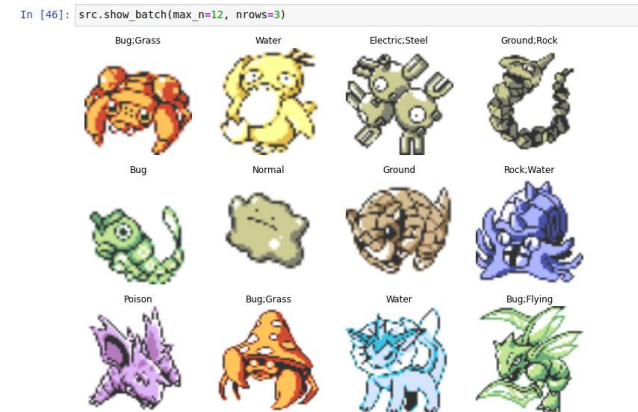
Outputs belong to classes:

- Malign or benign cancer
- heart attack occurs
- Credit is accepted or not
- Email is spam or not
- Sentiment analysis.
- A military radio signal is from “the enemy”

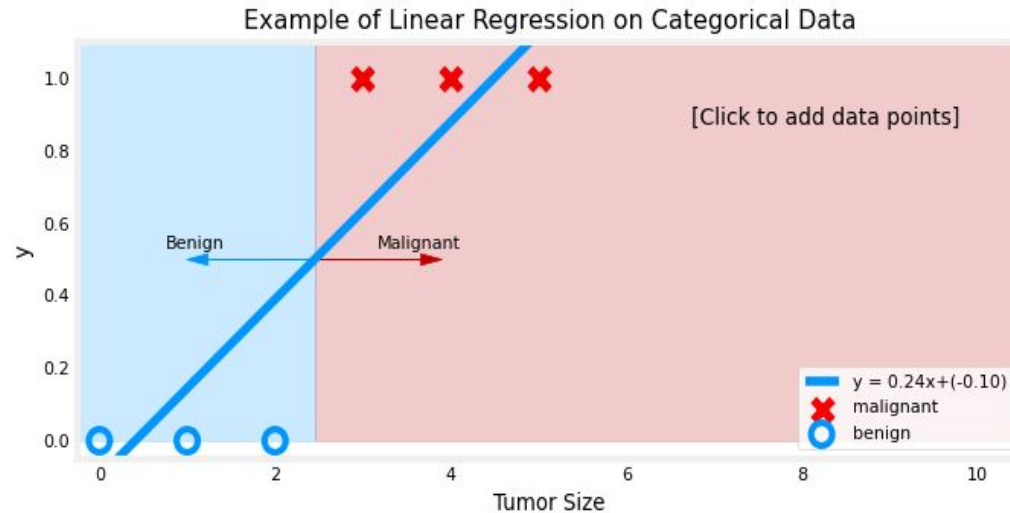


Also non-binary examples:

- Hand digit recognition
- (RF) Signal classification



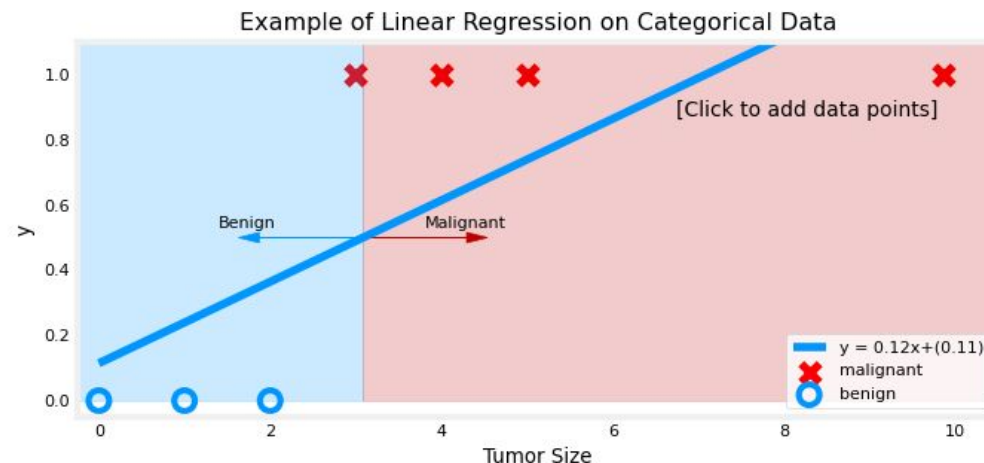
# Logistic Regression – Linear assumption



Approximate  $y$  as a linear function of  $x$

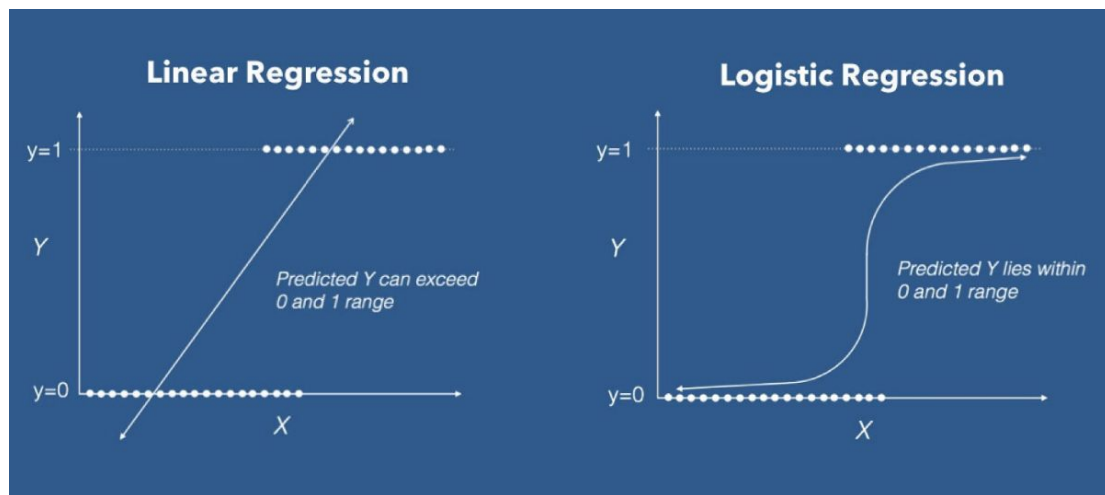
$$h(\mathbf{x}) = \sum_{i=0}^n \theta_i \cdot x_i = \theta^T \mathbf{x}$$

First perform linear regression and then establish a threshold, e.g. 0.5.



One extra data points leads to bad results.

# Logistic Regression – Sigmoid function.



Approximate  $y$  as a non-linear function of  $x$

$$h_{\theta}(\mathbf{x}) = g(\theta^T \mathbf{x}) = \frac{1}{1 + e^{-\theta^T \mathbf{x}}},$$

where

$$g(z) = \frac{1}{1 + e^{-z}}.$$

*The sigmoid function:*

- $0 \leq g(z) \leq 1$  is bounded.
- $g(z)$  can be interpreted as the probability that the class is 1.
  - $h_{\theta}(x) = P(Y = 1|x; \theta)$
  - $P(Y = 0|x; \theta) = 1 - h_{\theta}(x)$
  - Easy to establish a threshold (or at-least it makes sense to establish one).

- $g(z)$  is the sigmoid function.

# Logistic Regression – Cost function.

If we want to use square error cost, the function  $h_{\theta}(\mathbf{x})$  would no longer be convex.

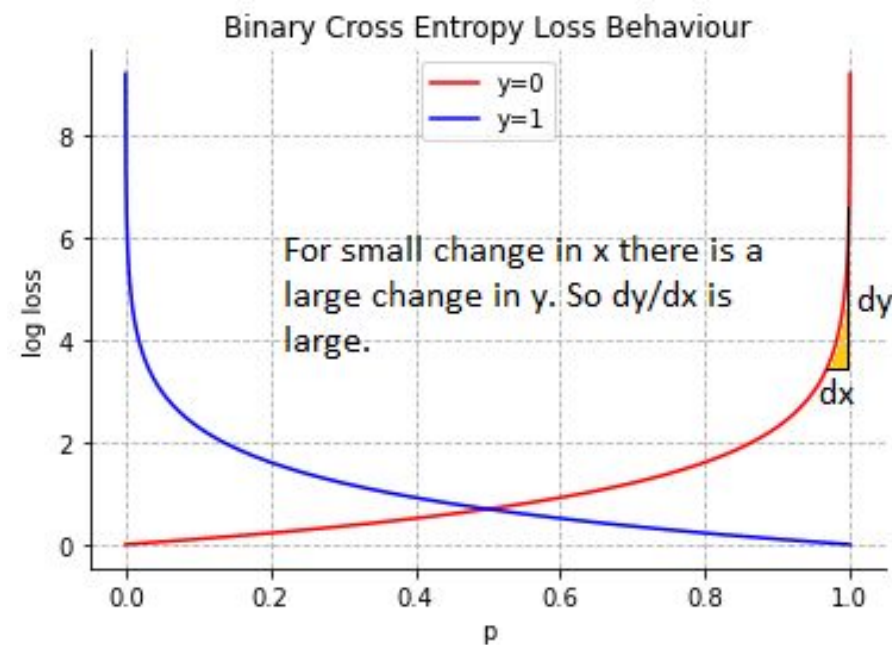
We will define the Binary Entropy loss as follows:

$$L(h_{\theta}(x^{(i)}), y^{(i)}) = \begin{cases} -\log(h_{\theta}(x^{(i)})) & \text{if } y^{(i)} = 1 \\ -\log(1 - h_{\theta}(x^{(i)})) & \text{if } y^{(i)} = 0 \end{cases}$$

or in 1 line,

$$L(h_{\theta}(x^{(i)}), y^{(i)}) = -y^{(i)} \cdot \log(h_{\theta}(x^{(i)})) - (1 - y^{(i)}) \cdot \log(1 - h_{\theta}(x^{(i)}))$$

- If  $y^{(i)} = 1$  we penalize very hard low probabilities.
- If  $y^{(i)} = 0$  we penalize very hard high probabilities.



# Logistic Regression – Cost function.

$$L(h_{\theta}(x^{(i)}), y^{(i)}) = \begin{cases} -\log(h_{\theta}(x^{(i)})) & \text{if } y^{(i)} = 1 \\ -\log(1 - h_{\theta}(x^{(i)})) & \text{if } y^{(i)} = 0 \end{cases}$$

or in 1 line,

$$L(h_{\theta}(x^{(i)}), y^{(i)}) = -y^{(i)} \cdot \log(h_{\theta}(x^{(i)})) - (1 - y^{(i)}) \cdot \log(1 - h_{\theta}(x^{(i)}))$$

And the cost function ends up being

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m L(h_{\theta}(x^{(i)}), y^{(i)})$$
$$\frac{1}{m} \sum_{i=1}^m \left( -y^{(i)} \cdot \log(h_{\theta}(x^{(i)})) - (1 - y^{(i)}) \cdot \log(1 - h_{\theta}(x^{(i)})) \right)$$

# Logistic Regression – Probabilistic interpretation

Another fit by maximum likelihood. Assume:

$$P(y^{(i)} = 1|x^{(i)}; \theta) = h_{\theta}(x^{(i)})$$

$$P(y^{(i)} = 0|x^{(i)}; \theta) = 1 - h_{\theta}(x^{(i)})$$

That can be compacted to

$$p(y^{(i)}|x^{(i)}; \theta) = h_{\theta}(x^{(i)})^{y^{(i)}} \cdot (1 - h_{\theta}(x^{(i)}))^{(1-y^{(i)})}$$

Taking into account the  $m$  i.i.d. training samples, the likelihood of the  $\theta$ 's can be written as

$$L(\theta) = p(y|X; \theta)$$

$$= \prod_{i=1}^m p(y^{(i)}|x^{(i)}; \theta)$$

$$= \prod_{i=1}^m h_{\theta}(x^{(i)})^{y^{(i)}} \cdot (1 - h_{\theta}(x^{(i)}))^{(1-y^{(i)})}$$

Once again, it is easier to maximize the log likelihood:

$$l(\theta) = \log L(\theta)$$

$$= y^{(i)} \cdot \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \cdot \log(1 - h_{\theta}(x^{(i)}))$$

(sum over  $i$  missing)

And this is equivalent to minimizing the negative!

# Logistic Regression – Optimization.

It turns out that it has become a convex optimization problem and we can apply gradient descent!

$$\begin{aligned}\frac{\partial J(\theta)}{\partial \theta_j} &= \frac{\partial \frac{1}{m} \sum_{i=1}^m (-y^{(i)} \cdot \log(h_\theta(x^{(i)})) - (1 - y^{(i)}) \cdot \log(1 - h_\theta(x^{(i)})))}{\partial \theta_j} \\&= \frac{1}{m} \sum_{i=1}^m \left( -\frac{y^{(i)}}{h_\theta(x^{(i)})} + \frac{1 - y^{(i)}}{1 - h_\theta(x^{(i)})} \right) \boxed{\frac{\partial h_\theta(x^{(i)})}{\partial \theta_j}} \\&= \frac{1}{m} \sum_{i=1}^m \left( \frac{h_\theta(x^{(i)}) - y^{(i)}}{h_\theta(x^{(i)}) \cdot (1 - h_\theta(x^{(i)}))} \right) \cdot h_\theta(x^{(i)}) \cdot (1 - h_\theta(x^{(i)})) \cdot \boxed{\frac{\partial(\theta^T \mathbf{x}^{(i)})}{\partial \theta_j}} \\&= \frac{1}{m} \sum_{i=1}^m (h_\theta(x^{(i)}) - y^{(i)}) \cdot x_j^{(i)}\end{aligned}$$

**HINT:**

$$\begin{aligned}g'(z) &= \frac{d}{dz} \left( \frac{1}{1 + e^{-z}} \right) \\&= \frac{1}{(1 + e^{-z})^2} \cdot e^{-z} \\&= \frac{1}{1 + e^{-z}} \cdot \frac{1 + e^{-z} - 1}{1 + e^{-z}} \\&= \frac{1}{1 + e^{-z}} \cdot \left( 1 - \frac{1}{1 + e^{-z}} \right) \\&= g(z) \cdot (1 - g(z))\end{aligned}$$

# Logistic Regression – Gradient descent.

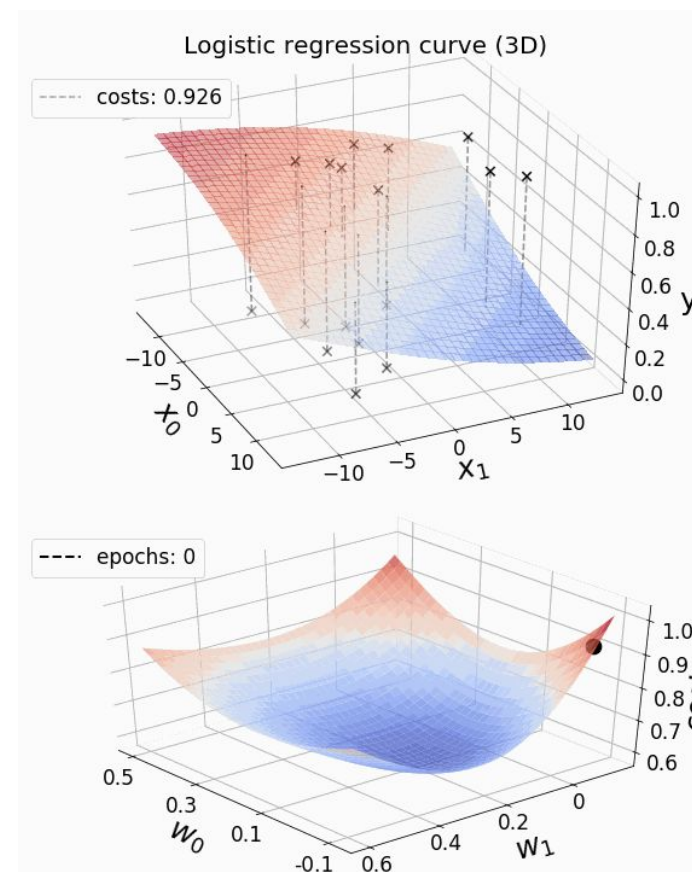
- Repeat until convergence (and simultaneously among all parameters, i.e. for  $j \in \{0, \dots, n\}$ ):

$$\begin{aligned}\theta_j &:= \theta_j - \alpha \cdot \frac{\partial J(\theta)}{\partial \theta_j} \\ &= \theta_j - \alpha \cdot \frac{1}{m} \sum_{i=1}^m \left( h_{\theta}(x^{(i)}) - y^{(i)} \right) \cdot x_j^{(i)}\end{aligned}$$

Very similar to linear regression! (with its own non-linear hypothesis)

Vectorized:

$$\theta = \theta - \alpha \cdot \frac{1}{m} X^T (g(\theta^T X) - y)$$



# Logistic Regression – Decision

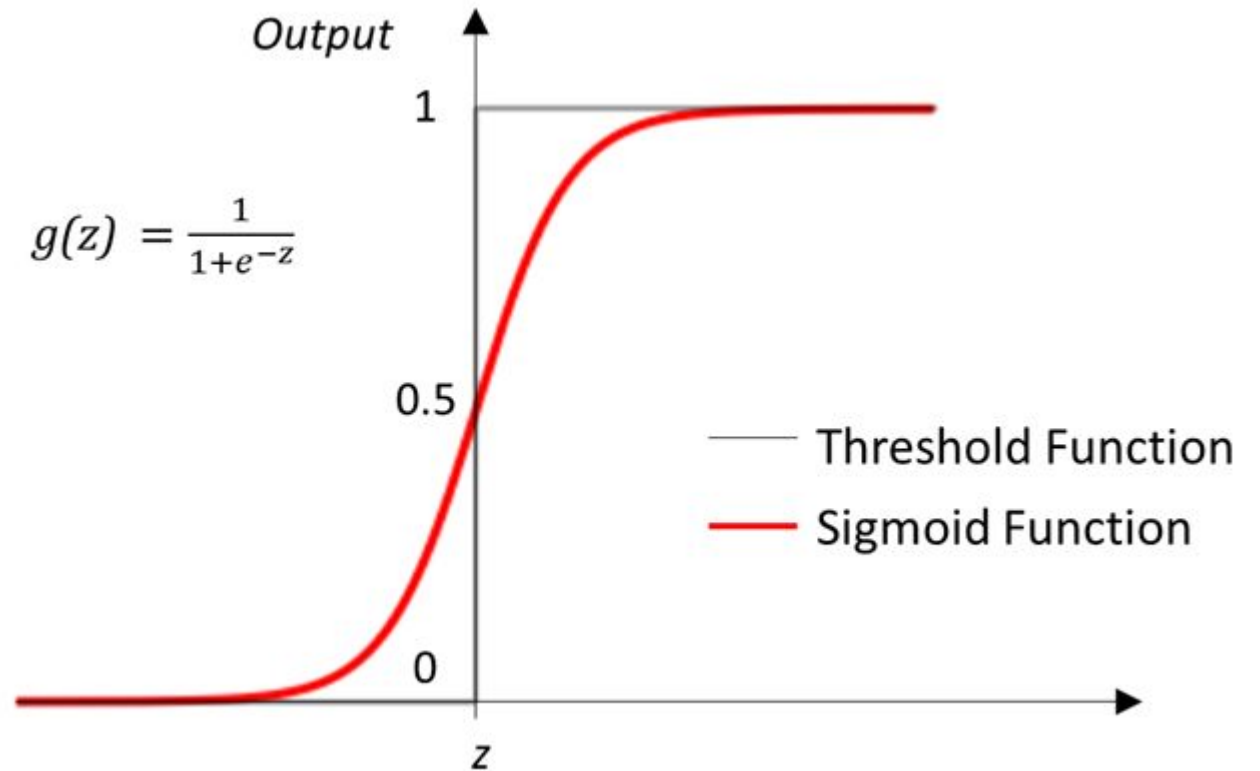
- Need to apply a certain threshold.

$$\hat{y} = \begin{cases} 1 & \text{if } h_{\theta}(x^{(i)}) \geq \text{threshold} \\ 0 & \text{otherwise} \end{cases}$$

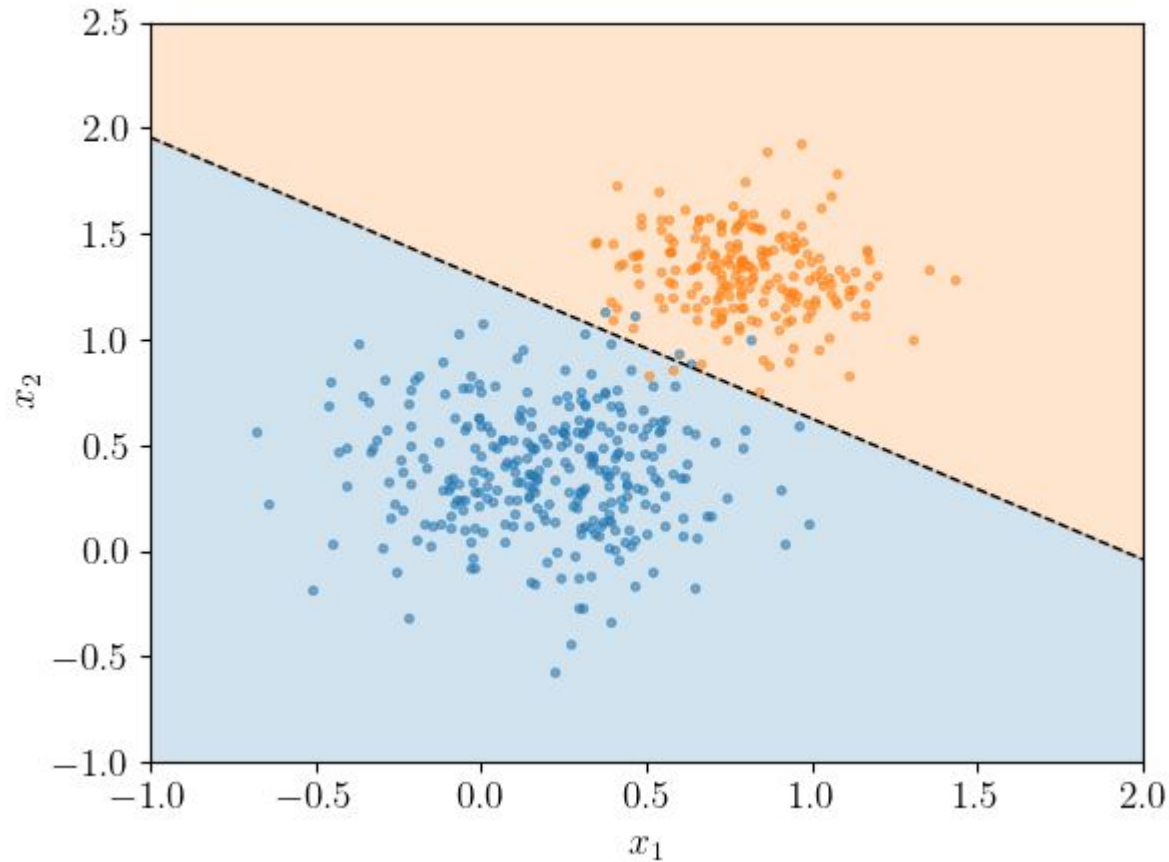
- A common threshold is 0.5 but can vary

It is application specific:

- Higher thresholds -> true positives
  - Spam detection.
- Lower thresholds -> false positives
  - Medical image (pre-doctor)



# Logistic Regression – Decision boundary



- Line given by  $h_{\theta}(x) = \text{threshold}$ .
- Separates the feature space in 2.

# Logistic Regression – Metrics.

Confusion matrix:

|              |          | Predicted Class                            |                                                            |                                                          |
|--------------|----------|--------------------------------------------|------------------------------------------------------------|----------------------------------------------------------|
|              |          | Positive                                   | Negative                                                   |                                                          |
| Actual Class | Positive | True Positive (TP)                         | False Negative (FN)<br><b>Type II Error</b>                | <b>Sensitivity</b><br>$\frac{TP}{(TP + FN)}$             |
|              | Negative | False Positive (FP)<br><b>Type I Error</b> | True Negative (TN)                                         | <b>Specificity</b><br>$\frac{TN}{(TN + FP)}$             |
|              |          | <b>Precision</b><br>$\frac{TP}{(TP + FP)}$ | <b>Negative Predictive Value</b><br>$\frac{TN}{(TN + FN)}$ | <b>Accuracy</b><br>$\frac{TP + TN}{(TP + TN + FP + FN)}$ |

Accuracy:

- Correct predictions divided by the total
- Bad for imbalanced datasets (majority vote)
- Every class is equally important

# Logistic Regression – Metrics.

Confusion matrix:

|              |          | Predicted Class                            |                                                            |                                                          |
|--------------|----------|--------------------------------------------|------------------------------------------------------------|----------------------------------------------------------|
|              |          | Positive                                   | Negative                                                   |                                                          |
| Actual Class | Positive | True Positive (TP)                         | False Negative (FN)<br><b>Type II Error</b>                | <b>Sensitivity</b><br>$\frac{TP}{(TP + FN)}$             |
|              | Negative | False Positive (FP)<br><b>Type I Error</b> | True Negative (TN)                                         | <b>Specificity</b><br>$\frac{TN}{(TN + FP)}$             |
|              |          | <b>Precision</b><br>$\frac{TP}{(TP + FP)}$ | <b>Negative Predictive Value</b><br>$\frac{TN}{(TN + FN)}$ | <b>Accuracy</b><br>$\frac{TP + TN}{(TP + TN + FP + FN)}$ |

Precision:

- Ratio of true positives and total positives predicted
- Higher precision means that an algorithm returns more relevant results than irrelevant ones.
- Example: potential clients. We do not want to waste money on ads.

# Logistic Regression – Metrics.

Confusion matrix:

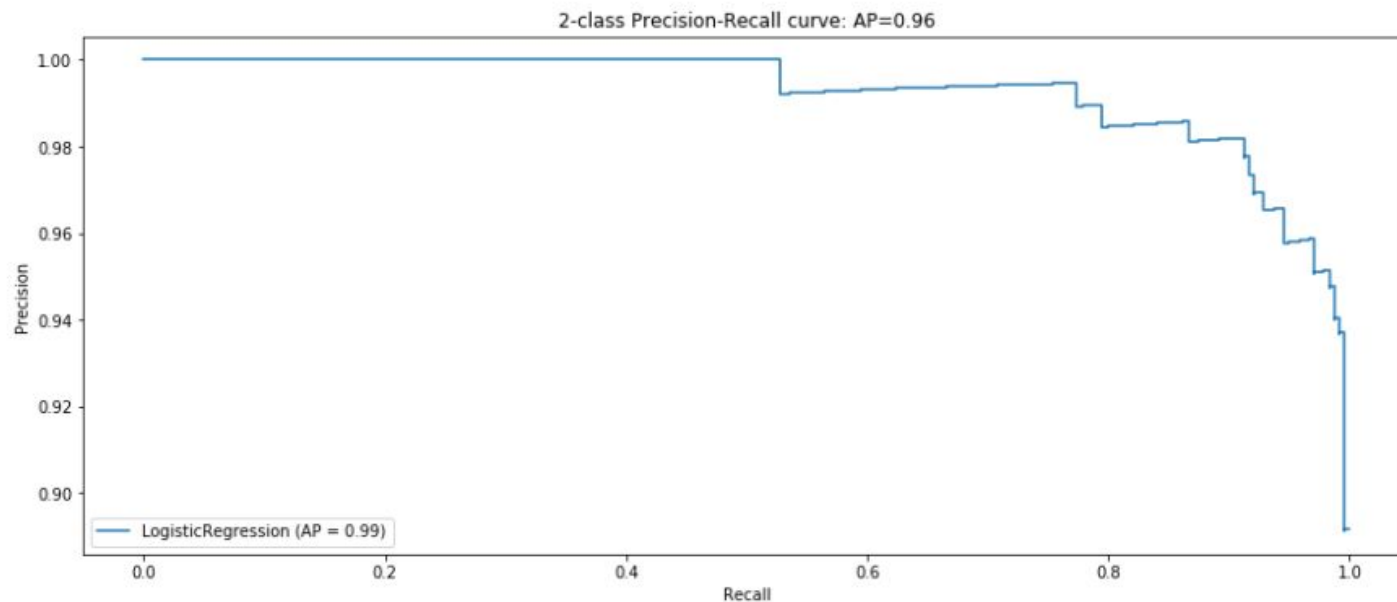
|              |          | Predicted Class                            |                                                            |                                                          |
|--------------|----------|--------------------------------------------|------------------------------------------------------------|----------------------------------------------------------|
|              |          | Positive                                   | Negative                                                   |                                                          |
| Actual Class | Positive | True Positive (TP)                         | False Negative (FN)<br><b>Type II Error</b>                | <b>Sensitivity</b><br>$\frac{TP}{(TP + FN)}$             |
|              | Negative | False Positive (FP)<br><b>Type I Error</b> | True Negative (TN)                                         | <b>Specificity</b><br>$\frac{TN}{(TN + FP)}$             |
|              |          | <b>Precision</b><br>$\frac{TP}{(TP + FP)}$ | <b>Negative Predictive Value</b><br>$\frac{TN}{(TN + FN)}$ | <b>Accuracy</b><br>$\frac{TP + TN}{(TP + TN + FP + FN)}$ |

Sensitivity or Recall:

- True positives to all the positives
- Model didn't miss any true positives.
- Example: Cancer patients, we do not want false negatives.

# Logistic Regression – Prec. vs Recall tradeoff.

By varying the threshold, you can either improve precision or sensitivity/recall – but not both.



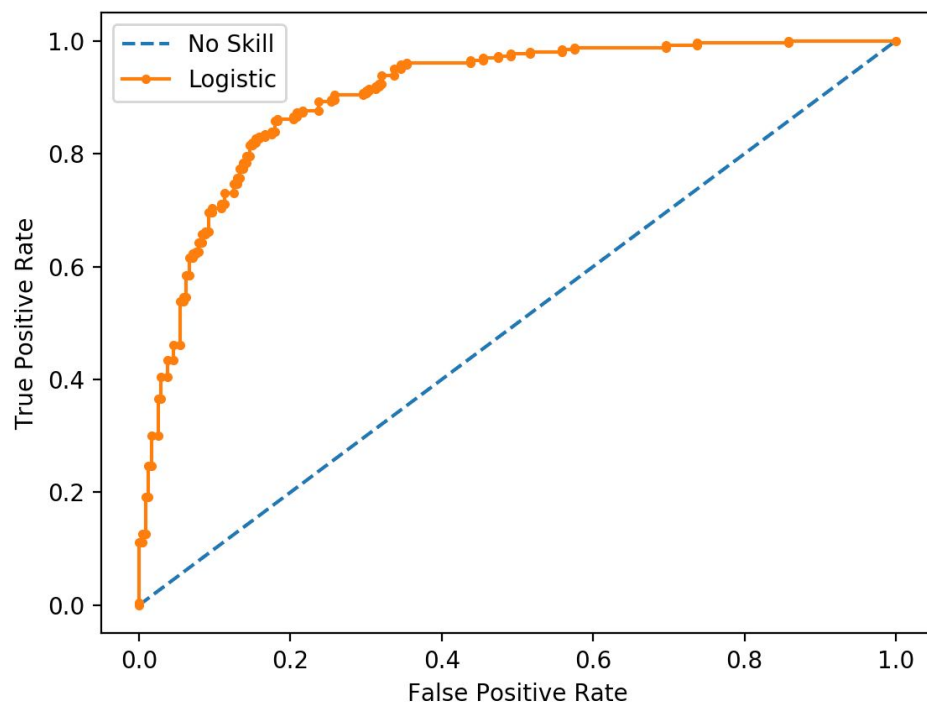
This tradeoff highly impacts real-world scenarios, therefore, precision and recall alone aren't very good metrics to rely on and work with.

F1 score: 
$$F_1 = \frac{2}{\frac{1}{Recall} + \frac{1}{Precision}}$$

- Widely used, “Good” for comparing.
- Good balance even on imbalanced classification problems.
- Low F1 doesn't say which cases it failed (poor information if alone).

# Logistic Regression – ROC.

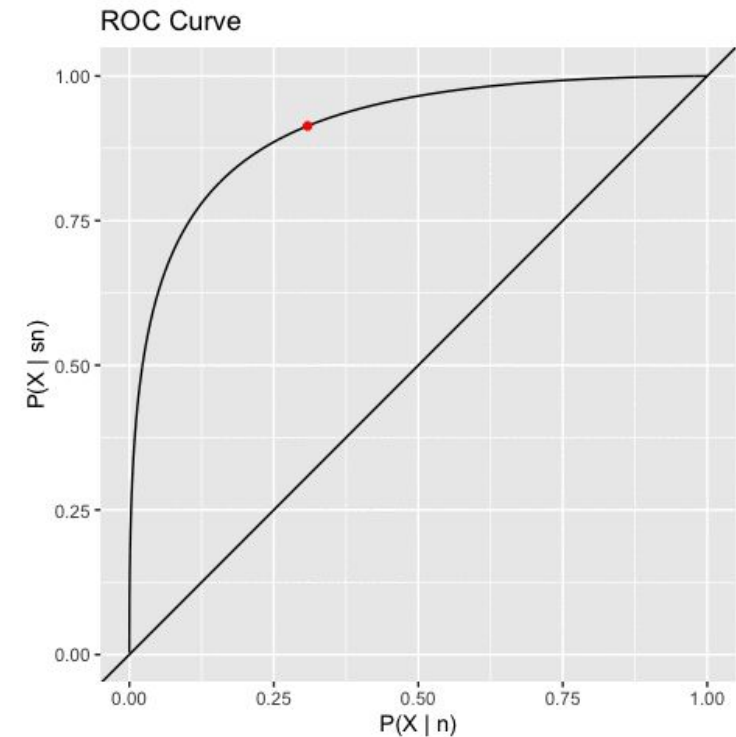
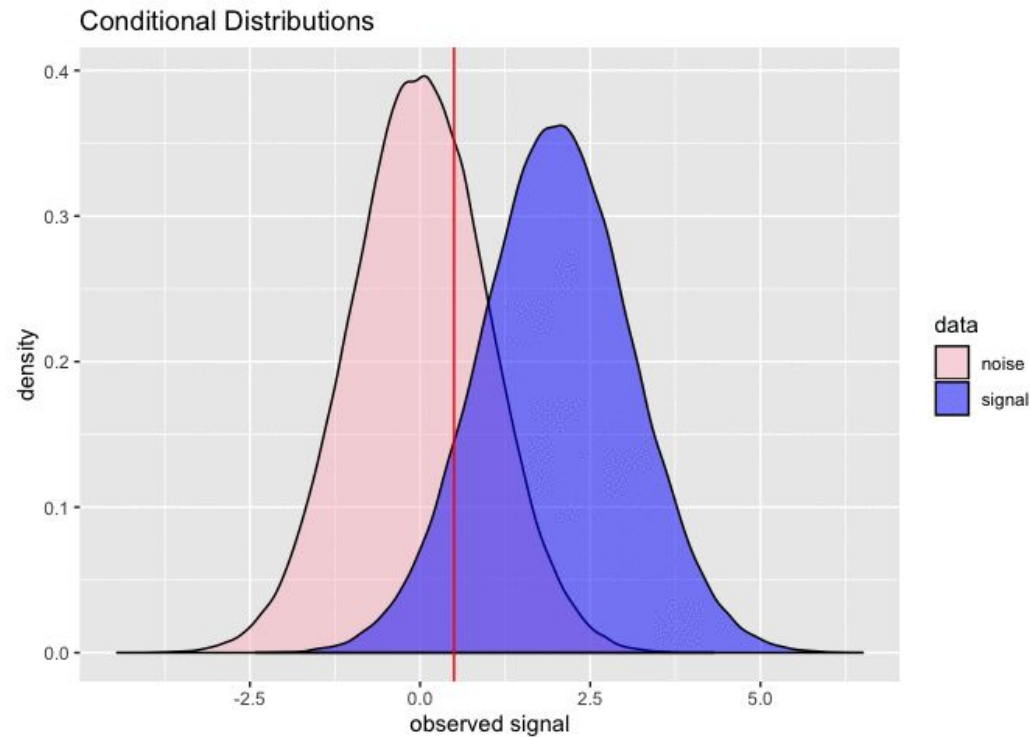
Receiver operating characteristics curve. Making use of true positive rates (TPR) and false positive rates (FPR) and computing them with many different thresholds we can plot them on a single graph.



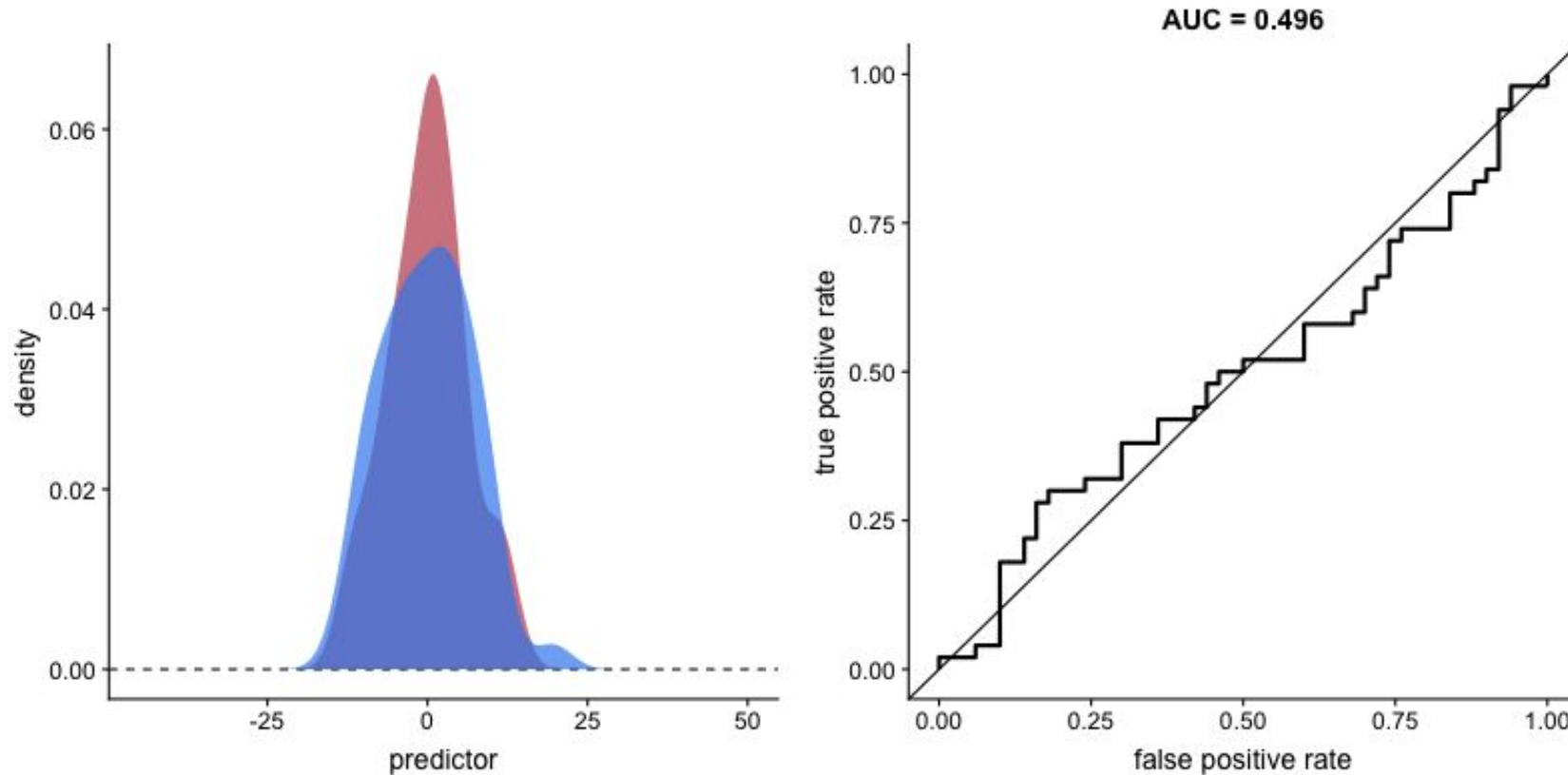
- Blue line assumes random selection. (and balanced dataset)
- Also Good for comparing different classifiers.
- Not Good when large class imbalances.
- Good to answer this question: What TPR can I obtained if I want to fix FPR to 0.02?

1 Metric -> AUR-ROC (Area under Receiver operating characteristics curve)

# Logistic Regression – ROC (threshold).



# Logistic Regression – ROC (compare models).



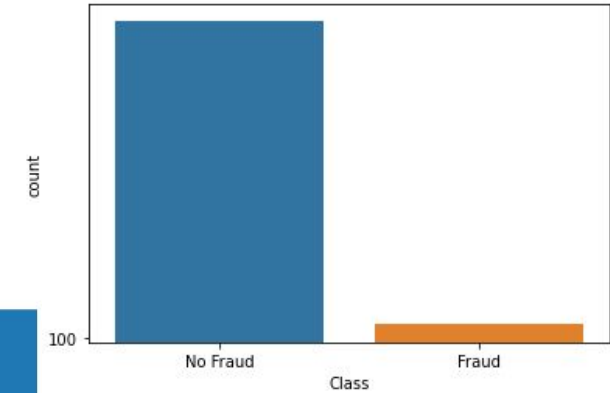
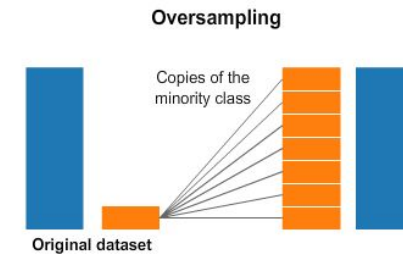
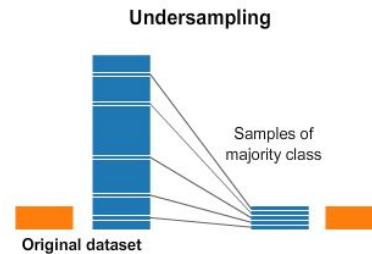
# Class imbalances.

Appear in many domains, such as: Fraud detection, Spam filtering, ...

- Problem! high accuracy just by predicting the **majority class**.

Solutions:

- Resampling Technique.
  - Amount of data.
  - Can cause overfitting (if exact copies)

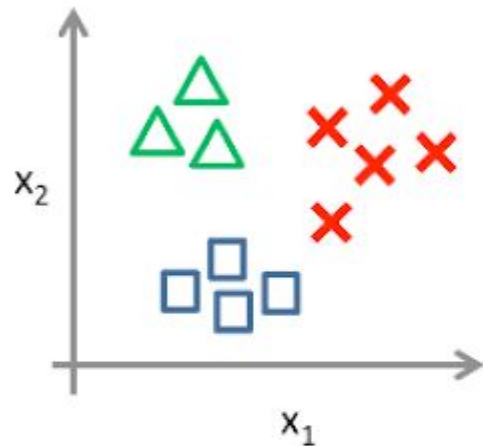


Improvements - Tomek links and SMOTE techniques



# Logistic Regression – More than 2 classes?

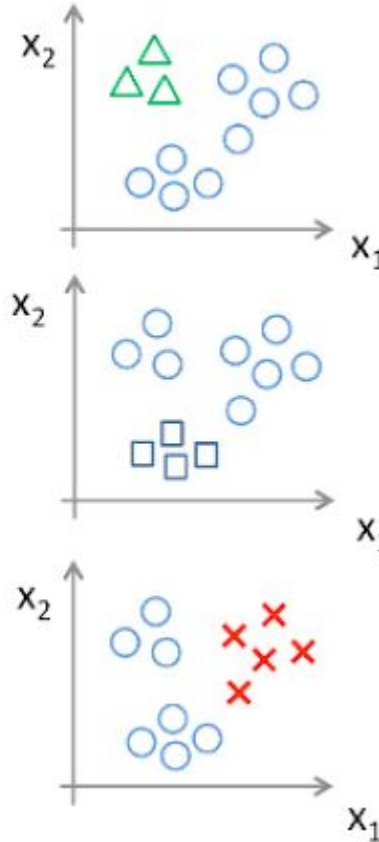
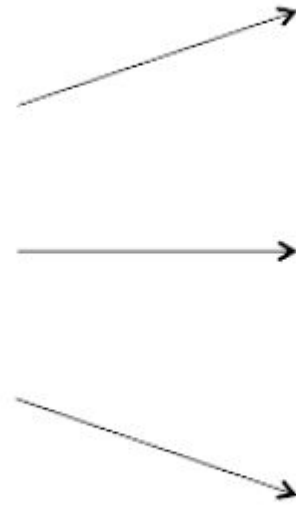
One-vs-all (one-vs-rest):



Class 1: Green

Class 2: Blue

Class 3: Red



If we have  $K$  classes, repeat for each class:

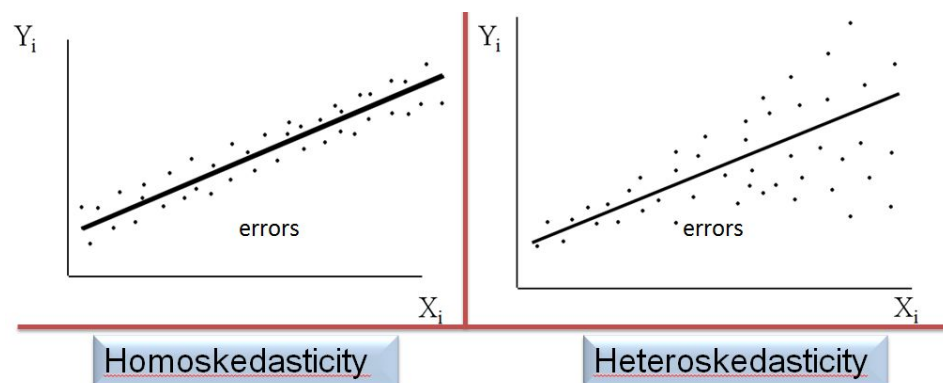
- The class label is set to 1 and the rest to 0.
- Learn a binary classifier.

Run all classifiers and select the one with the highest  $h_{\theta}(x)$ .

Note: softmax function (later on the master).

# Linear regression models - Summary

- Pros:
  - Simple model and interpretability
  - Computationally efficient
- Cons:
  - Overly-Simplistic (linearity assumptions)
  - Affected by Outliers
  - Assumes that the features are not correlated (rare) – multicollinearity!
    - Solution: dimensionality reduction techniques (like PCA).
  - Assumes Homoskedasticity - constant variance around the mean



# Logistic Regression – lab 2: LogReg from scratch

## Python implementation of Logistic Regression from scratch

```
[ ] %matplotlib inline
    from IPython.display import HTML
    import numpy as np
    import matplotlib.pyplot as plt
    import seaborn as sns

[ ] !ls

data.csv sample_data

[ ] import urllib
    urllib.request.urlretrieve("https://github.com/imagra93/MINT-ML/blob/main/data2.csv?raw=true", "data.csv")

('data.csv', <http.client.HTTPMessage at 0x7f3d89cef690>)

[ ] !ls

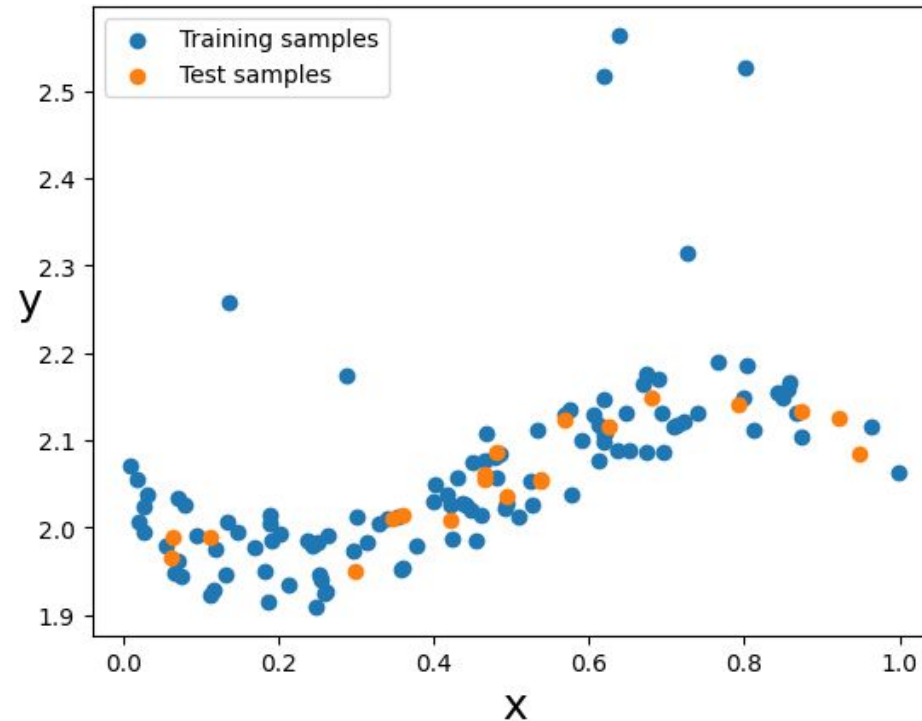
data.csv sample_data
```

### Import the data

Here, we're using a dataset with two columns containing the amount of hours studied and the test scores students achieved, respectively.

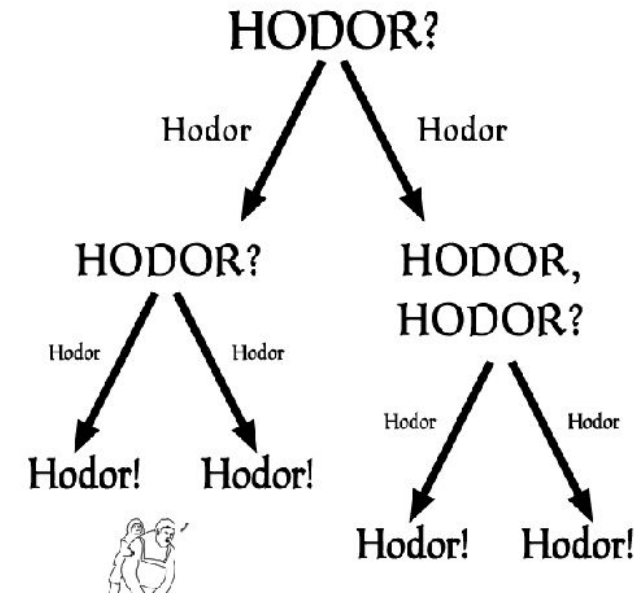
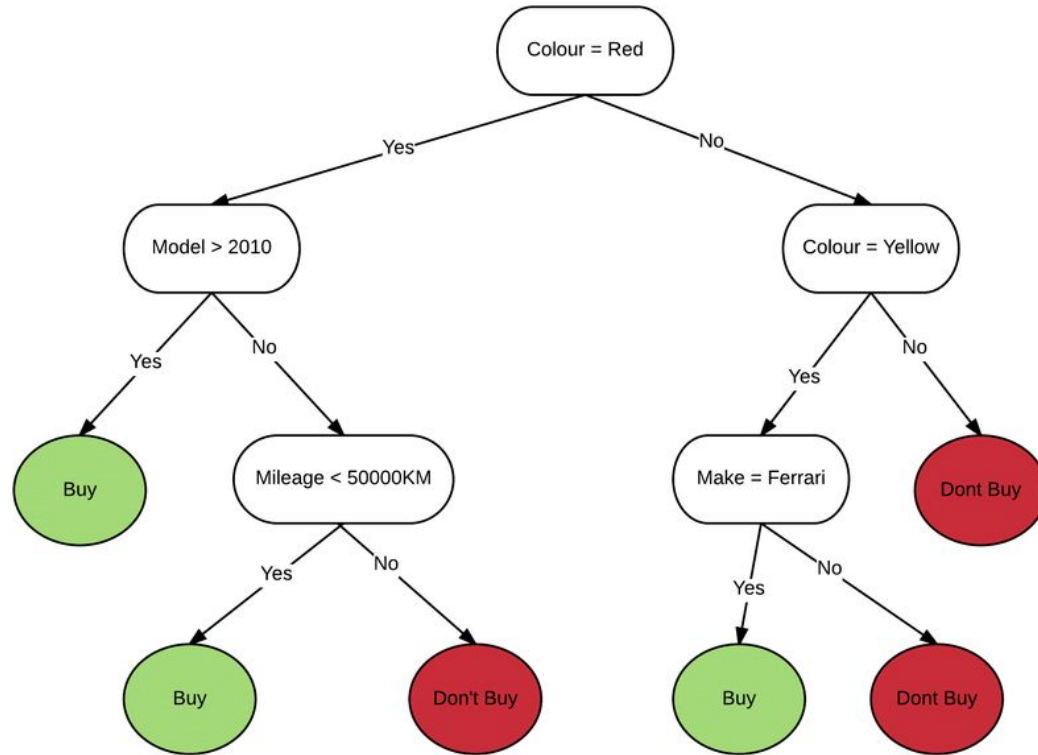
[Lab logistic regression](#)

# Regression – lab 3: Polynomial regression

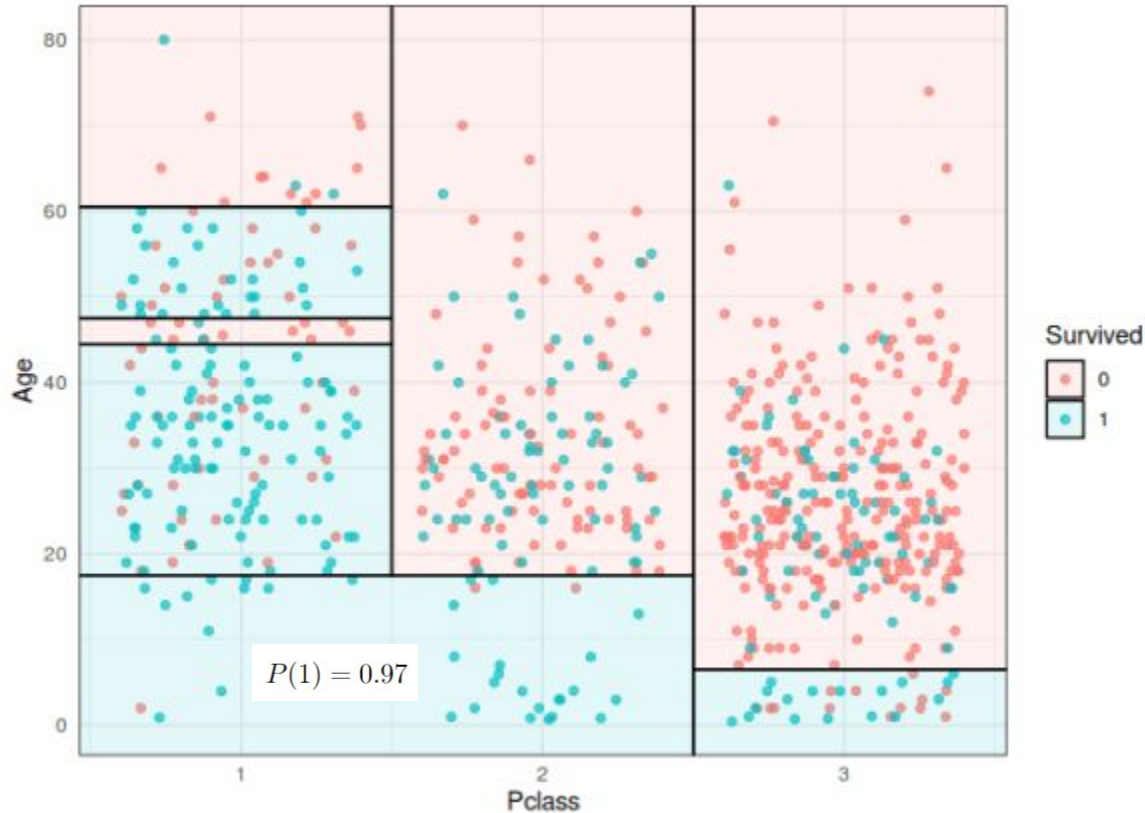


[Polynomial regression](#)

# Decision Trees: Example of Decision Trees.



# Decision Trees: General idea



- Root Node: Entire population
  - Splitting: divides a node into sub-nodes (depends on the variable and its type).
  - Decision node: can be splitted.
  - Leaf node: terminal node.
- 
- Classification -> Probability of each class depending on population of leaf node.
  - Regression -> Take the mean of the leaf population.

How do we design the tree?!

# Decision Trees: Entropy

Entropy measures the **uncertainty** (or impurity) of a set of examples.

For a dataset  $S$  with  $k$  classes:

$$H(S) = - \sum_{i=1}^k p_i \log_2(p_i)$$

- $p_i$  = proportion of examples in class  $i$
- $H(S)$  = entropy of dataset  $S$

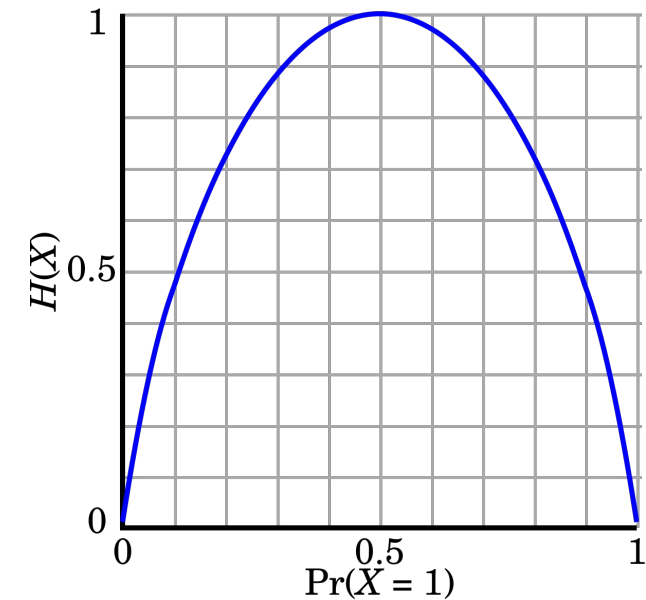
If all examples are in one class:  $H(S) = 0$  (pure).

If examples are evenly split across classes:  $H(S)$  is maximized.

If dataset has 8 Yes and 8 No:

$$p_{\text{Yes}} = \frac{8}{16} = 0.5, \quad p_{\text{No}} = 0.5$$

$$H(S) = -(0.5 \log_2 0.5 + 0.5 \log_2 0.5) = 1$$



# Decision Trees: Information Gain

When splitting on an attribute  $A$ , entropy usually decreases.

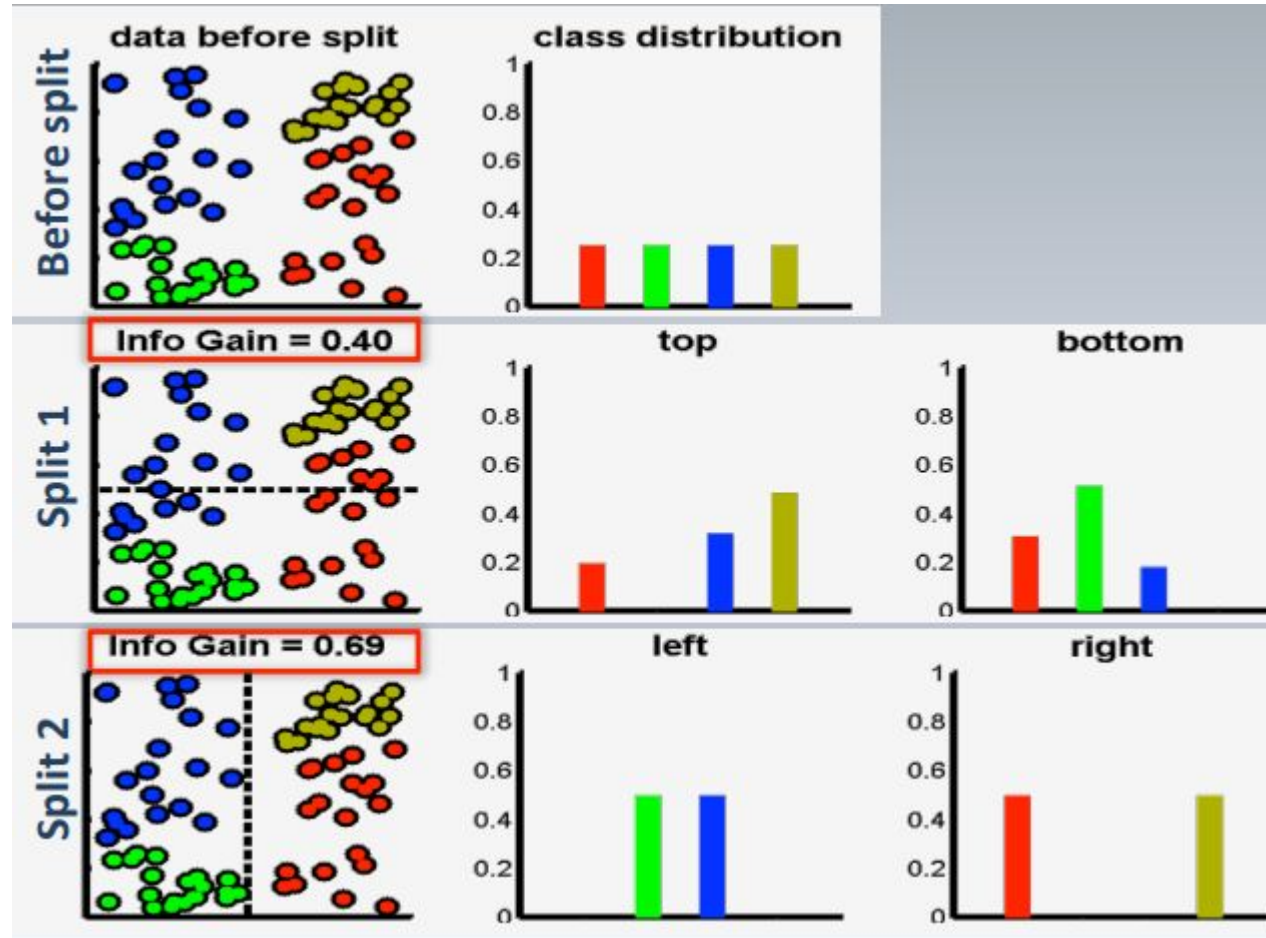
The **information gain (IG)** is the reduction in entropy:

$$IG(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

- $S_v$  = subset of  $S$  where attribute  $A = v$
- Weighted sum = expected entropy after splitting on  $A$
- $IG(S, A)$  = how much information we gain about the class by splitting on  $A$

The decision tree picks the attribute with **highest information gain**.

# Decision Trees: Entropy and Information

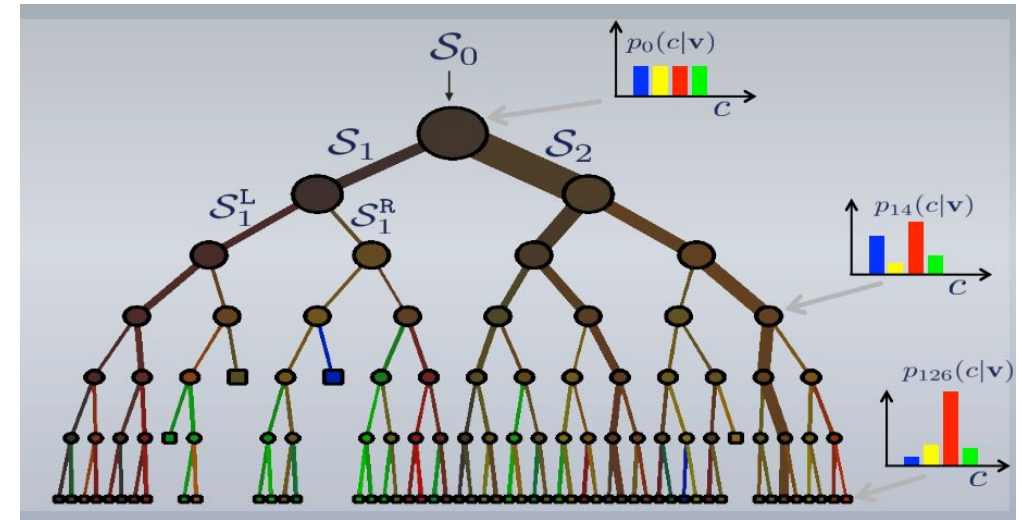


Probabilities or regression values are taken from the samples in the split.

# Decision Trees: Algorithm.

Begin the tree with the root node, says  $S$ , which contains the complete dataset, and then:

- **Step - 1:** Find the best attribute (in terms of Information Gain) in the dataset using.
- **Step - 2:** Divide the  $S$  into subsets that contains possible values for the best attributes.
  - Note: there are other criteria, like Gini Index
- **Step - 3:** Generate the decision tree node, which contains the best attribute.
- **Step - 4:** Recursively make new decision trees using the subsets of the dataset created in step - 2. Continue this process until:
  - all matching records have the same output value
  - if data points are identical on remaining attributes



[https://www2.cs.uregina.ca/~dbd/cs831/notes/ml/dtrees/dt\\_prob1.html](https://www2.cs.uregina.ca/~dbd/cs831/notes/ml/dtrees/dt_prob1.html)

$$Gini = 1 - \sum_{k=1}^K p_k^2$$

$p_k$  is the proportion of samples belonging to class  $k$ .

$K$  is the total number of classes.

# Decision Trees: Pros and Cons.

## Advantages:

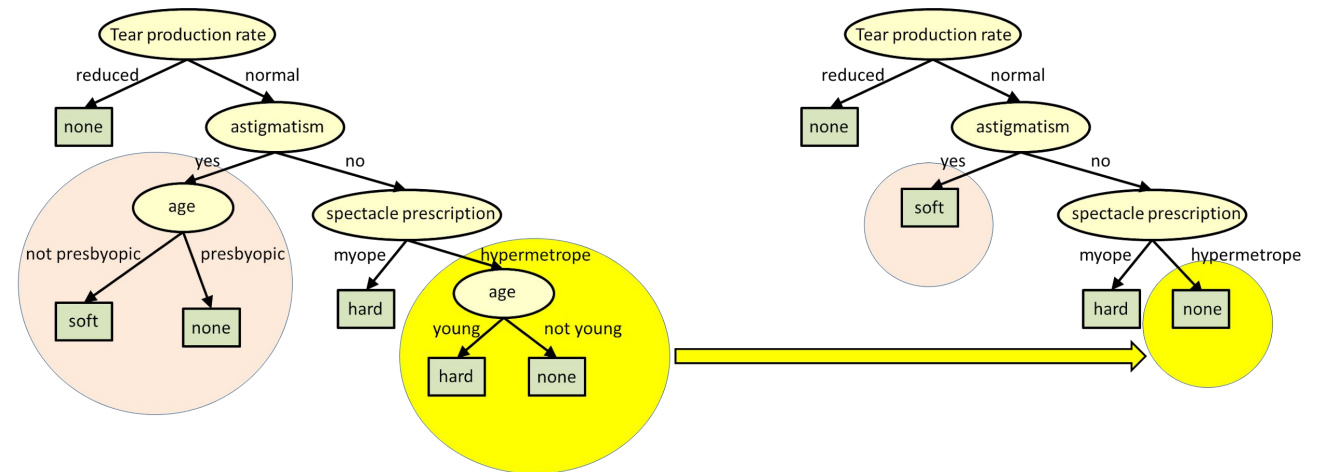
- There are no assumptions about the spatial distribution and the classifier structure.
- Compared to other algorithms decision trees requires less effort for data preparation during pre-processing.
- A decision tree does not require normalization of data.
- It can capture nonlinear relationships
- A Decision tree model is very intuitive and easy to explain to technical teams as well as stakeholders.

## Disadvantage:

- A small change in the data can cause a large change in the structure of the decision tree causing instability.
- Decision tree often involves higher time to train the model.
- Are very prone to overfitting:
  - With “m-1” splits you learn your data.

# Decision Trees: Regularization.

- Define a maximum depth (lower than  $m-1$ )
  - Many use 10 as an absolute maximum.
- Do not split if the information gain is less than a threshold.
- Pruning. This makes the model more understandable to the user and, perhaps, more accurate on fresh data that was not used to train the classifier. For example:
  - While the number of leaves in the tree exceeds the desired number: Find the twig with the least Information Gain.
  - Prune the tree to maximize classification performance on a validation set.



# Ensemble methods – idea!

Class exercise:  
How much does it cost?  
Give independent answers.



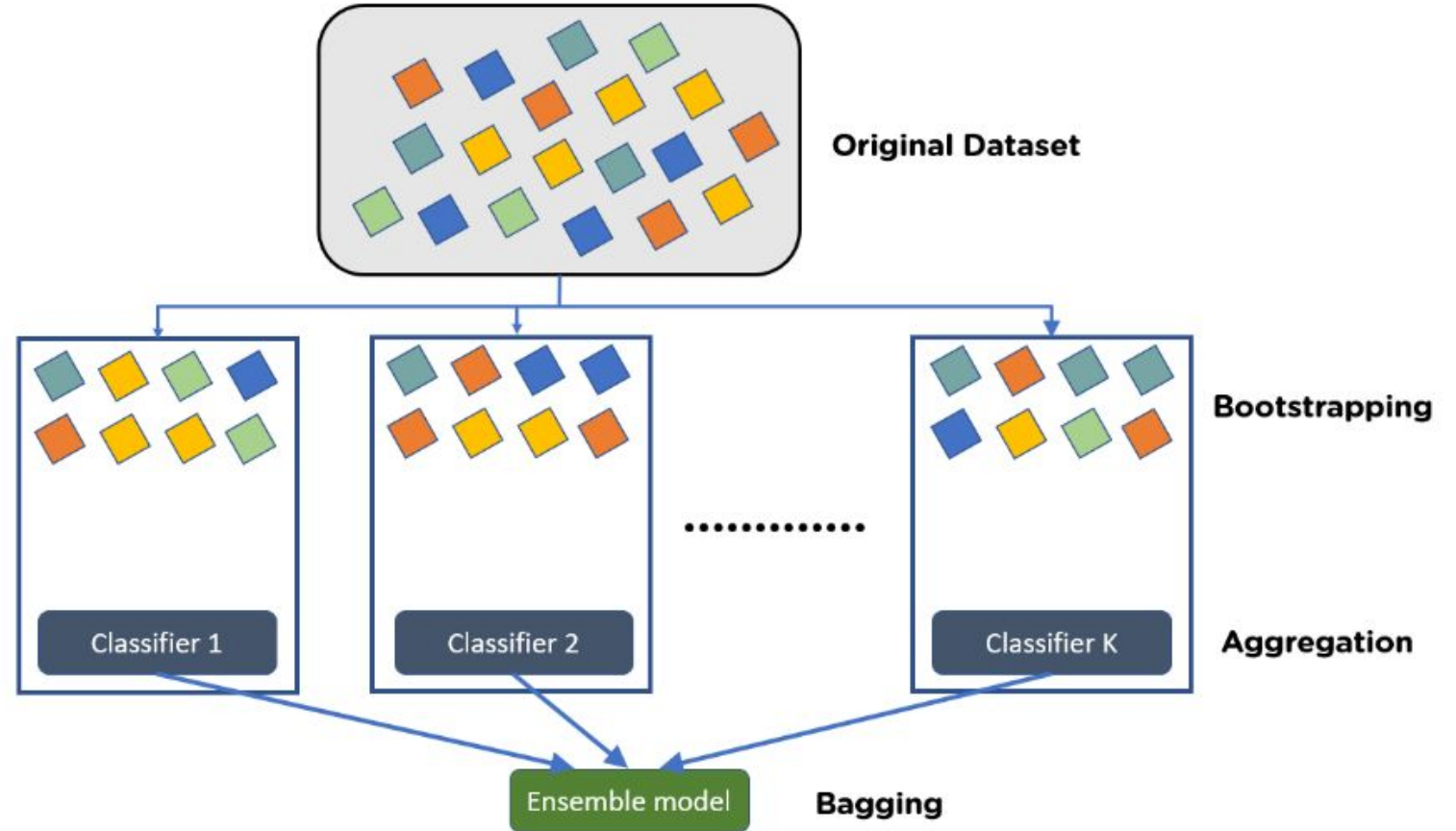
39€

Averaging reduces variance (if independent):

$$\text{Var}(\hat{f}_{\text{bag}}(x)) = \frac{1}{M} \text{Var}(f(x))$$

# Ensemble methods – Bagging

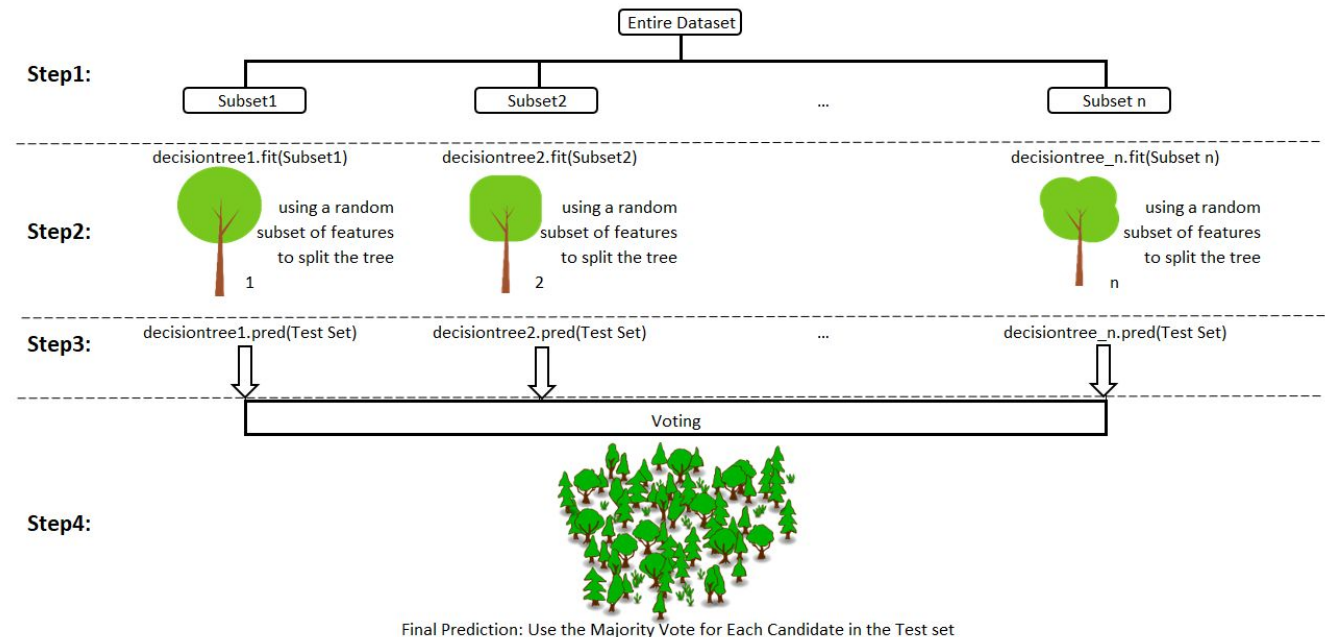
- Train K classifiers in K sub-samples of the dataset.
- Aggregate the output.
- Good global model.



# Ensemble methods – Random Forest

Random Forests contain N different decision “weaker” trees.

- **Step - 1:** Divide the dataset into N subsets.
- **Step -2:** Train (in parallel) the decision trees associated with the selected data points and a subset of the features.
- **Step - 3:** For new data points, find the predictions of each decision tree.
- **Step - 4:** Ensemble the results from all trees. (Majority vote)

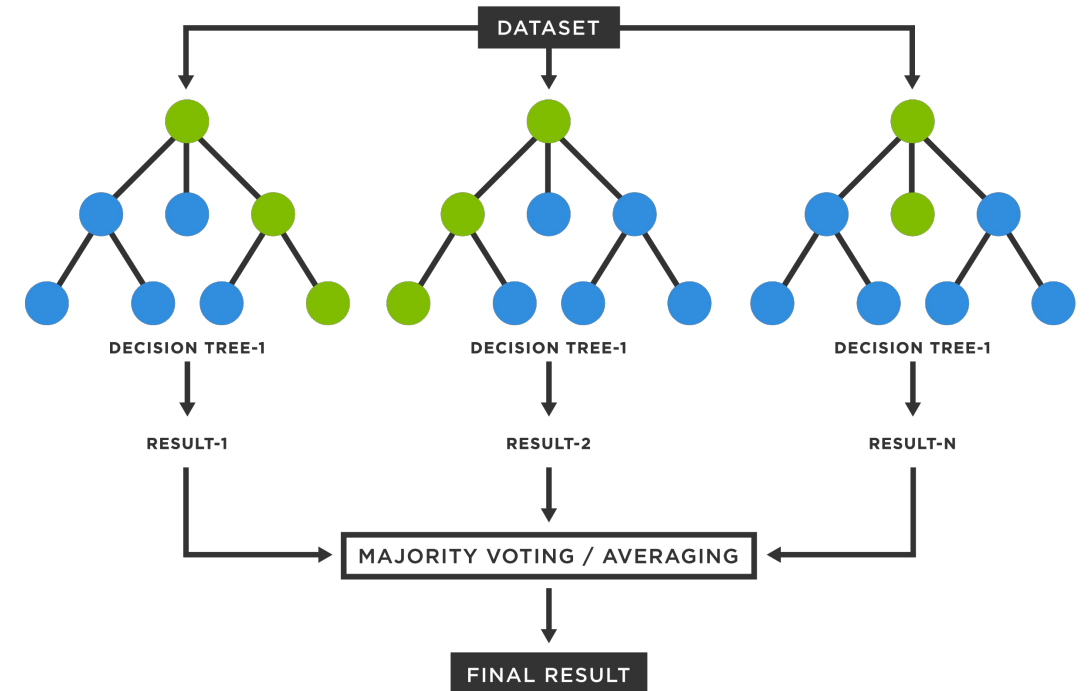


Many ensemble methods:

- Average (regression).
- Mix in probability.

# Random Forest – Review

- Pros:
  - They reduce the overfitting risks of decision trees.
  - They achieve very good accuracy on classification tasks.
  - Very used!
  - Work well on non-linear data
  - Robust to outliers (many trees will not learn that).
- Cons:
  - Slow training.
  - Less interpretable than decision trees.
  - Not so suitable for very sparse features.



# Random Forest – Lab

## ▼ Decision Trees and Random Forests

*Random forests* are a powerful non-parametric algorithm consisting on an *ensemble* of simpler estimators (decision trees).

The idea is that the sum can be greater than the parts: that is, a majority vote among a number of estimators can end up being better than any of the individual estimators.

We will see examples of this in the following sections.

Let us begin by import and defining some usefull modules and functions.

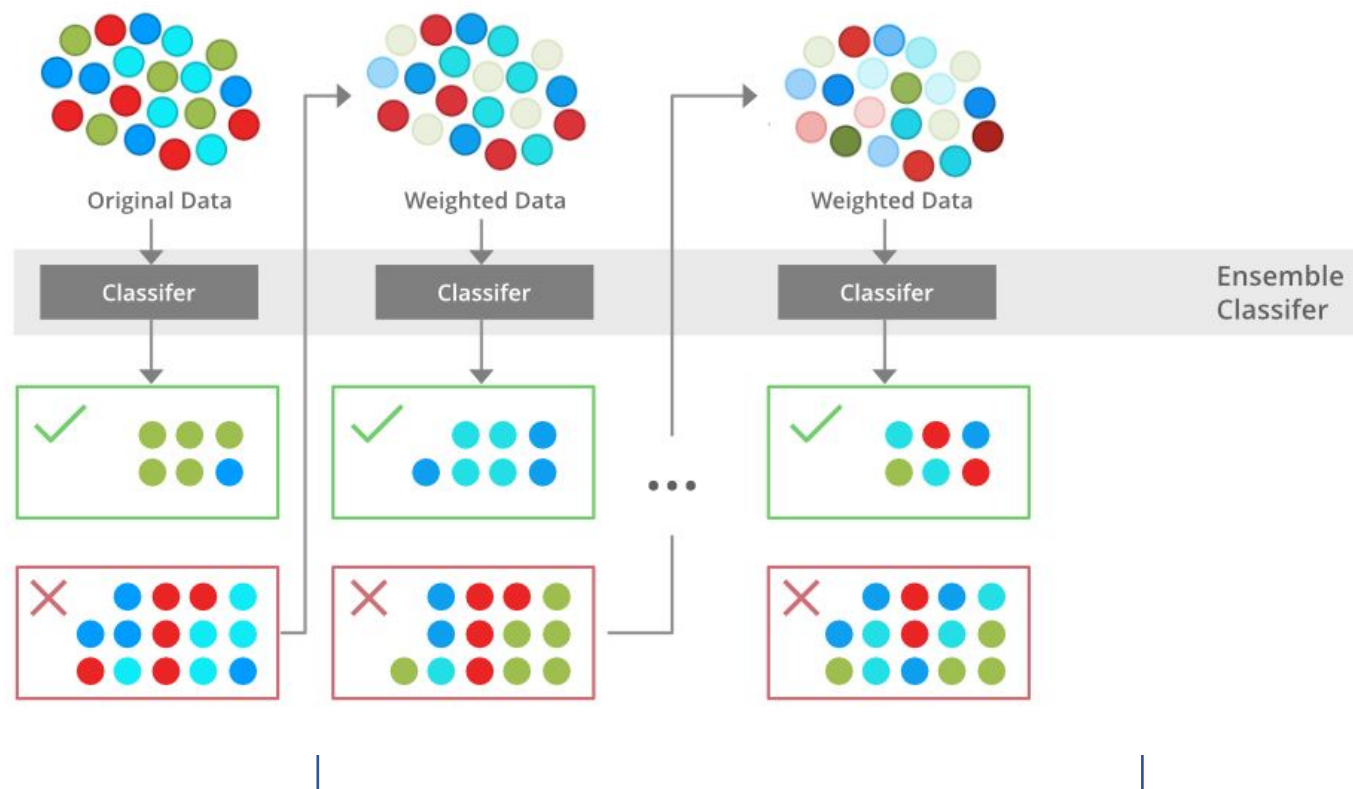
[Lab Trees](#)

[Trees from scratch](#)

Test material ends here.

# Boosting

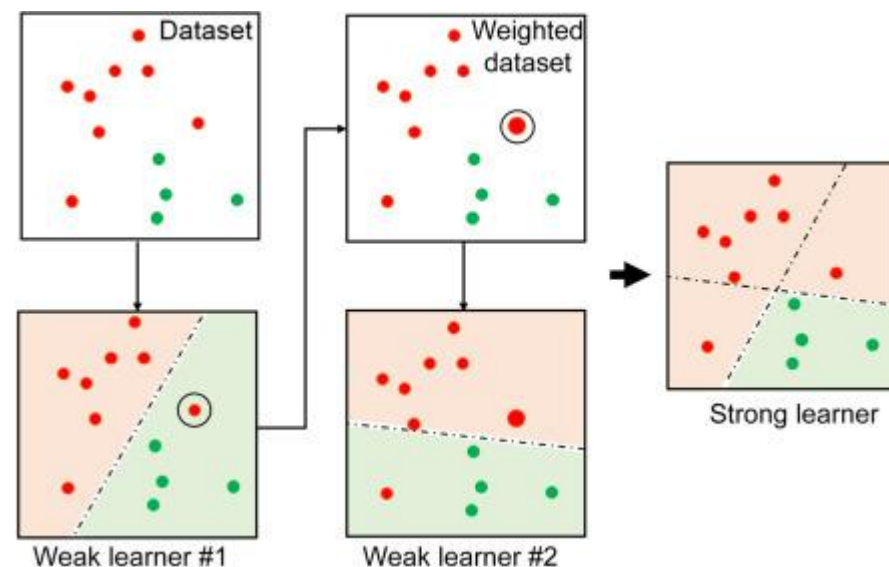
- Bagging:** Training a bunch of individual models in a parallel way. Each model is trained by a random subset of the data
- Boosting:** Training a bunch of individual models in a sequential way. Each individual model learns from mistakes made by the previous model.



# AdaBoost

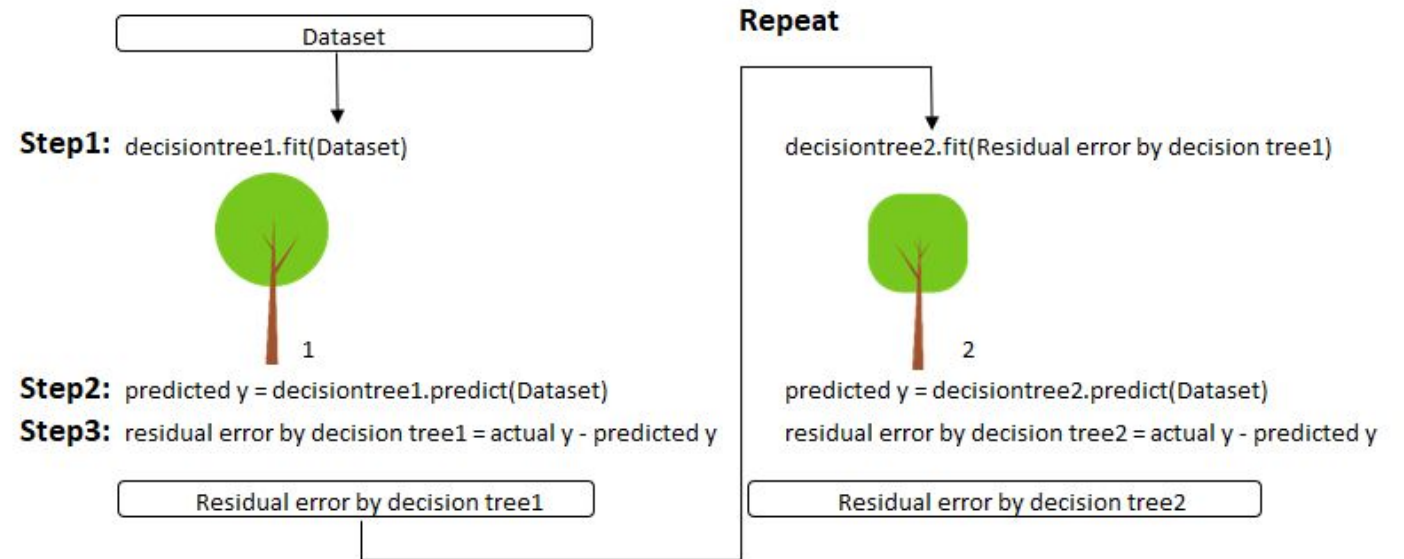
## Key Ideas in AdaBoost:

- **Weak Learners:** Typically, AdaBoost uses decision stumps (trees with a single split) as weak learners.
- **Weights on Data Points:** In AdaBoost, each training example starts with equal weight. After each weak learner is trained, the weights of the incorrectly classified examples are **increased**, making the next weak learner focus more on the hard-to-classify points.
- **Weights on Models:** Each weak learner is assigned a weight based on its performance, and the final prediction is a weighted vote of all weak learners.



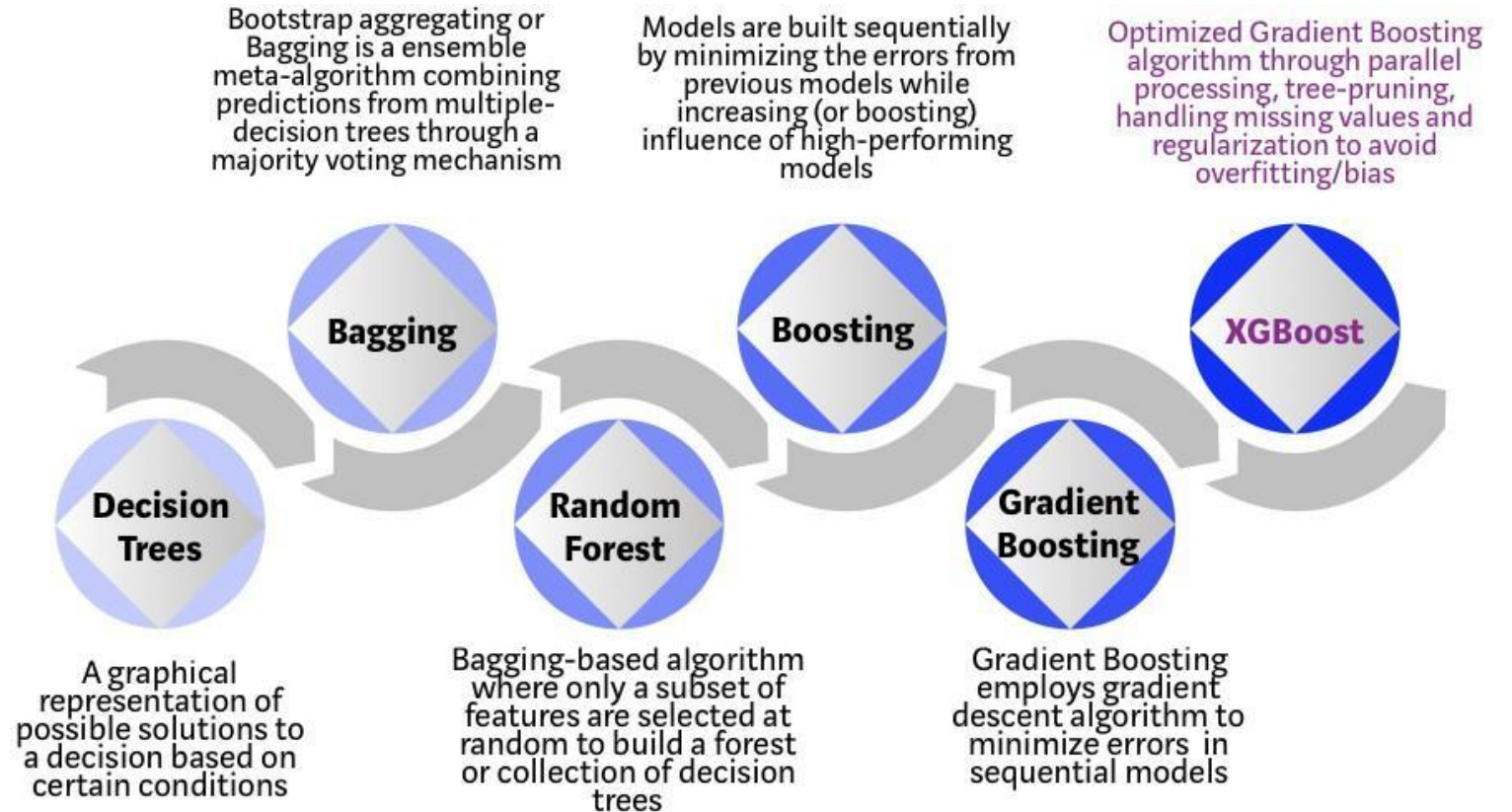
# Gradient Boosting

- Gradient Boosting learns from the mistake — residual error directly, rather than update the weights of data points.



# XGBoost

- Extremely efficient and powerful for tabular data.
- Is used in many Kaggle winning solutions.
- Not so good for formatted data (sequences, images, etc....)
- For those cases Deep Learning performs the best!



# XGBoost - Lab

→ XGBoost: Examples for Regression and Classification.

→ Imports & Usefull material

We will the use of XGBoost and its integration with Scikit-Learn.

Some useful links:

- [XGBoost documentation](#)
- [Parameters](#)
- [Python package](#)

Objective is to demonstrate:

• regression /

[Lab XGBoost and miscellaneous](#)

# Support Vector Machine (SVM): Examples

## Facial Expression Classification using SVM



Happy

Sad

Surprised

Angry

## annietk/Breast-Cancer- Classification-using-SVM

Predicting if the cancer diagnosis is benign or malignant based on several observations/features



1

Contributor

0

Issues

1

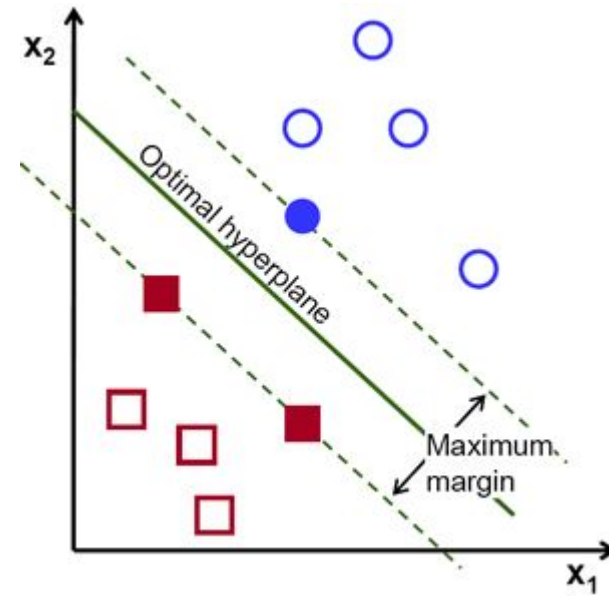
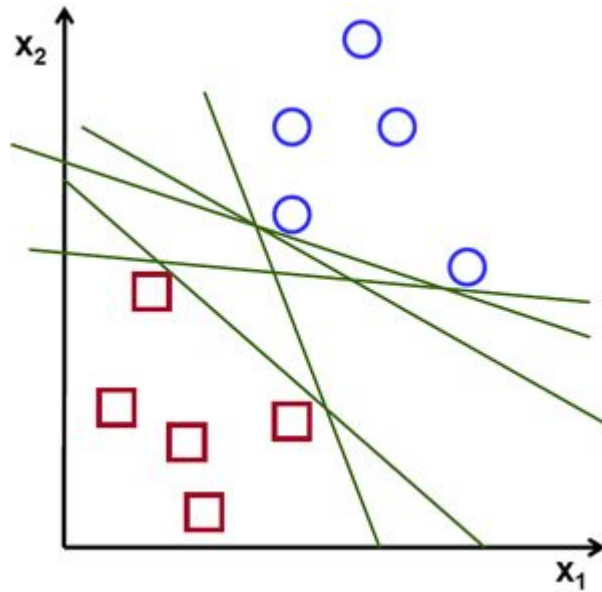
Star

0

Forks

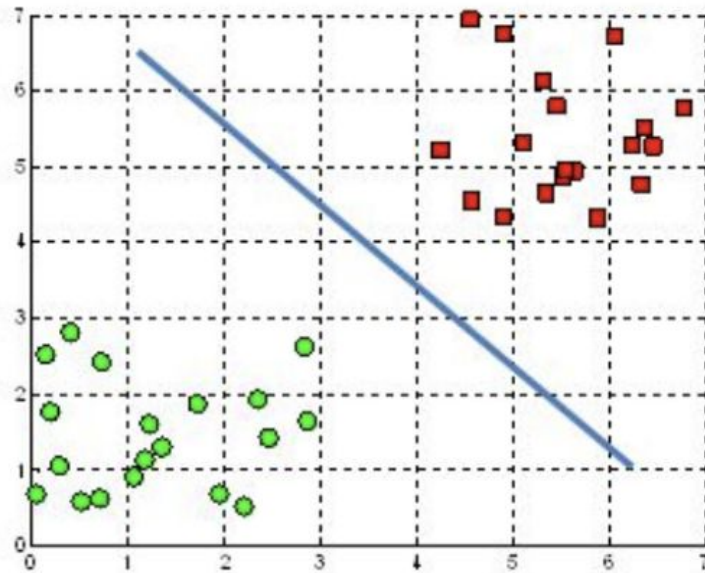


# Support Vector Machine (SVM): Motivation

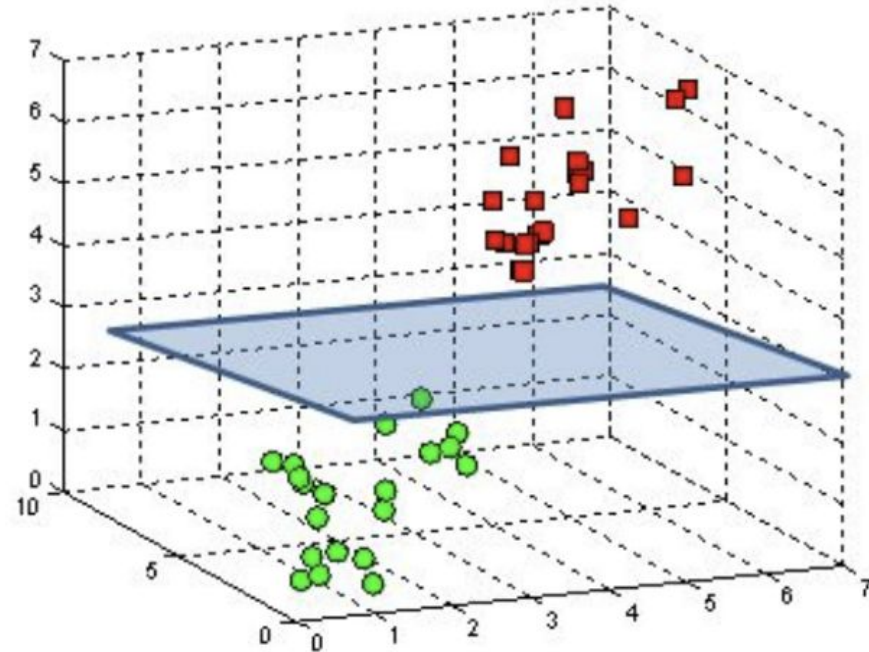


# Support Vector Machine(SVM): Motivation

A hyperplane in  $\mathbb{R}^2$  is a line

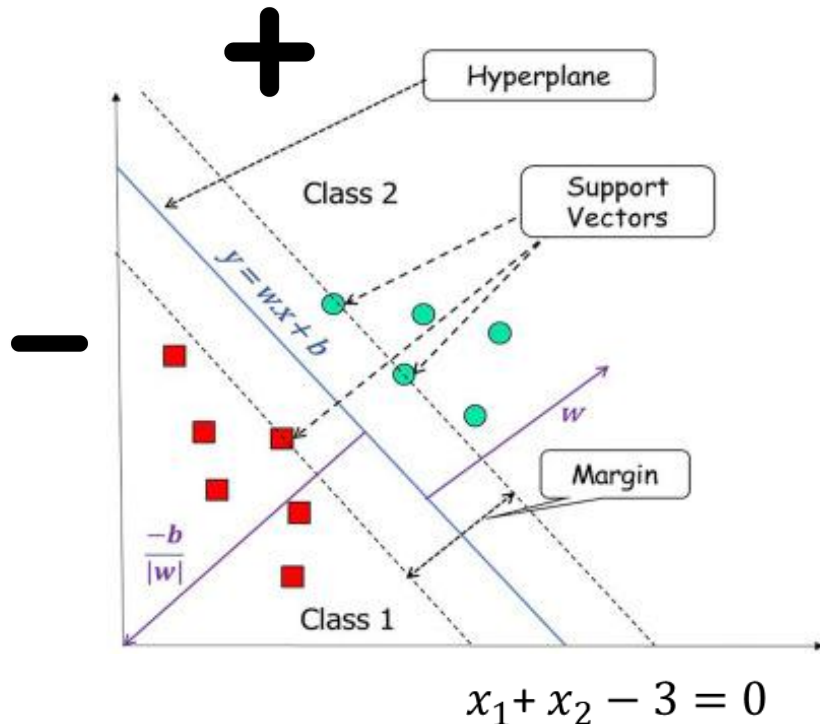


A hyperplane in  $\mathbb{R}^3$  is a plane



# Support Vector Machine(SVM): Fundamentals

Suppose  $x \in R^n$ , and  $w \in R^n$  are the system's parameters. We can define an hyperplane as:



$$w \cdot x + b = 0$$

$$w_1 \cdot x_1 + w_2 \cdot x_2 + b = 0$$

Note: 1 equation, 2 independent variables!

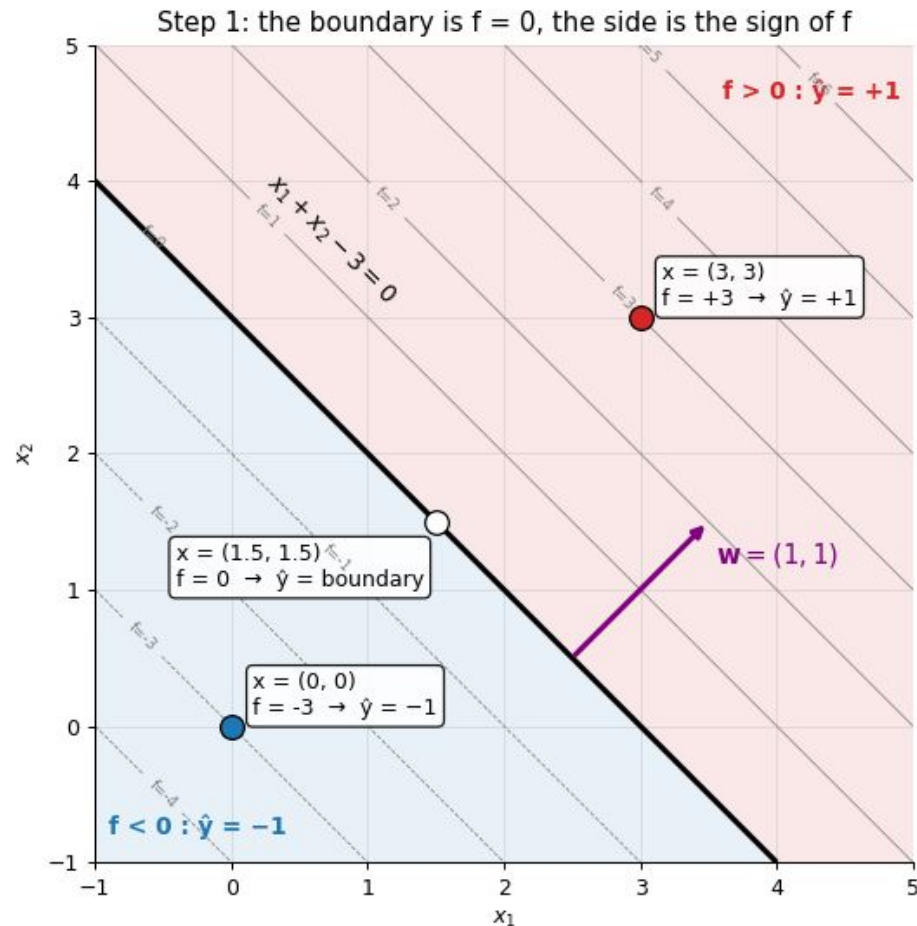
$$\text{Decision boundary} = \begin{cases} 1 & \text{if } w \cdot x + b > 0 \\ -1 & \text{if } w \cdot x + b < 0 \end{cases}$$

Given  $w = (1,1)$ ,  $b = -3$  the output for and  $x^{(1)} = (3,3)$  are:  $x^{(2)} = (0,0)$

$$x = (3,3) \rightarrow 3$$

$$x = (0,0) \rightarrow -3$$

# Support Vector Machine(SVM): Fundamentals



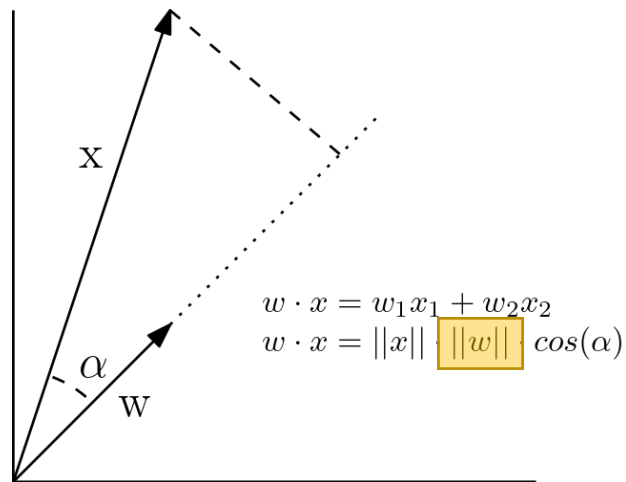
The hyperplane cuts the space in two halves, and the **sign** of  $f$  tells us which half a point is in:

$$\hat{y} = \begin{cases} +1 & \text{if } \mathbf{w}^\top \mathbf{x} + b > 0 \\ -1 & \text{if } \mathbf{w}^\top \mathbf{x} + b < 0 \end{cases}$$

**Worked example.**  $\mathbf{w} = (1, 1)$ ,  $b = -3$ , i.e. the line  $x_1 + x_2 - 3 = 0$ :

| point                       | $f(\mathbf{x}) = x_1 + x_2 - 3$ | prediction      |
|-----------------------------|---------------------------------|-----------------|
| $\mathbf{x}^{(1)} = (3, 3)$ | +3                              | +1              |
| $\mathbf{x}^{(2)} = (0, 0)$ | -3                              | -1              |
| (1.5, 1.5)                  | 0                               | on the boundary |

# Support Vector Machine(SVM): Dot Product



It has two equivalent forms:

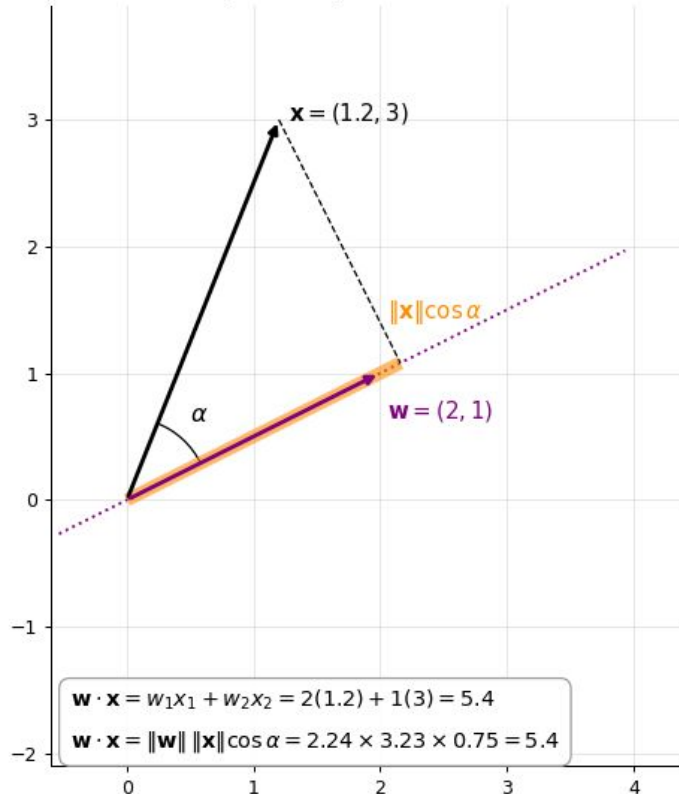
$$\mathbf{w}^\top \mathbf{x} = w_1 x_1 + \dots + w_n x_n = \|\mathbf{w}\| \|\mathbf{x}\| \cos \alpha, \quad \|\mathbf{w}\| = \sqrt{w_1^2 + \dots + w_n^2} = \sqrt{\sum_{j=1}^n w_j^2}.$$

**For this purpose  $\mathbf{w}$  is usually normalised.** The part we care about is  $\|\mathbf{x}\| \cos \alpha$ , the length of the **projection** (the "shadow") of  $\mathbf{x}$  on  $\mathbf{w}$  (figure below, left). The factor  $\|\mathbf{w}\|$  only gets in the way, so we divide it out and use the unit vector

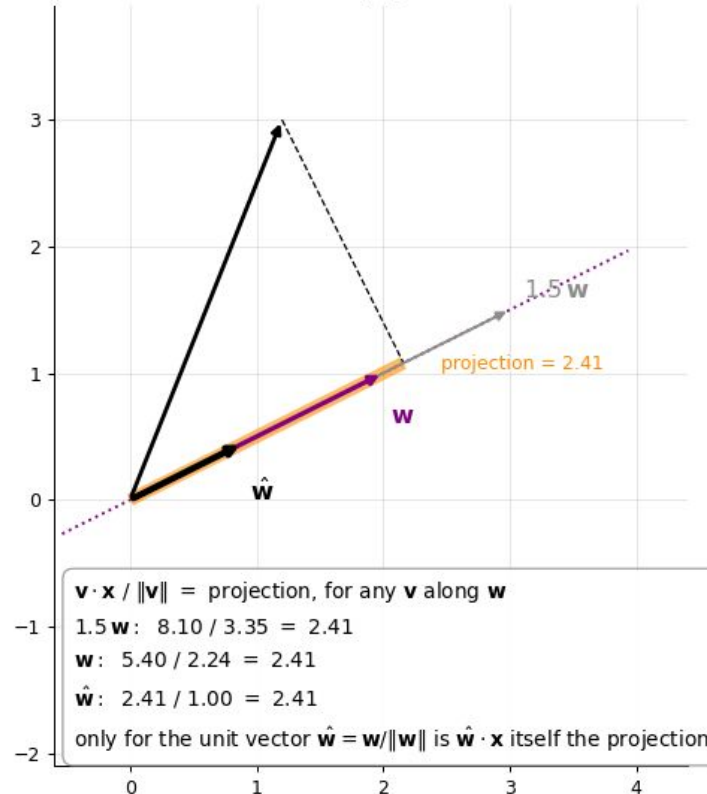
$$\hat{\mathbf{w}} = \frac{\mathbf{w}}{\|\mathbf{w}\|} \quad \Rightarrow \quad \hat{\mathbf{w}}^\top \mathbf{x} = \frac{\mathbf{w}^\top \mathbf{x}}{\|\mathbf{w}\|} = \|\mathbf{x}\| \cos \alpha = \text{projection of } \mathbf{x} \text{ on } \mathbf{w}$$

# Support Vector Machine(SVM): Dot Product

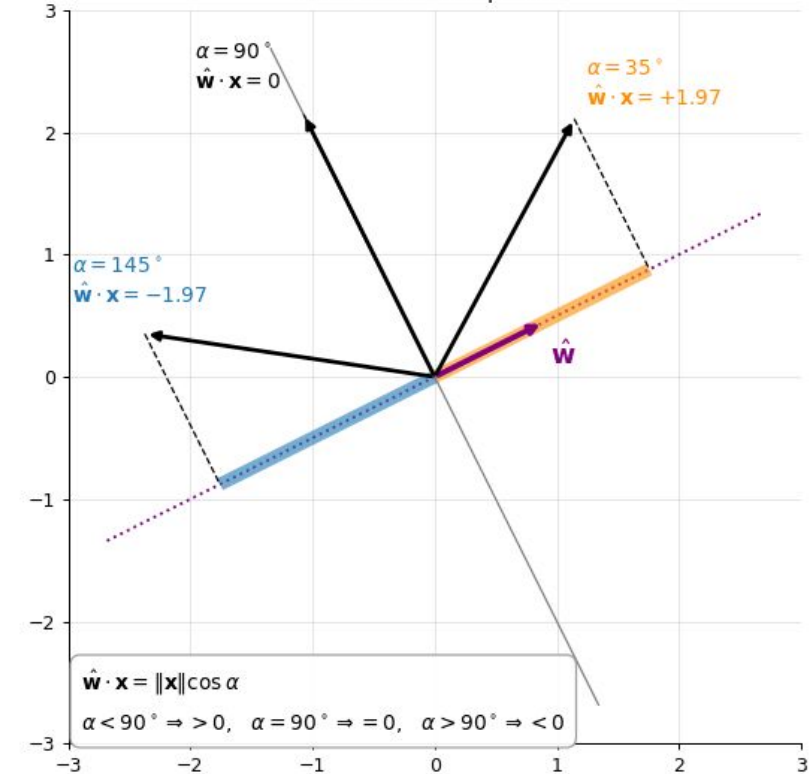
Two ways to compute the same number



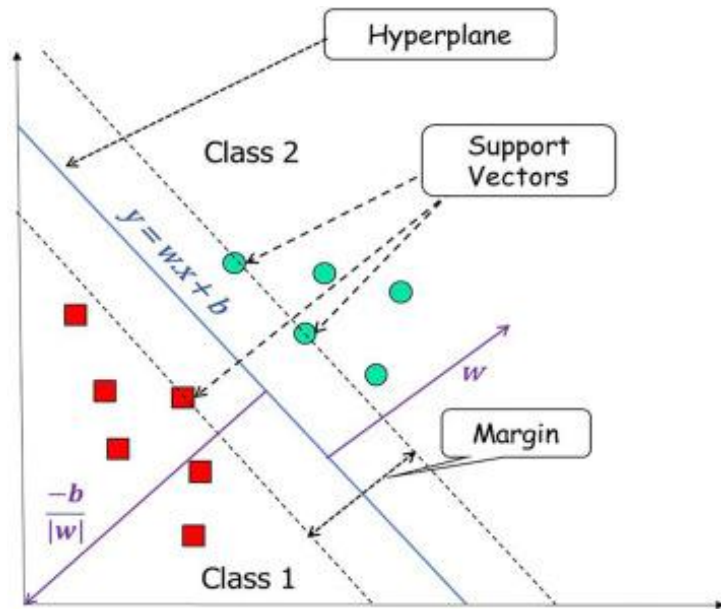
Why we normalise: the length of  $\mathbf{w}$  rescales  $\mathbf{w} \cdot \mathbf{x}$ , not the projection



The sign of the projection says on which side of the line  $\perp \mathbf{w}$  the point is



# Support Vector Machine(SVM): Margin



$$\text{Decision boundary} = \begin{cases} 1 & \text{if } w \cdot x + b > 1 \\ -1 & \text{if } w \cdot x + b < -1 \end{cases}$$

The goal is to find  $w$  and  $b$  (the hyperplane), such that:

$$\begin{cases} w \cdot x + b > 1, & \forall i \text{ that } y^{(i)} = 1 \\ w \cdot x + b < -1, & \forall i \text{ that } y^{(i)} = -1 \end{cases}$$

Or equivalently (and only for support vectors):

$$y^{(i)} \cdot (w \cdot x^{(i)} + b) - 1 = 0$$

$w$  of any scale

# Support Vector Machine(SVM): Margin

How do we measure the margin? Projecting!

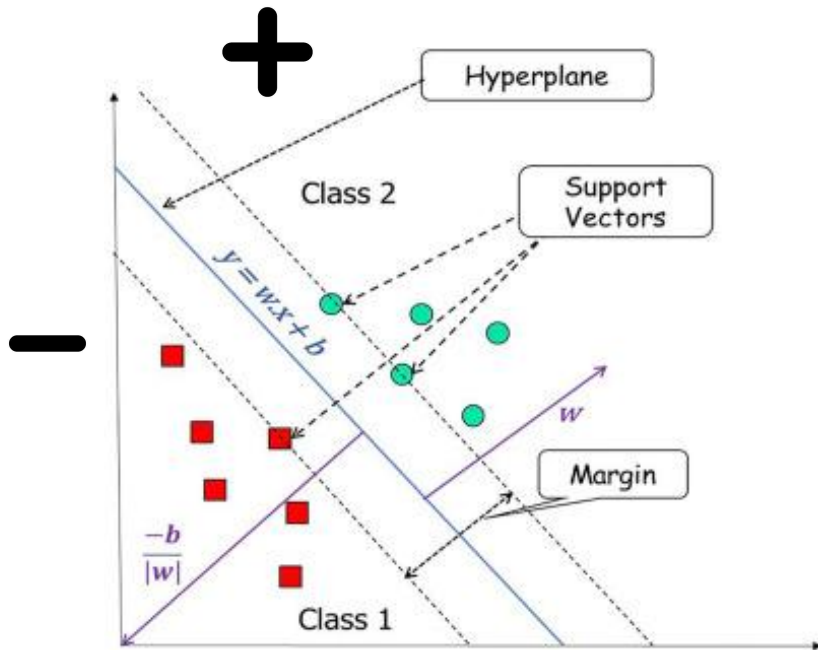
$$\left. \begin{aligned} d_+ &= \frac{w \cdot x_+}{\|w\|} \\ d_- &= \frac{w \cdot x_-}{\|w\|} \end{aligned} \right\} d = d_+ - d_- = \frac{1}{\|w\|} \cdot (w \cdot x_+ - w \cdot x_-).$$

Distance between two points of opposite site.

And for the particular case of the support vectors:

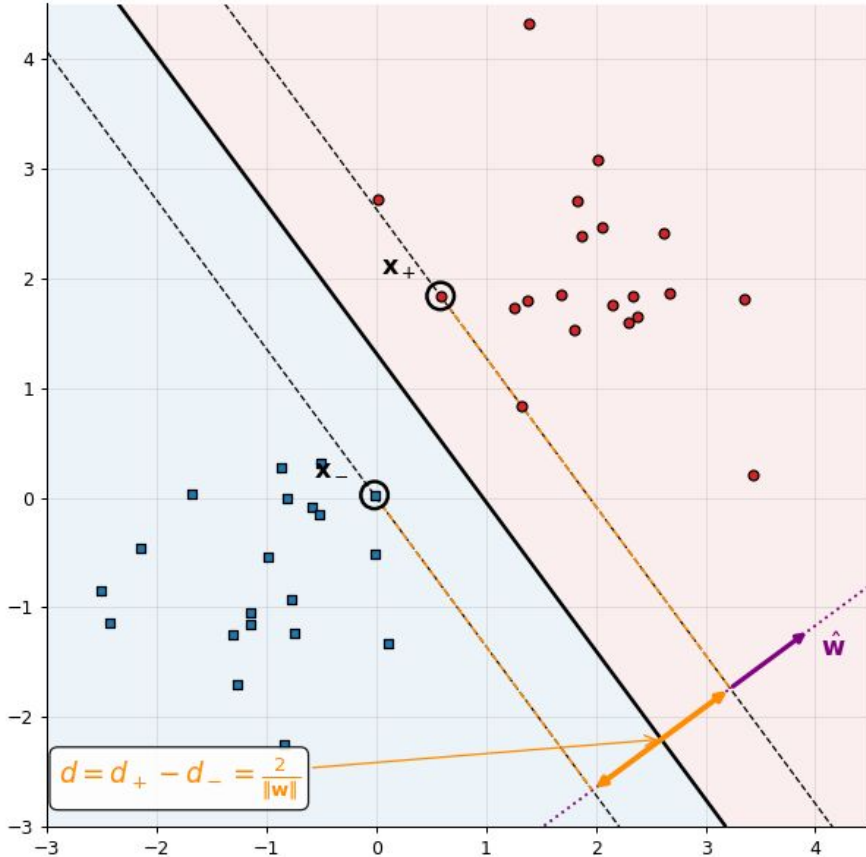
$$\begin{aligned} d &= \frac{1}{\|w\|} \cdot (w \cdot x_+ - w \cdot x_-) \\ &= \frac{1}{\|w\|} \cdot (1 - b + 1 + b) \\ &= \frac{2}{\|w\|} \end{aligned}$$

$$y^{(i)} \cdot (w \cdot x^{(i)} + b) - 1 = 0$$



# Support Vector Machine(SVM): Margin

Step 4: project the two support vectors onto the  $w$  direction and subtract



How do we measure the margin? Projecting!

$$\left. \begin{aligned} d_+ &= \frac{w \cdot x_+}{\|w\|} \\ d_- &= \frac{w \cdot x_-}{\|w\|} \end{aligned} \right\} d = d_+ - d_- = \frac{1}{\|w\|} \cdot (w \cdot x_+ - w \cdot x_-).$$

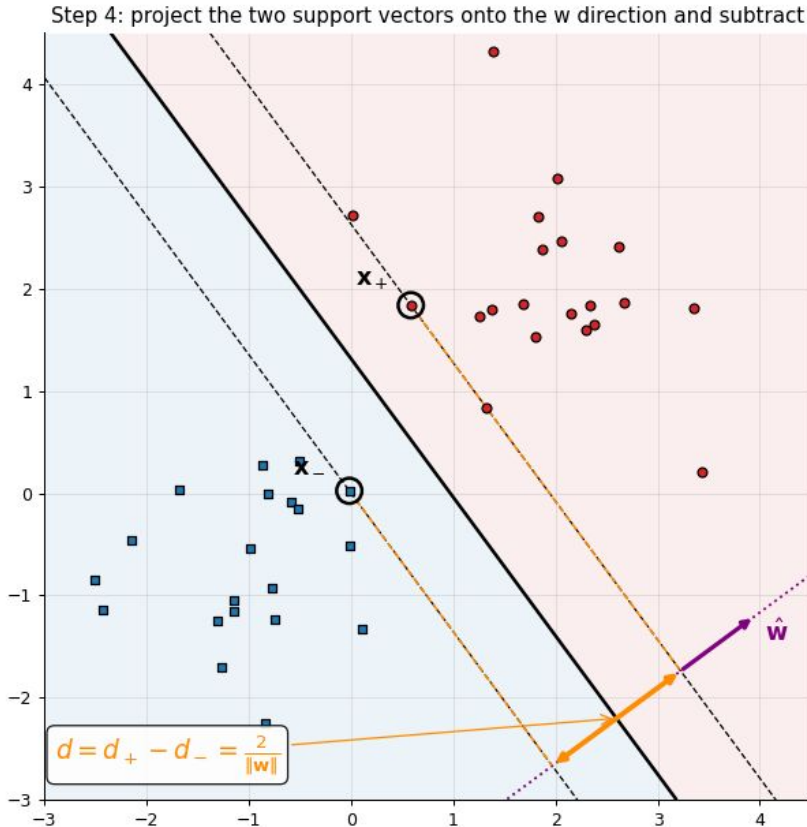
Distance between two points of opposite site.

And for the particular case of the support vectors:

$$\begin{aligned} d &= \frac{1}{\|w\|} \cdot (w \cdot x_+ - w \cdot x_-) \\ &= \frac{1}{\|w\|} \cdot (1 - b + 1 + b) \\ &= \frac{2}{\|w\|} \end{aligned}$$

$$y^{(i)} \cdot (w \cdot x^{(i)} + b) - 1 = 0$$

# Support Vector Machine(SVM): Optimization



Can be solved with  
Lagrange Multipliers!!!

$$\max_{w,b} \frac{2}{\|w\|}$$

subject to  $y^{(i)} (w \cdot x^{(i)} + b) - 1 \geq 0, \forall i$

$$\min_{w,b} \frac{1}{2} \|w\|^2 = \frac{1}{2} w^T w$$

subject to  $y^{(i)} (w \cdot x^{(i)} + b) - 1 \geq 0, \forall i$  Only some will affect!

This optimization problem is convex and can be solved using commercial quadratic programming (QP) software!!

$$\min_{w,b} \frac{1}{2} \|w\|^2 = \frac{1}{2} w^T w$$

subject to  $y^{(i)} (w \cdot x^{(i)} + b) - 1 = 0, \text{ for support vectors}$

# Support Vector Machine(SVM): Optimization

To maximize the margin, we minimize the norm of  $\mathbf{w}$ , which gives the following optimization problem:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2$$

subject to the constraints:

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 \quad \forall i$$

## Optimization with inequality constraints

We call the problem as Primal Optimization Problem, and to solve it, we first construct a generalized Lagrangian.

Optimization problem:

$$\begin{aligned} \min_w \quad & f(w) \\ \text{s.t.} \quad & h_i(w) = 0 \\ & g_i(w) \leq 0 \end{aligned}$$

Generalized Lagrangian:

$$\mathcal{L}(w, \alpha, \beta) = f(w) + \sum \alpha_i g_i(w) + \sum \beta_i h_i(w)$$

# Support Vector Machine(SVM): Optimization

To maximize the margin, we minimize the norm of  $\mathbf{w}$ , which gives the following optimization problem:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2$$

subject to the constraints:

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 \quad \forall i$$

The Lagrangian for the primal problem is:

$$L(\mathbf{w}, b, \alpha) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^n \alpha_i [y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1]$$

where  $\alpha_i \geq 0$  are the **Lagrange multipliers**.

The goal is to minimize the Lagrangian with respect to  $\mathbf{w}$  and  $b$ , and to maximize it with respect to the non-negative Lagrange multipliers  $\alpha_i$ .

# Support Vector Machine(SVM): Optimization

**Minimizing with respect to  $\mathbf{w}$ :**

The term involving  $\mathbf{w}$  in the Lagrangian is:

$$\frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^n \alpha_i y_i \mathbf{w}^T \mathbf{x}_i$$

Taking the derivative with respect to  $\mathbf{w}$  and setting it to zero:

$$\frac{\partial L}{\partial \mathbf{w}} = \mathbf{w} - \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i = 0$$

This gives:

$$\mathbf{w} = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i$$

This is a key result: the optimal weight vector  $\mathbf{w}$  is a linear combination of the training points  $\mathbf{x}_i$ , weighted by  $\alpha_i y_i$ .

# Support Vector Machine(SVM): Optimization

**Minimizing with respect to  $b$ :**

Taking the derivative of the Lagrangian with respect to  $b$  and setting it to zero:

$$\frac{\partial L}{\partial b} = - \sum_{i=1}^n \alpha_i y_i = 0$$

This gives the condition:

$$\sum_{i=1}^n \alpha_i y_i = 0$$

# Support Vector Machine(SVM): Optimization

Now we substitute the expressions for  $\mathbf{w}$  and the constraint on  $\alpha_i$  into the Lagrangian.

The Lagrangian becomes:

$$L(\alpha) = \frac{1}{2} \left( \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i \right)^T \left( \sum_{j=1}^n \alpha_j y_j \mathbf{x}_j \right) - \sum_{i=1}^n \alpha_i$$

Expanding the quadratic term:

$$L(\alpha) = \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j - \sum_{i=1}^n \alpha_i$$

This is the **dual formulation** of the SVM optimization problem. It is a quadratic programming problem that only depends on the dot products of the input vectors  $\mathbf{x}_i^T \mathbf{x}_j$ , and not directly on  $\mathbf{w}$ .

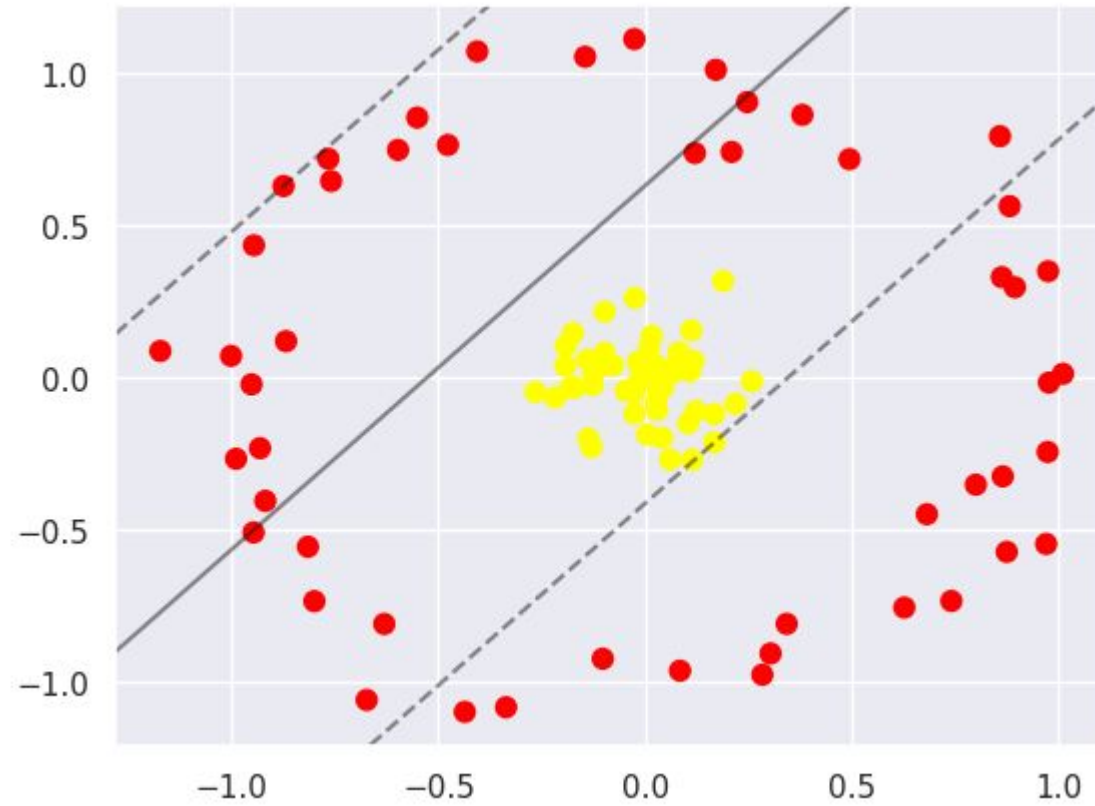
Thus, the dual optimization problem is:

$$\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$$

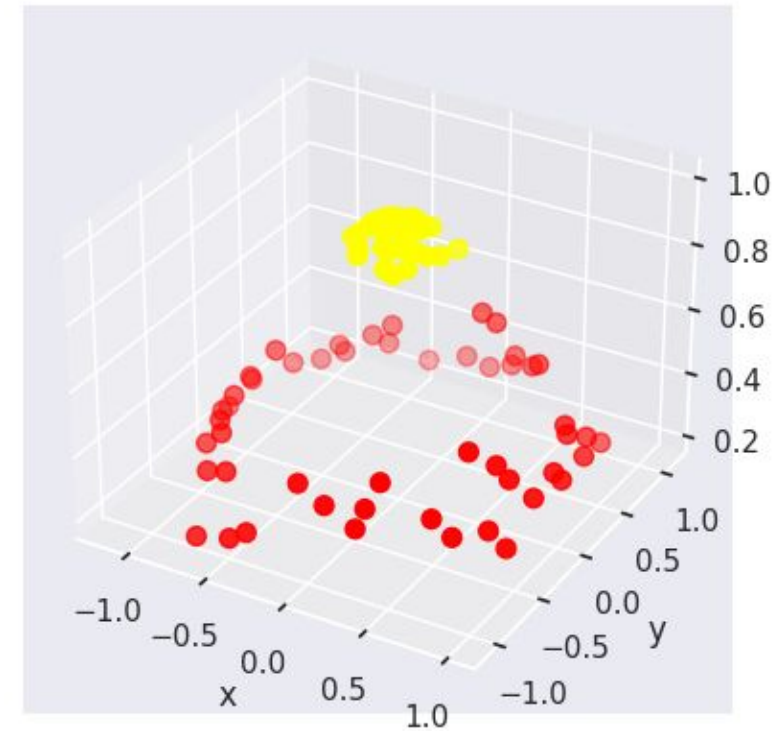
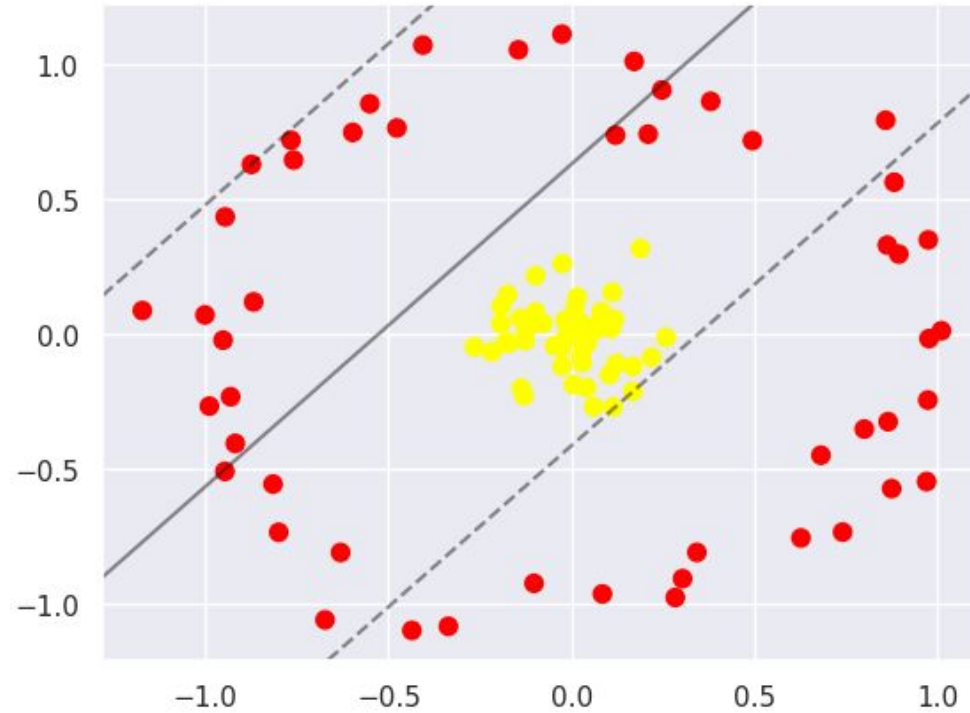
Subject to the constraints:

$$\sum_{i=1}^n \alpha_i y_i = 0, \quad \alpha_i \geq 0 \quad \forall i$$

# Support Vector Machine(SVM): Kernels

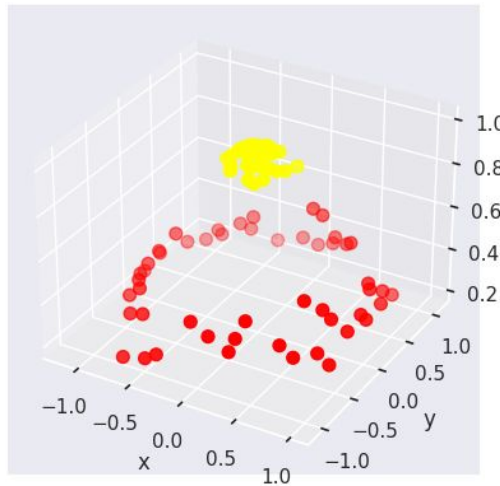
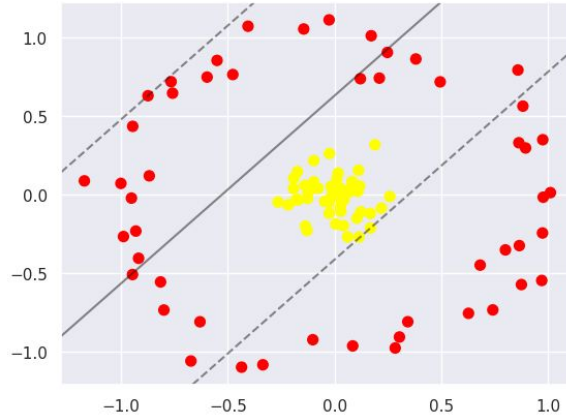


# Support Vector Machine(SVM): Kernels



$$(x_1, x_2) \in \mathbb{R}^2 \mapsto (x_1, x_2, e^{-(x_1^2 + x_2^2)}) \in \mathbb{R}^3$$

# Support Vector Machine(SVM): Kernels



We defined the mapping:

$$\phi(x) = (x_1, x_2, e^{-(x_1^2+x_2^2)}) \in \mathbb{R}^3$$

For two input vectors  $x = (x_1, x_2)$  and  $z = (z_1, z_2)$ , their dot product in the new space is:

$$\langle \phi(x), \phi(z) \rangle = x_1 z_1 + x_2 z_2 + e^{-(x_1^2+x_2^2)} e^{-(z_1^2+z_2^2)}$$

- The inner product in the bottom part depends on a function of the values in the R2 space.
- Instead of first creating the vectors in R3, we can define functions that **directly compute the inner products** in R3.
- We only need the results of the inner products, never use the vectors in R3 for anything else!

# Support Vector Machine(SVM): Kernel trick

## 6. Kernel Trick

Now comes the crucial part where the **kernel trick** is applied. The dual problem only involves the dot product  $\mathbf{x}_i^T \mathbf{x}_j$  between pairs of input vectors. This is where we introduce the **kernel function**  $K(\mathbf{x}_i, \mathbf{x}_j)$ , which replaces the dot product:

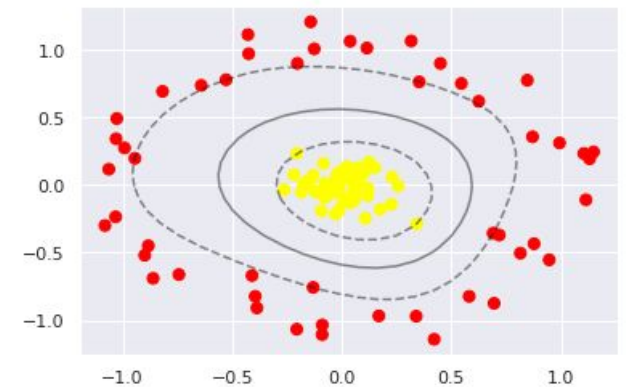
$$K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$$

Here,  $\phi(\cdot)$  is a mapping from the original space to a higher-dimensional feature space where the data might be linearly separable. However, thanks to the kernel trick, we don't need to compute  $\phi(\mathbf{x})$  explicitly — we only compute the kernel function  $K(\mathbf{x}_i, \mathbf{x}_j)$ , which gives us the dot product in the higher-dimensional space.

### Common Kernels:

- **Linear kernel:**  $K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$
- **Polynomial kernel:**  $K(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i^T \mathbf{x}_j + c)^d$
- **Radial Basis Function (RBF) or Gaussian kernel:**  $K(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{2\sigma^2}\right)$

By substituting the kernel function into the dual problem, the SVM can effectively operate in a high-dimensional space without explicitly mapping the data to that space.



# Support Vector Machine(SVM): Kernel trick

## 7. The Final Dual Problem with Kernels

The final dual optimization problem, incorporating the kernel, becomes:

$$\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j)$$

Subject to the same constraints:

$$\sum_{i=1}^n \alpha_i y_i = 0, \quad \alpha_i \geq 0 \quad \forall i$$

By solving this dual problem, we find the optimal values of the Lagrange multipliers  $\alpha_i$ . These multipliers determine the **support vectors** (the data points with non-zero  $\alpha_i$ ). The final decision function for classifying a new point  $\mathbf{x}$  is:

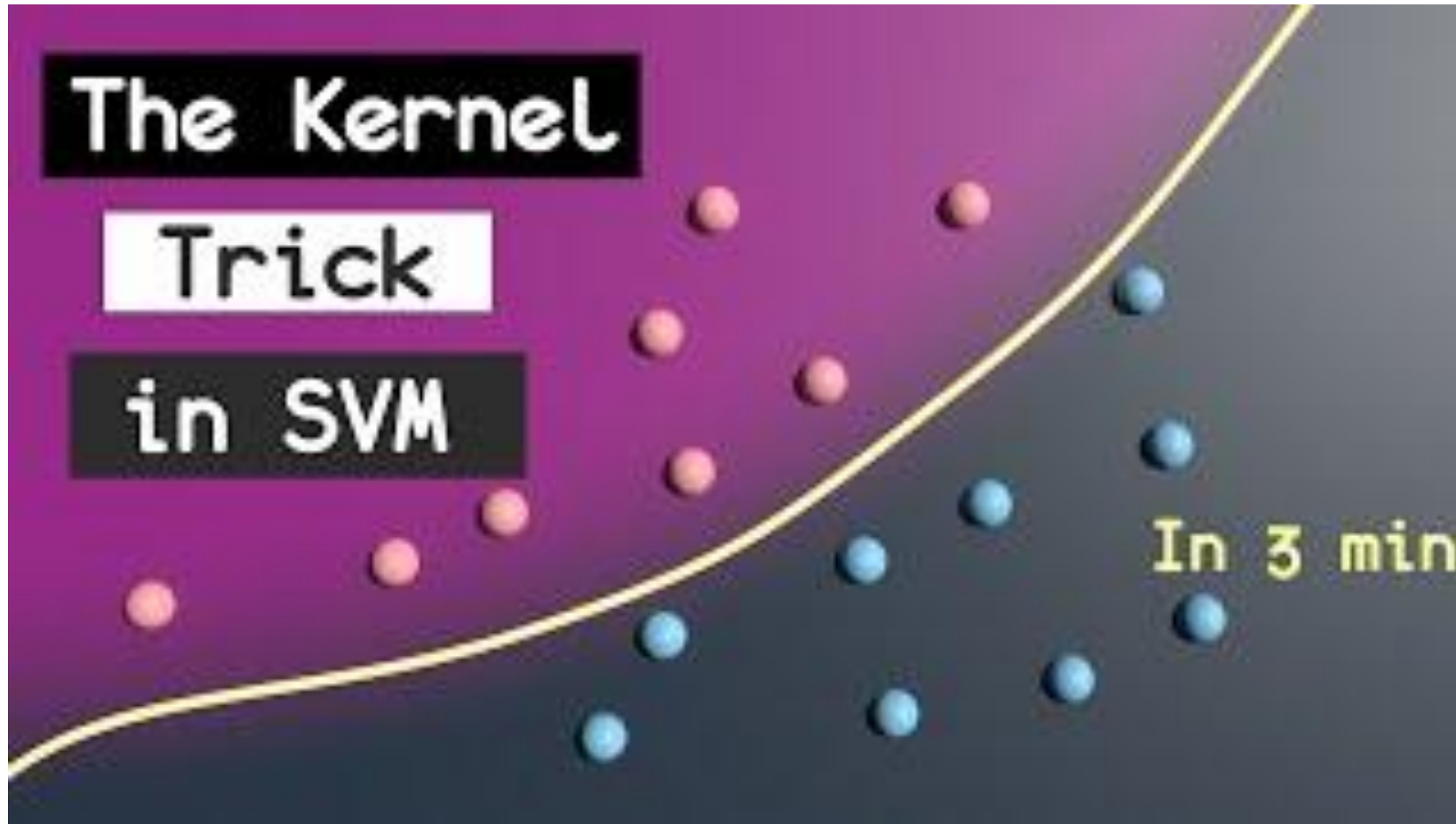
$$f(\mathbf{x}) = \sum_{i \in \text{SV}} \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b$$

Where "SV" denotes the set of support vectors. The kernel  $K(\mathbf{x}_i, \mathbf{x})$  computes the dot product in the high-dimensional feature space.

In summary:

- The dual formulation allows us to solve the SVM optimization problem efficiently.
- The kernel trick allows SVM to handle non-linearly separable data by implicitly mapping data into higher-dimensional spaces, where a linear separator is found.
- We never need to explicitly compute this mapping, as we only need the kernel function, which computes the dot product in the higher-dimensional space.

# Support Vector Machine(SVM): Kernels



# Support Vector Machine(SVM): Mercer's cond.

A fundamental property of a good kernel is that it must satisfy **Mercer's Theorem**, which ensures that the kernel corresponds to a valid dot product in some feature space. In other words, the kernel function must generate a **positive semi-definite** Gram matrix, also called the **kernel matrix**.

Given a set of data points  $\{x_1, x_2, \dots, x_n\}$ , the kernel matrix  $K$  is defined as:

$$K_{ij} = K(x_i, x_j)$$

The matrix  $K$  should be positive semi-definite, which means that for any vector  $\mathbf{v}$ , the quadratic form  $\mathbf{v}^T K \mathbf{v} \geq 0$ . This property ensures that the optimization problem in SVM remains convex, making the solution computationally feasible.

**Key takeaway:** Any valid kernel must generate a positive semi-definite kernel matrix.

# Support Vector Machine(SVM): Soft-Margin

## Primal Formulation for Soft-Margin SVM

The optimization problem for soft-margin SVM is modified as follows:

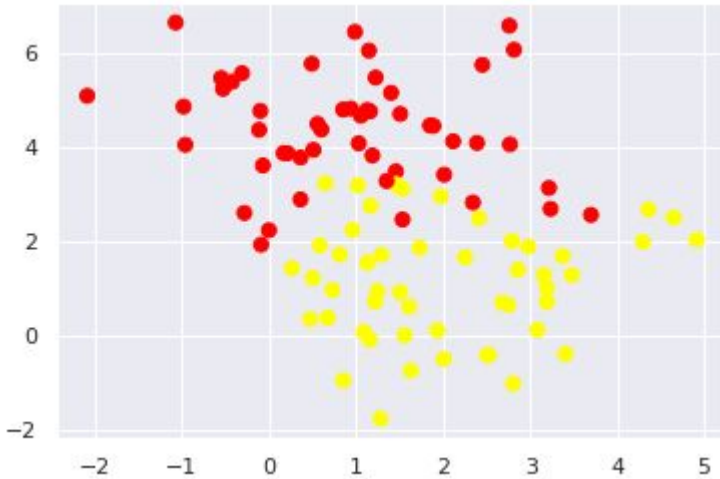
$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i$$

subject to:

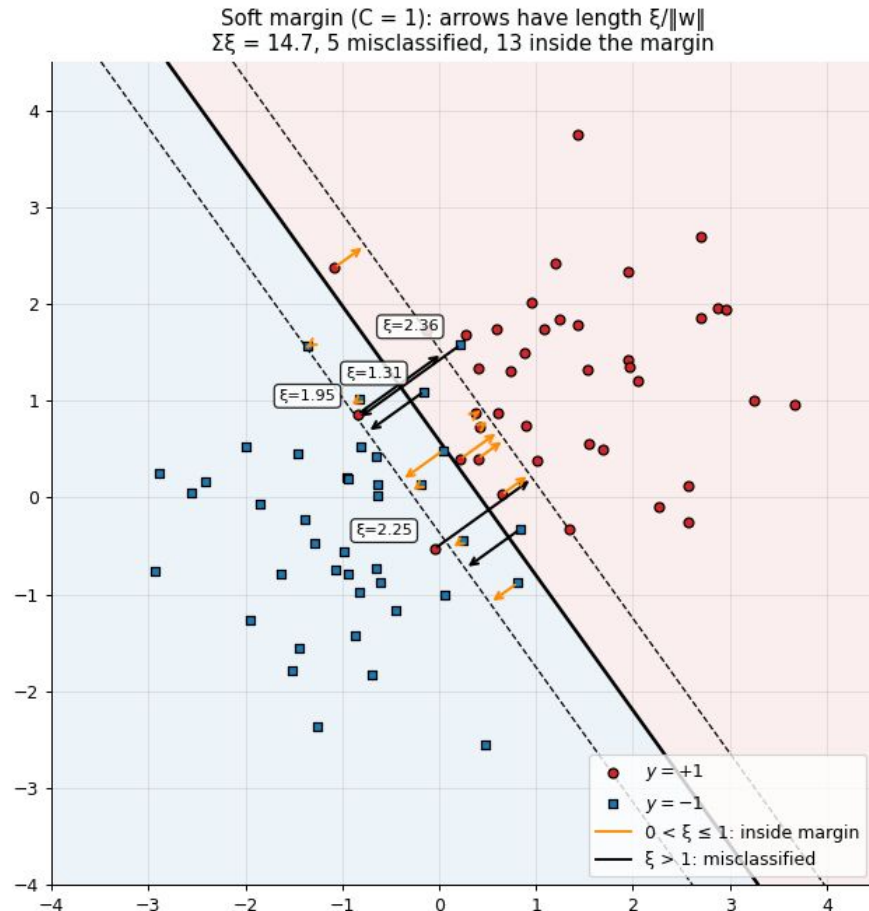
$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \quad \forall i$$

Where:

- $\mathbf{w}$  is the weight vector (as before).
- $b$  is the bias term (as before).
- $\xi_i$  are slack variables that allow some points to be on the wrong side of the margin or the decision boundary.
- $C$  is the **regularization parameter** that controls the trade-off between maximizing the margin and minimizing the misclassification error (or the magnitude of violations).



# Support Vector Machine(SVM): Soft-Margin



## Primal Formulation for Soft-Margin SVM

The optimization problem for soft-margin SVM is modified as follows:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i$$

subject to:

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \quad \forall i$$

Where:

- $\mathbf{w}$  is the weight vector (as before).
- $b$  is the bias term (as before).
- $\xi_i$  are slack variables that allow some points to be on the wrong side of the margin or the decision boundary.
- $C$  is the regularization parameter that controls the trade-off between maximizing the margin and minimizing the misclassification error (or the magnitude of violations).

# Support Vector Machine (SVM): Review

- Can make use of Kernels!
- Very fast classification!
- Good performance, even with “smaller” datasets. (only relies on support vectors)



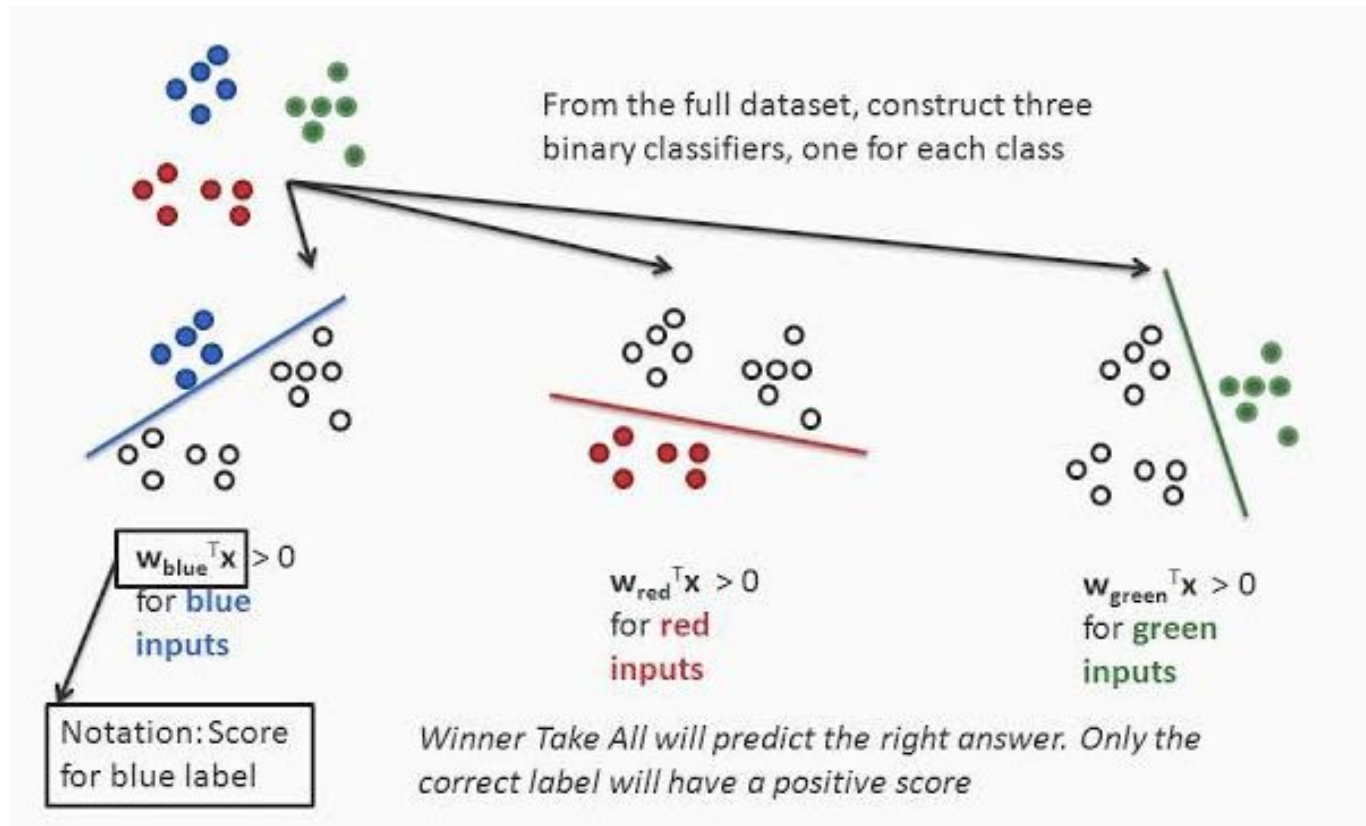
- The training can be slow.
- Only for 2 classes.



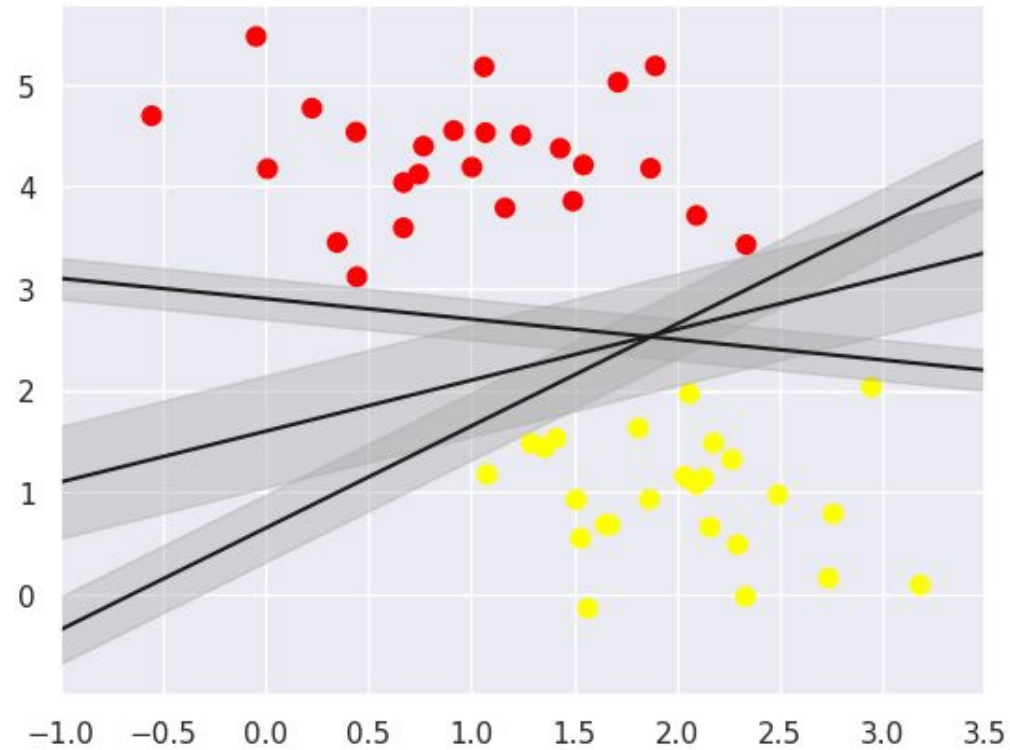
Interesting reads:

- Hinge loss and SVM with gradient descent:  
<https://towardsdatascience.com/support-vector-machine-introduction-to-machine-learning-algorithms-934a444fca47>
- Support Vector Regressor:  
<https://towardsdatascience.com/unlocking-the-true-power-of-support-vector-regression-847fd123a4a0>

# One-vs-All multiclass classification.

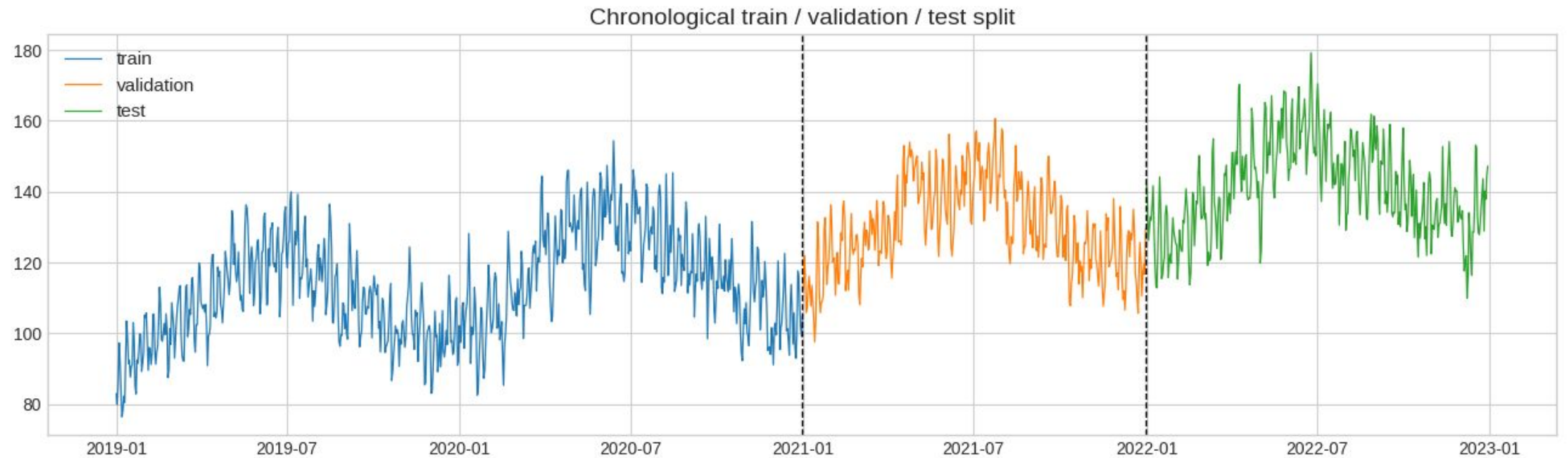


# Support Vector Machine(SVM): Lab

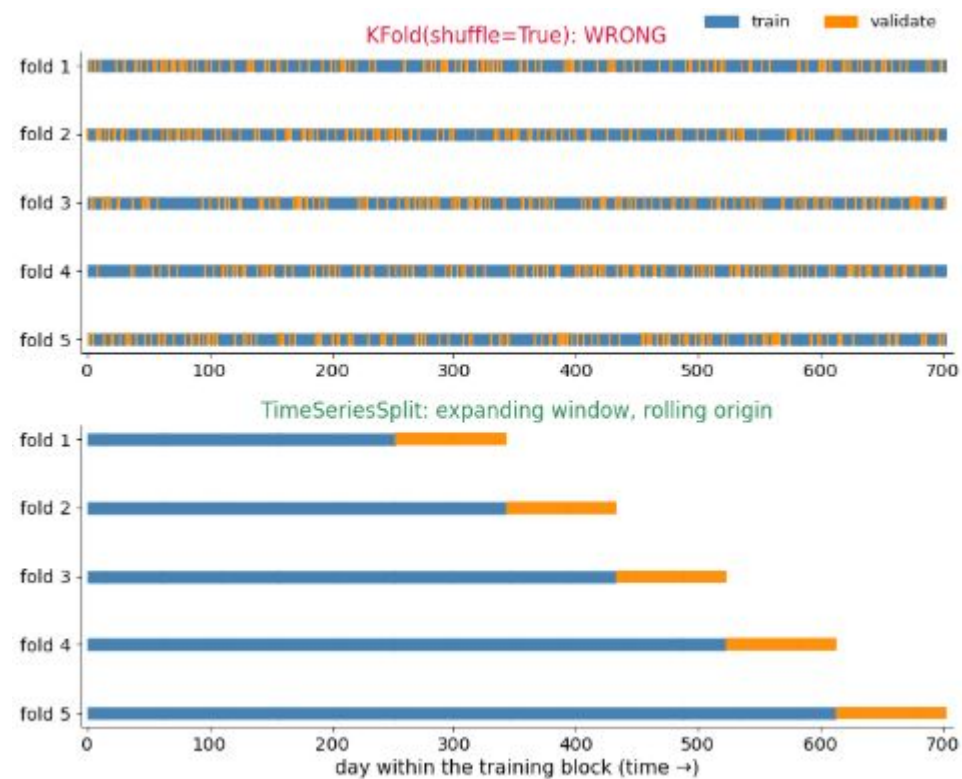


[Lab SVM](#)

# Example – Times series



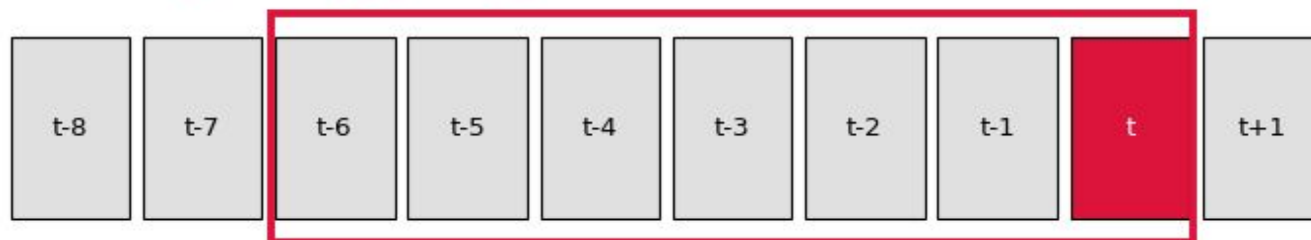
# Example – Times series



# Example – Times series

The look-ahead bug: one feature, two definitions

`y.rolling(7).mean()` window ends AT  $t$ : contains the target  $\times$

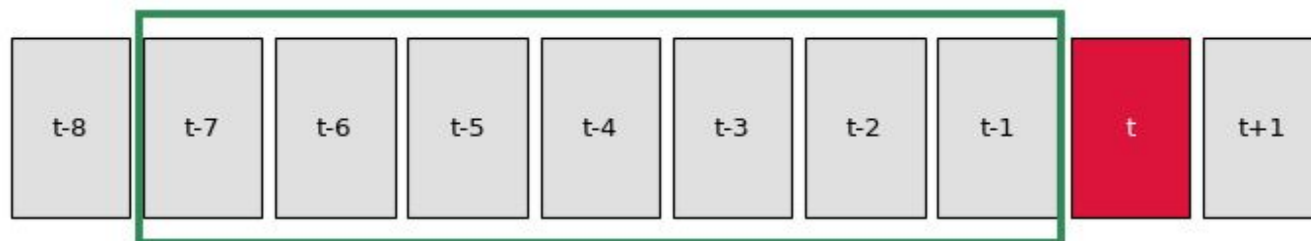


$$\tilde{y}_t^{(7)} = \frac{1}{7}(y_t + y_{t-1} + \dots + y_{t-6})$$

with lags 1-6 as features:  $y_t = 7\tilde{y}_t^{(7)} - (y_{t-1} + \dots + y_{t-6})$  exactly

validation: MAE = 0.0000,  $R^2 = 1.00000$  ← too good to be true

`y.shift(1).rolling(7).mean()` window ends at  $t-1$  ✓



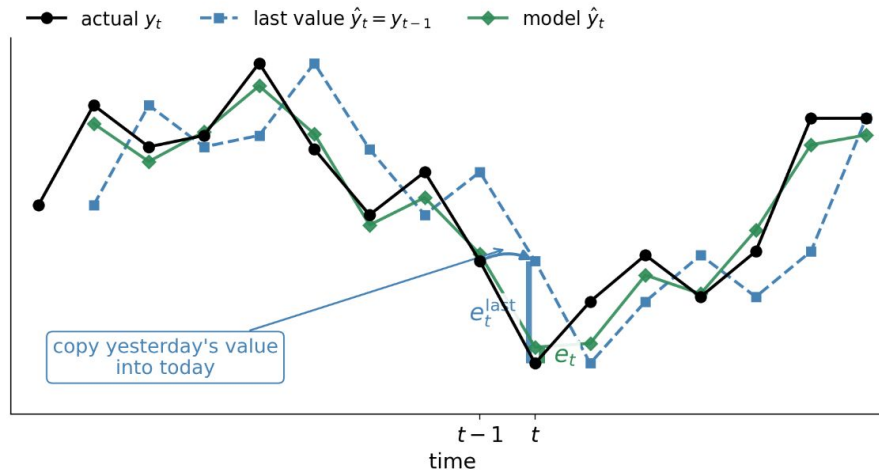
$$\tilde{y}_{t-1}^{(7)} = \frac{1}{7}(y_{t-1} + \dots + y_{t-7})$$
 only the past

validation: MAE = 5.00, MASE = 0.668,  $R^2 = 0.748$  ← the honest number

# Example – Times series

Forecast metrics: measure the error, then compare it with repeating the last value

The simplest honest forecast: repeat the last value



1 · How big are the errors?

error:  $e_t = y_t - \hat{y}_t$

MAE =  $\frac{1}{n} \sum_t |e_t|$  average miss, units of y

RMSE =  $\sqrt{\frac{1}{n} \sum_t e_t^2}$  units of y,   
 large misses weigh more (what least squares minimises)

Both say how big the error is,   
 but not whether it is good: that needs   
 a reference forecast to compare with.

2 · Is the model better than repeating the last value?

Score the model's errors against the errors of the last-value forecast:

$$R_{last}^2 = 1 - SSE_{model} / SSE_{last}$$

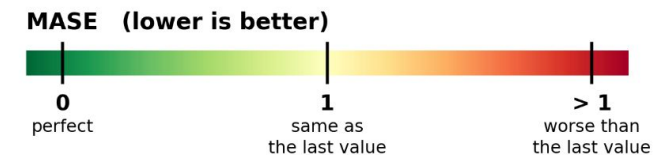
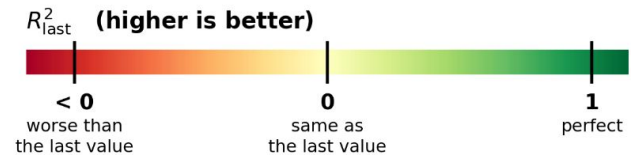
$$SSE_{model} = \sum_t (y_t - \hat{y}_t)^2, \quad SSE_{last} = \sum_t (y_t - y_{t-1})^2$$

$$MASE = MAE_{model} / MAE_{last}$$

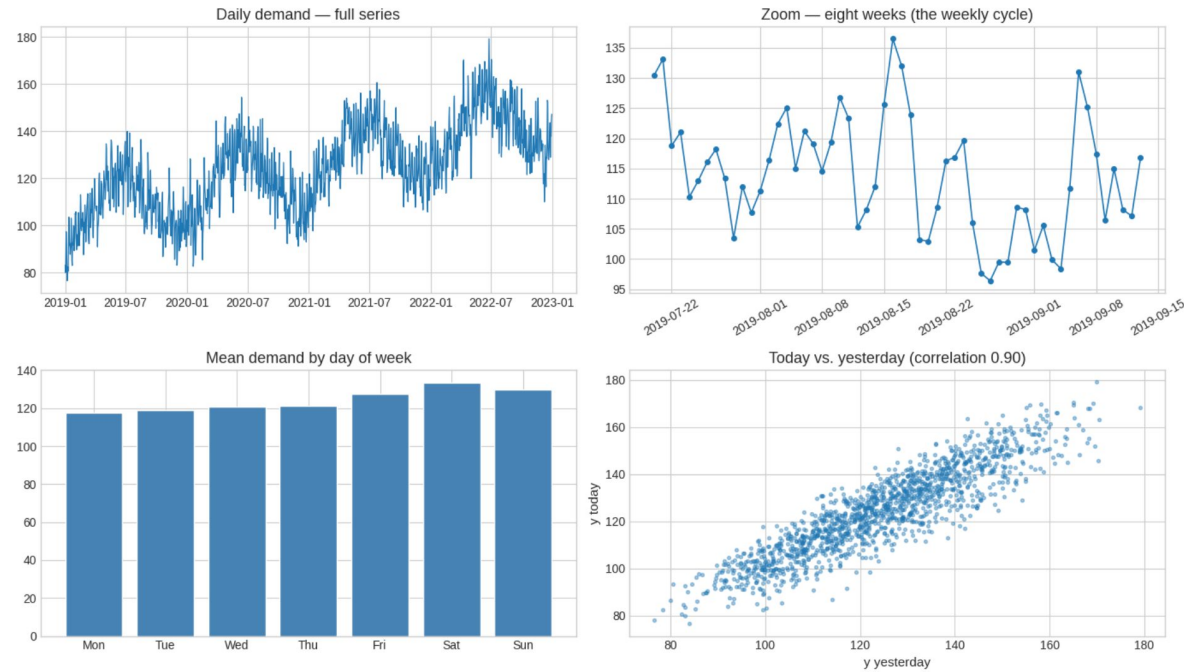
$$MAE_{model} = \frac{1}{n} \sum_t |y_t - \hat{y}_t|, \quad MAE_{last} = \frac{1}{n} \sum_t |y_t - y_{t-1}|$$

- the last value is free, always available, and hard to beat when today looks like yesterday (autocorrelation)
- compute the denominator once, on the training data, so the scale does not move when the test period changes
- seasonal data: use the value one season ago,  $y_{t-m}$ , instead

3 · How to read them



# Example – Times series



[Time series demo](#)

# Final Demo – Face Detection pipeline

## Application: A Face Detection Pipeline

This chapter has explored a number of the central concepts and algorithms of machine learning. But moving from these concepts to real-world application can be a challenge. Real-world datasets are noisy and heterogeneous, may have missing features, and data may be in a form that is difficult to map to a clean `[n_samples, n_features]` matrix. Before applying any of the methods discussed here, you must first extract these features from your data: there is no formula for how to do this that applies across all domains, and thus this is where you as a data scientist must exercise your own intuition and expertise.

One interesting and compelling application of machine learning is to images, and we have already seen a few examples of this where pixel-level features are used for classification. In the real world, data is rarely so uniform and simple pixels will not be suitable: this has led to a large

[Demo Faces](#)

# Machine Learning

---

Máster en Innovación Tecnológica (MINT)

*imanolgranada@gmail.com*